

CHAPTER 4

ALGORITHMS

Syllabus: Algorithms: Analysis, Asymptotic notation, Notions of space and time complexity, Worst- and average-case analysis; Design: Greedy approach, Dynamic programming, Divide-and-conquer; Tree and graph traversals, Connected components, Spanning trees, Shortest paths; Hashing, Sorting, Searching. Asymptotic analysis (best, worst, average cases) of time and space, Upper and lower bounds, Basic concepts of complexity classes – P, NP, NP-hard, NP-complete.

4.1 INTRODUCTION

In computer science, an algorithm provides a set of step-by-step instructions for solving various problems. The set of instructions should be unambiguous, finite and provide a clear termination point. An algorithm accepts some input values and produces outputs. The algorithm can be built in simple English language or in programming language. This subject helps in solving and analysing complex problems through logical thinking.

4.2 ALGORITHM

An algorithm is a well-defined procedure which takes a set of values as an input and produces a set of values as the output.

4.2.1 Properties of Algorithm

There are certain properties algorithms have:

1. An algorithm has zero or more inputs.
2. It should produce at least one output.
3. It should terminate within the finite time.
4. It should be deterministic (ambiguous).
5. It is generic to all programming languages.

4.2.2 Steps to Solve a Problem

1. Define the problem, which means what is input and what should be the output.
2. Identify the constraints which are to be satisfied.
3. Design the algorithm for a given problem.
4. Draw the flowchart.
5. Verify the process.
6. Implement the coding.
7. Perform time and space analysis.

4.2.3 Algorithm Analysis

If a problem has more than one solution, then the technique used to decide which solution is best is known as algorithm analysis.

4.2.3.1 A Posteriori and A Priori Analysis

Analysis is done on two parameters: time complexity and space complexity. Time complexity means time taken by an algorithm to solve a problem. Space complexity means memory space required by an algorithm to solve a problem. There are two types of analysis:

1. **A Posteriori Analysis:** It is a software- and hardware-dependent analysis. It keeps on changing from one system to other system. This type of analysis tells the exact time and space complexity, meaning how much an algorithm takes time and space for solving a problem on the given system.
2. **A Priori Analysis:** It is a software- and hardware-independent analysis. It is constant for all the systems. This type of analysis tells the approximate time and space complexity. It is the determination of the order of magnitude of a statement, meaning how many times a statement is executing.

4.2.3.2 Asymptotic Analysis

Suppose there are given two algorithms for a task, how do we find out which one is better?

This can be done by actually implementing both the algorithms and running them on the same computer with different input values. Then the comparison is made in terms of time. Algorithm that takes less time is considered better. But there are so many problems with this approach.

1. It might be possible that for certain inputs first algorithm performs better than second and for other set of inputs second algorithm performs better. So, the decision of choosing best among them becomes difficult.
2. With the change in operating machine, performance of algorithm varies. On one machine first algorithm can work better and on another machine, the second works better. This is another problem associated with this approach.

Above issues can be handled using asymptotic analysis techniques for analysing algorithms. In asymptotic analysis, analysis of algorithm is done in terms of input size. Actual running time is not computed, variation of time with input size is calculated.

Now, understand that how the issues with above technique are resolved by asymptotic analysis technique using the below example:

Example 4.1

Let us consider an example of a search problem (searching a given item) in a sorted array. One way to search is linear search (the order of growth is linear) and the other way is binary search. Suppose we have two machines with different processing speeds. We run linear search on the fast machine and binary search on slow machine. For small input size, the linear search will perform better. But as we increase the size of input, binary search will perform better. The reason behind this is, the order of growth of binary search is logarithmic and that of linear search is linear. So, by using this technique the issues of machine dependency are resolved.

4.2.4 Asymptotic Notation

Asymptotic notation is based on two assumptions, which hold in most of the cases and have their importance. It is very much important to understand the assumptions and limitations of asymptotic notations:

1. **Input size:** It is interesting to know how the running time of an algorithm grows for a large input size n .
2. **Constant:** The running time of an algorithm also depends on various constants. But for a large input n , the constant factors would be ignored.

For representing best-case, average-case, and worst-case complexity, there are three different notations as given in the following sub-sections.

4.2.4.1 Theta Notation (Θ)

Theta notation shows the tightest upper bound and the tightest lower bound to the given function (Fig. 4.1). It is used to define the average-case running time. For a given function $f(n)$, we denote $\Theta(g(n))$ as $\Theta(g(n)) = \{f(n) : \text{there exists positive constants } c_1 > 0, c_2 > 0 \text{ and } n_0 > 0 \text{ such that } 0 < c_1(g(n)) \leq f(n) \leq c_2(g(n)) \forall n \geq n_0\}$

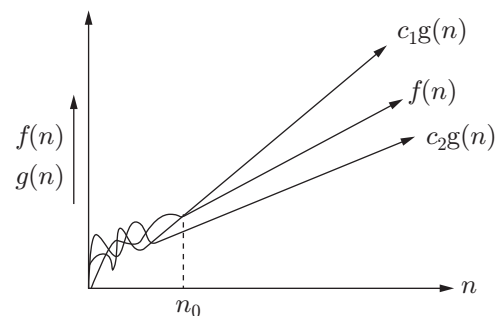


Figure 4.1 | Theta notation (Θ)

Problem 4.1: Consider the following functions:

$$f(n) = n$$

$$g(n) = \frac{n}{8}$$

For $f(n) = \Theta(g(n))$, find the values of c_1 , c_2 and n_0 .

Solution: From the definition of theta (Θ) notation, we have

$$0 < c_1(g(n)) \leq f(n) \leq c_2(g(n)) \quad \forall n \geq n_0$$

By putting $c_1 = 1, 2, 3$ to satisfy the above condition, we get

$$c_1(g(n)) \leq f(n)$$

$$1\left(\frac{n}{8}\right) \leq n$$

$$c_1 = 1$$

By putting $c_2 = 1, 2, 3$ to satisfy the above condition, we get

$$f(n) \leq c_2(g(n))$$

$$8\left(\frac{n}{8}\right) \leq n$$

$$c_2 = 8$$

By putting $n_0 = 1, 2, 3, \dots$, to satisfy the above condition, we get $n_0 = 1$.

4.2.4.2 Big-O Notation (O)

Big-O notation shows the upper bound to the given function (Fig. 4.2). It is used to define the worst-case running time. For a given function $f(n)$, we denote $O(g(n))$ as $O(g(n)) = \{f(n): \text{there exist positive constants } c > 0 \text{ and } n_0 > 0 \text{ such that } 0 < f(n) \leq c(g(n)) \forall n \geq n_0\}$

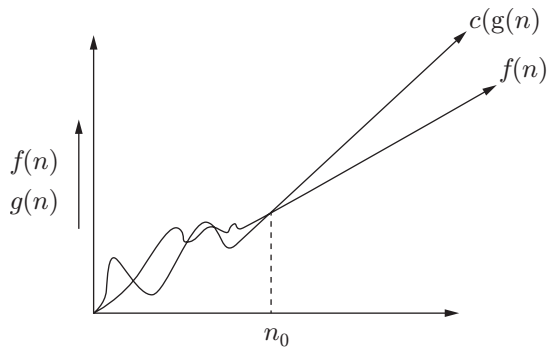


Figure 4.2 | Big-O notation (O).

Problem 4.2: Consider the following functions:

$$f(n) = n$$

$$g(n) = n$$

For $f(n) = O(g(n))$, find the values of c and n_0 .

Solution: From the definition of Big-O notation, we have

$$f(n) \leq c(g(n)) \quad \forall n \geq n_0$$

By putting $c = 1$ and $n = 1$, we get

$$n \leq c \cdot n \quad \forall n \geq n_0$$

The above condition is satisfied. So, $c = 1$ and $n_0 = 1$.

4.2.4.3 Omega Notation (Ω)

Omega notation shows the lower bound to the given function (Fig. 4.3). It is used to define the best-case running time. For a given function $f(n)$, we denote $\Omega(g(n))$ as $\Omega(g(n)) = \{f(n): \text{there exist positive constants } c > 0 \text{ and } n_0 > 0 \text{ such that } 0 < c(g(n)) \leq f(n) \forall n \geq n_0\}$.

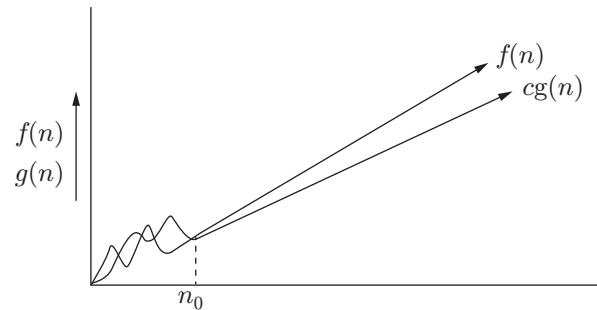


Figure 4.3 | Omega notation (Ω).

Problem 4.3: Consider the following functions:

$$f(n) = n^2$$

$$g(n) = n$$

For $f(n) = \Omega(g(n))$, find values of c and n_0 .

Solution: From the definition of Omega notation, we have

$$c(g(n)) \leq f(n) \quad \forall n \geq n_0$$

By putting $c = 1$ and $n = 1$, we get

$$c \cdot n \leq n^2 \quad \forall n \geq n_0$$

The above condition is satisfied. So, $c = 1$ and $n_0 = 1$.

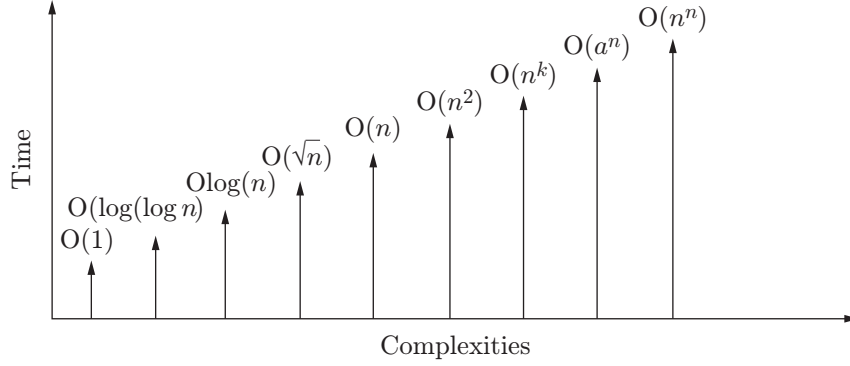


Figure 4.4 | Time complexities in increasing order.

4.2.4.4 Small-o Notation (o)

Small- o represents an upper bound which is not a tight upper bound. It is also used to define the worst-case running time. For a given function $f(n)$, we denote $o(g(n))$ as $o(g(n)) = \{f(n): \text{there exist positive constants } c > 0 \text{ and } n_0 > 0 \text{ such that } 0 < f(n) < c(g(n)) \forall n \geq n_0\}$.

4.2.4.5 Small Omega Notation (ω)

Small omega (ω) represents a lower bound which is not a tight lower bound. It is used to define the best-case running time. For a given function $f(n)$, we denote $\omega(g(n))$ as $\omega(g(n)) = \{f(n): \text{there exist positive constants } c > 0 \text{ and } n_0 > 0 \text{ such that } 0 < c(g(n)) < f(n) \forall n \geq n_0\}$.

4.2.4.6 Properties of Asymptotic Notations

1. **Transitivity:** All asymptotic notations show transitivity.

If $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$, then $f(n) = \Theta(h(n))$.

If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

If $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$, then $f(n) = \Omega(h(n))$.

If $f(n) = o(g(n))$ and $g(n) = o(h(n))$, then $f(n) = o(h(n))$.

If $f(n) = \omega(g(n))$ and $g(n) = \omega(h(n))$, then $f(n) = \omega(h(n))$.

2. **Reflexivity:** Only Theta, Big-O, and Omega notations show reflexivity. Small- o and small-omega do not show reflexive property.

$$f(n) = \Theta(f(n)) \quad f(n) = O(f(n)) \quad f(n) = \Omega(f(n))$$

3. **Symmetric:** Only Theta notation is symmetric.

$$f(n) = \Theta(g(n)) \text{ if and only if } g(n) = \Theta(f(n)).$$

4. **Transpose symmetry:**

$$f(n) = O(g(n)) \text{ if and only if } g(n) = \Omega(f(n))$$

$$f(n) = o(g(n)) \text{ if and only if } g(n) = \omega(f(n))$$

4.2.4.7 Types of Complexities

1. Constant time complexity: $O(1)$
2. Logarithmic time complexity: $O(\log(\log n))$, $O(\sqrt{\log n})$ and $O(\log n)$
3. Linear time complexity: $O(\sqrt{n})$ and $O(n)$
4. Polynomial time complexity: $O(n^k)$, where k is a constant and is > 1
5. Exponential time complexity: $O(a^n)$, where $a > 1$

The complexities in increasing order are represented in Fig. 4.4.

4.2.5 Recurrence Relations

Recurrence relations are recursive definitions of mathematical functions or sequences. For example, the recurrence relation defines the famous Fibonacci sequence 1, 1, 2, 3, 5, 8, 13, ...:

$$f(n) = f(n-1) + f(n-2)$$

$$f(1) = 1$$

$$f(0) = 1$$

Given a function defined by a recurrence relation, we want to find a 'closed form' of the function. In other words, we would like to eliminate recursion from the function definition. There are several techniques for solving recurrence relations, and these are discussed in the following sub-sections.

4.2.5.1 Iteration Method

This method is accurate but involves a lot of algebraic expressions which are difficult to keep track of. It can get very challenging for further complicated recurrence relations.

Problem 4.4: We have $T(n) = 1$ if $n = 1$

$$T(n) = 4T\left(\frac{n}{2}\right) + n \quad \text{if } n > 1$$

$$T(n) = 4T\left(\frac{n}{2}\right) + n = 4\left(4T\left(\frac{n}{4}\right) + \frac{n}{2}\right) + n$$

$$\begin{aligned}
&= 4 \left(4 \left(4T\left(\frac{n}{8}\right) + \frac{n}{4} \right) + \frac{n}{2} \right) + n \\
&= 64T\left(\frac{n}{8}\right) + 4n + 2n + n \\
&= n + 2n + 4n + 64T\left(\frac{n}{8}\right) \\
&= n + 2n + 4n + \dots + 2^j n + \dots + 4^i T\left(\frac{n}{2^i}\right)
\end{aligned}$$

Find the time complexity when $(n/2^i) = 1$.

Solution:

We have $i = \log n$. So, the time complexity of n can be represented as

$$T(n) = n + 2n + 4n + 8n + 2^i n + 4^{\log n} O(1)$$

We get
$$n \left(\sum_{i=0}^{\log(n-1)} 2^i \right) + 4^{\log n} O(1)$$

We Know that
$$\sum_{k=0}^m x^k = \frac{x^{m+1} - 1}{x - 1}$$

So,
$$n \left(\frac{2^{\log(n-1)+1} - 1}{2 - 1} \right) + 4^{\log n} O(1)$$

Solving, we get

$$\begin{aligned}
n2^{\log n} - n + 4^{\log n} O(1) &\Rightarrow n^2 - n + n^{\log 4} O(1) \\
&\Rightarrow 2n^2 O(1) - n \Rightarrow O(n^2)
\end{aligned}$$

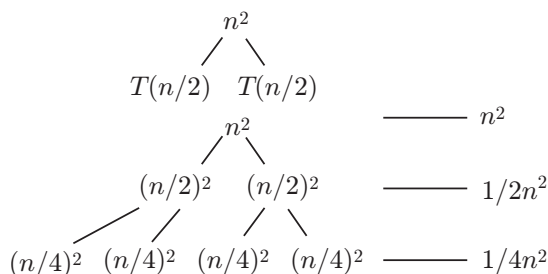
So, the running time complexity is given as: $T(n) = O(n^2)$.

4.2.5.2 Recursion Tree

A recursion tree is useful for visualizing what happens when a recurrence is iterated. It represents a tree of recursive calls and the amount of work done at each call is shown with the help of a diagram.

Example 4.2

Consider the recurrence $T(n) = 2T(n/2) + n^2$.



How deep does the tree go?

We stop at the leaf, and we know we are at a leaf when we have a problem of size 1:

$$1 = \left(\frac{n}{2^i}\right)^2 \Rightarrow n^2 = 2^{2i} \Rightarrow n = 2^i \Rightarrow i = \log n$$

The amount of work done is given as

$$\sum_{i=0}^{\log n} \frac{n^i}{2} = \Theta n^2$$

This is geometrically decreasing in size, so it would not get any bigger than n^2 .

4.2.5.3 Master Theorem

Master theorem provides a method to solve various recurrence relations. Although all the relations can't be solved using this method, but it is one of the useful method of solving complex relations of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where a and b are constants. f is a function of n . this recursive algorithm breaks up the input into sub-problems of size n/b each.

Three cases of master theorem are as follows:

- Case 1:** If $f(n)$ is $O(n^{\log_b a - \epsilon})$ then $T(n)$ is $\Theta(n^{\log_b a})$. The rate of growth at leaves is faster than f , more work is done on leaves.
- Case 2:** If $f(n)$ is $\Theta(n^{\log_b a})$ then $T(n)$ as $\Theta(n^{\log_b a} \log n)$. The rate of growth of leaves is same as f . So, the same amount of work is done at every level of tree. The leaves grow at the same rate as f , so the same order of work is done at every level of the tree. The tree has $O(\log n)$ times the work done on one level, yielding $T(n)$ as $\Theta(n^{\log_b a} \log n)$.
- Case 3:** $f(n)$ is $\Omega(n^{\log_b a + \epsilon})$. Here f grows faster than the number of leaves, which means that asymptotically the total amount of work is dominated by the work done at the root node. For the upper bound, we also need an extra smoothness condition on f in this case, namely, that $af(n/b) \leq cf(n)$ for some constant $c < 1$ and large n . In this case, $T(n)$ is $\Theta(f(n))$.

Example 4.3

Consider the following recurrence:

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

In the given relation, $a = 4$ and $b = 2$.

For this recurrence, there are $a = 4$ sub-problems, each dividing the input by $b = 2$, and the work done on each call is $f(n) = n$. Thus, $n^{\log_b a}$ is n^2 and $f(n)$ is $O(n^{2-\epsilon})$ for $\epsilon = 1$, and Case 1 applies. Thus, $T(n)$ is $\Theta(n^2)$.

Example 4.4

Consider the following recurrence:

$$T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

For this recurrence, there are again $a = 4$ sub-problems, each dividing the input by $b = 2$; but now the work done on each call is $f(n) = n^2$. Again, $n^{\log_b a}$ is n^2 and $f(n)$ is thus $\Theta(n^2)$, so Case 2 applies. Thus, $T(n)$ is $\Theta(n^2 \log n)$. Note that increasing the work on each recursive call from linear to quadratic has increased the overall asymptotic running time by only a logarithmic factor.

Example 4.5

Consider the following recurrence:

$$T(n) = 4T\left(\frac{n}{2}\right) + n^3$$

For this recurrence, there are again $a = 4$ sub-problems, each dividing the input by $b = 2$, but now the work done on each call is $f(n) = n^3$. Again, $n^{\log_b a}$ is n^2 and $f(n)$ is thus $\Omega(n^{2+\epsilon})$ for $\epsilon = 1$. Moreover, $4(n/2)^3 \leq kn^3$ for $k = 1/2$, so Case 3 applies. Thus, $T(n)$ is $\Theta(n^3)$.

4.3 HASHING

Hashing is technique that transforms a variable length input to a fixed length output. This technique is used for indexing, which helps in constant time search for insertion, deletion and information retrieval.

4.3.1 Hash Table

A hash table is a data structure used for hashing. It is an associative array that contains keys with values. In an array, indices are required to be integer, but a hash table allows floating point numbers, strings, array or structures as keys. Hash tables provide efficient lookup operation, that is, whenever a key is given, hash table provides associated value in almost constant time complexity. This transformation is done using hash function.

4.3.2 Hashing Functions

A hashing function is a transformation function, coded to return a value based on key. If h is a hash function that takes key k as parameter, will compute the index at which value is placed. If h is a hash function that takes key as a parameter, it will compute index where value

is placed. It is important to note that hash function returns the same index value every time for same key.

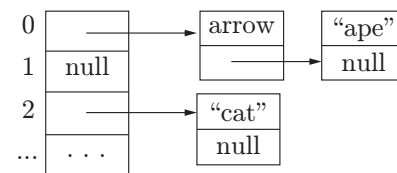
4.3.3 Collisions

Collision occurs when one key value stores more than one value. Hash function produce the same position for two values. Hash table cannot ensure to have single value corresponding to a key. This is because a hash table has fixed number of keys. Any new entry in hash table, after all the slots are filled, causes collision. However, good hash tables use some mechanism to avoid the situations of collisions or to handle them efficiently.

4.3.3.1 Separate Chaining

1. If more than one item is assigned the same hash index, “chain” them together.
2. In hash table, each position act as bucket that can store multiple data items.
3. There are the following two implementations:
 - Each bucket is itself an array: The disadvantages of bucket as array are
 - (a) Memory wastage
 - (b) Problem of overflow occurs when we try to add new item to a full bucket.
 - Each bucket is a linked list: The disadvantage of bucket as linked list is

Pointers require more memory.



4.3.3.2 Open Addressing

1. Collision occurs if the hash function produces the same position occupied by another item. In this case, another open position is searched.

Example: Hash code produced for ‘White’ is 22. But position 22 is already occupied, so it is placed at position 23.

2. There are three ways of finding an open position. This process is known as **probing**. To search any item in hash table, probing may be required.

Example: When item “white” is searched, first we look for it at position 22. If it is not found there, we move to next location. The search will be stopped only when an empty position is encountered.

0	Arrow
1	
2	Cat
3	
4	Eye
5	
...	
22	When
23	White
24	Yellow
25	Zero

4.3.3.3 Linear Probing

In case of linear probing the probe sequence is $h(\text{key})$, $h(\text{key}) + 1$, $h(\text{key}) + 2$, ..., uptill required.

Examples: The item 'Ape' ($h = 0$) will be placed at position 1, because position 0 is already occupied. then 'Ball' ($h = 1$): try to place it at position 1, $1+1$, $1+2$. Position 3 is open so it will be placed here.

Advantage: If open position is available, linear probing will definitely find it.

Disadvantage: If there are clusters of filled positions, then length of subsequent probes increases. Probe length is the number of positions considered during a probe.

0	Arrow
1	Ape
2	Cat
3	Ball
4	Eye
5	
...	
22	When
23	White
24	Yellow
25	Zero

4.3.3.4 Quadratic Probing

The probe sequence is

$h(\text{key})$, $h(\text{key}) + 1$, $h(\text{key}) + 4$, $h(\text{key}) + 9$, ..., wrapping around as necessary

The offsets are perfect squares: $h + 12$, $h + 22$, $h + 32$,

Examples:

- 'Ape' ($h = 0$): try 0, $0 + 1$ – open!
- 'Ball' ($h = 1$): try 1, $1 + 1$, $1 + 4$, $1 + 9$ – open!

Advantage: It reduces clustering.

0	Arrow
1	Ape
2	Cat
3	
4	Eye
5	Ball
...	
22	When
23	White
24	Yellow
25	Zero

Disadvantage: It may fail to find an existing open position. For example, let table size = 10, and x denote the occupied blocks. Trying to insert a key with $h(\text{key}) = 0$ offsets the probe sequence shown in *italic*.

0	x		5	x	25
1	x	1 81	6	x	16 36
2			7		
3			8		
4	x	4 64	9	x	9 49

4.3.3.5 Double Hashing

It uses the following two hash functions:

- h_1 computes the hash code.
- h_2 computes the increment for probing.

The probe sequence is: h_1 , $h_1 + h_2$, $h_1 + 2 \times h_2$,

Examples:

1. h_1 = our previous h
2. h_2 = number of characters in the string
3. 'Ape' ($h_1 = 0$, $h_2 = 3$): try 0, 0 + 3 – open!
4. 'Ball' ($h_1 = 1$, $h_2 = 4$): try 1 – open!

Advantages:

1. It combines the good features of linear and quadratic probing:
2. It reduces clustering
3. It will find an open position if there is one, provided the table size is a prime number

0	Arrow
1	Ball
2	Cat
3	Ape
4	Eye
5	
...	
22	When
23	White
24	Yellow
25	Zero

4.3.3.6 Removing Items Under Open Addressing**Consider the following scenario:**

Using linear probing:

- (a) Insert 'Ape' ($h = 0$): Try 0, 0 + 1 – open!
 - (b) Insert 'Ball' ($h = 1$): Try 1, 1 + 1, 1 + 2 – open!
 - (c) Remove 'Ape'.
 - (d) Search for 'Ape': Try 0, 0 + 1 – no item.
 - (e) Search for 'Ball': Try 1 – no item, but 'Ball' is further down in the table.
1. On removing an item from a position, leave a special value in that position to indicate that an item was removed.
 2. There are three types of positions: occupied, empty and removed.
 3. Stop probing on encountering an empty position, but not when encountering a removed position.
 4. Insert items in either empty or removed positions.

0	Arrow
1	1
2	Cat
3	Ball
4	Eye
5	
...	
22	When
23	White
24	Yellow
25	Zero

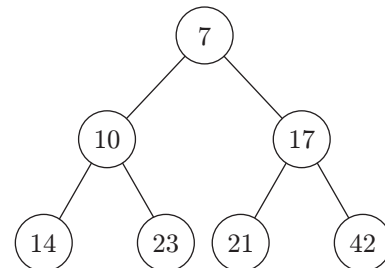
4.4 BINARY HEAP

A binary heap is a binary tree with two additional constraints:

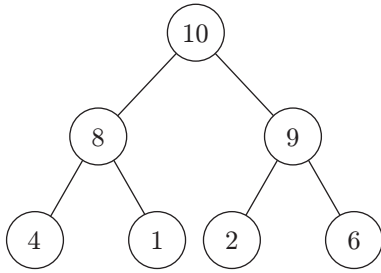
1. Binary heap should satisfy the property of complete binary tree.
2. It should satisfy the heap ordering property, that is, all the nodes should be either greater than or equal to or less than or equal to its children.

According to the heap ordering property, binary heaps are of two types:

1. **Min-Heap Tree:** In Min heap ordering, the value of each node is less or equal to its child node. Minimum element of the tree will be the root element. The largest element exists in the last level. The tree should be an almost complete binary tree.



2. **Max-Heap Tree:** In max heap, each node in the tree will be greater or equal to its child node. The maximum element of tree will be the root element. The smallest element exists in the last level. The tree should be an almost complete binary tree.



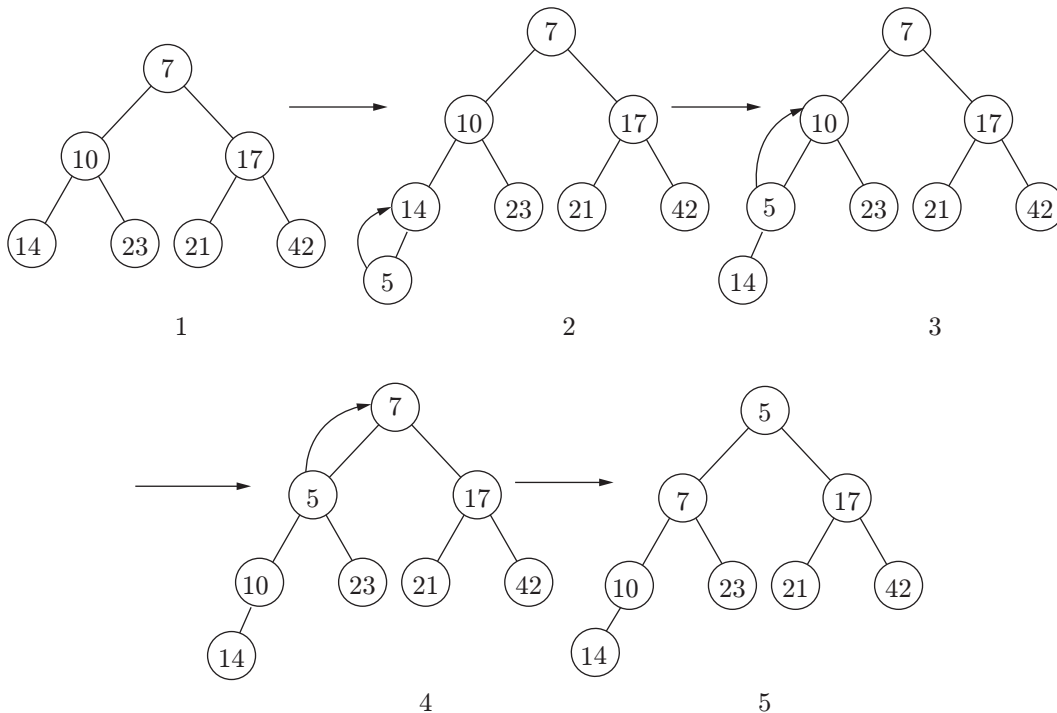
4.4.1 Insertion in Min-Heap Tree

The following are the steps to follow for insertion in the min-heap tree:

1. Add the new element to the last level of heap.
2. Compare the new element with its parent node.
3. If the order is correct then stop, otherwise swap the element with its parent.
4. Perform steps 2 and 3 until a new element is at position.

Example 4.6

1. Insert the given value (5) in the given min-heap tree.
2. Element 5 is inserted at position of left child of node 14. To satisfy the property of min-heap, swap (5, 14).
3. To satisfy the property of min-heap, swap (5, 10).
4. To satisfy the property of min-heap, swap (5, 7).
5. Now, element 5 is in its right position. This is the final tree.



4.4.2 Deletion in Min-Heap Tree

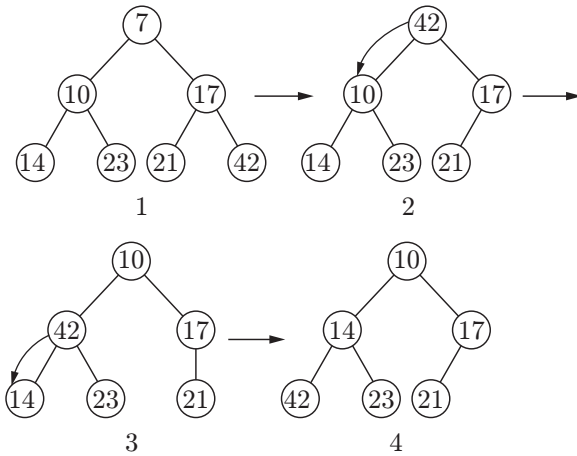
The following are the steps to follow for deletion in the min-heap tree:

1. Root of the heap is replaced with the element at the last level.
2. New root element is compared with its children; if it maintains ordering then stop.
3. Otherwise, swap the root with one of its children and return to previous step. (Smaller element is swapped for min-heap and larger child for max-heap)

Example 4.7

1. Deletion always occurs at the root level, so element 7 will be deleted. Before deletion of element 7 at the root, the element (42) is swapped to the root node from the extreme right-hand side of the last level and then 7 is deleted.
2. To maintain the min-heap property of the tree, swap 42 and min(left-child, right-child), which means swap 42 with 10.
3. To maintain the min-heap property of the tree, swap 42 and min(left-child, right-child), which means swap 42 with 14.

4. This represents the final tree, which is in the min-heap property.



4.4.3 Time Complexity

1. The worst-case running time for insertion is $O(\log n)$ because at most $(\log n)$ swaps are required to stable the min-heap property of the tree.
2. The worst-case running time for deletion is $O(\log n)$ because at most $(\log n)$ swaps are required to stable the min-heap property of the tree.

Some important points of binary heap are as follows:

1. Because of its structure, a heap with height k will have between 2^k and $2^{k+1} - 1$ elements. Therefore, a heap with n elements will have height $= \log_2 n$.
2. In heap data structure, element with highest or lowest priority is stored as root element. Heap is not considered as sorted structure, rather it is partially ordered. There is no relation among nodes at a given level.

Heap is very useful data structure for implementing priority queues. Element with highest priority can be easily removed.

3. Since a binary heap is a complete binary tree, it can be easily represented as an array, and an array-based representation is space efficient. If the parent node is stored at index I , the left child can be calculated by $2 \times I + 1$ and right child by $2 \times I + 2$.

4.5 SEARCHING AND SORTING

Searching and sorting are the most common activities performed by the computer. These operations are the most important part for managing the data.

1. **Searching** is the algorithmic process of finding an item in a collection of items. The search result is either true or false depending on whether or not the item is present or not in the list.
2. **Sorting** is ordering a list of objects according to a given condition or property. For example, ordering

a set of numbers in an ascending or descending order.

3. In **internal sorting**, the data which is to be sorted fits into the main memory. Internal sorting has an advantage of the random access nature of the main memory.
4. In **external sorting**, large amount of data is sorted. This large amount of data cannot be fit in main memory, so the data is placed in auxiliary memory. When sorting has to perform, small chunks of data are brought into main memory, sorted and placed back to a temporary file. At last all the sub-files are merged in one large file.
5. In **stable sorting**, the order of equal elements is preserved in a sorted list. Some of the sorting algorithms that show characteristics of stable sort are insertion sort, bubble sort, merge sort, etc.; whereas heap sort, quick sort, etc., are not stable sorts.
6. **In-place sorting:** In this, the input is transformed using a data structure with a small, constant amount of extra storage space. The input is usually overwritten by the output as the algorithm executes. Quicksort is commonly described as an in-place algorithm.
7. **Measurement of time complexity:** The time complexity of a sorting algorithm is measured by the number of critical operations performed. Examples of critical operations are as follows:
 - Key comparisons
 - Movements of records
 - Interchanges of two records
8. We use asymptotic analysis to denote the time efficiency of an algorithm. Efficiency of an algorithm also depends on the type of data for input, so we define best-, worst- and average-case efficiencies.

4.5.1 Linear Search

Linear search is a sequential way of finding an element in a given list. Linear search is a special case of brute force search. Its worst case is proportional to the number of elements in list.

4.5.1.1 Pseudocode for Linear Search

```

LINEAR_SEARCH (A[ ], x)
1. i = 1
2. while (A[i] == x and i < n)
3. i = i + 1
4. if (A[i] == x)
5. return i
6. else return 0
  
```

4.5.1.2 Time Complexity

From the analysis of above pseudocode, it can be observed that with the growth of input list, average and worst case complexities also grow linearly. In general, if there are n elements in the list, the worst-case requires n

comparisons. So, the complexity of linear search can be expressed in terms of some linear function.

So, running time $T(n) = O(n)$.

4.5.2 Binary Search

Binary search relies on a divide-and-conquer strategy to find an element within an already sorted list. It compares the element to be searched with the middle element of array. If the element is equal to the middle element, the search stops here and the position of element is returned. If it is less or greater than the middle element, then the values of sub-array are adjusted accordingly. The algorithm returns a unique value, that is, position of the element in the list, if the element is present.

4.5.2.1 Pseudocode for Binary Search

```
BINARY_SEARCH (A, value, left, right)
1. if right < left
2. return 'not found'
3. mid = floor((right-left)/2)+left
4. if A[mid] = value
5. return mid
6. if value < A[mid]
7. return BINARY_SEARCH(A, value, left, mid-1)
8. else
9. return BINARY_SEARCH(A, value, mid+1, right)
```

4.5.2.2 Time Complexity

The running time of the binary search is $\Theta(\log n)$.

4.5.3 Bubble Sort

Bubble sort algorithm is a comparison-based algorithm. It compares each element in the list with the next element. Elements are swapped if required. In every pass, one element comes to its position. This process repeats until a pass is made without disturbing any of the elements from their position.

4.5.3.1 Pseudocode for Bubble sort

```
BUBBLE_SORT (A)
1. for i ← 1 to n-1
2. for j ← 1 to n-1
3. If (A(j) > A(j+1))
4. Temp = A(j)
5. A(j) = A(j+1)
6. A(j+1) = Temp
```

Example 4.8

Sort the given elements (7, 5, 2, 4, 3 and 9) by using bubble sort.

After the first pass, we get

7, 5, 2, 4, 3, 9

5, 7, 2, 4, 3, 9

5, 2, 7, 4, 3, 9

5, 2, 4, 7, 3, 9

5, 2, 4, 3, 7, 9

After the first pass, the first largest element (9) is on the top of array. So we have

5, 2, 4, 3, 7, 9

After the second pass, we get

2, 4, 3, 5, 7, 9

After the third pass, we get

2, 3, 4, 5, 7, 9

After the fourth pass, we get

2, 3, 4, 5, 7, 9

After the fifth pass, we get

2, 3, 4, 5, 7, 9

The sorted list is 2, 3, 4, 5, 7, 9

4.5.3.2 Time Complexity

It can be seen from the above example that in the bubble sort, each element passes one at a time and is placed in its correct position.

So, the number of passes = (number of elements – 1) = $(n - 1)$.

Cost per pass = $O(n)$.

Total cost = $O(n)(n - 1)$.

So, the worst-case runtime complexity is $O(n^2)$.

The average-case and best-case runtime complexity is $O(n^2)$.

4.5.4 Selection Sort

The selection sort first selects the smallest element from the unsorted list. Then that element is swapped with the first element in the list. In the next step, the size of the list is reduced by one because one element is present at its position. Next, the smallest element is swapped with the second element in list, and so on.

4.5.4.1 Pseudocode for Selection Sort

```
SELECTION_SORT (A)
1. for j ← 1 to n-1
2. smallest ← j
3. for i ← j + 1 to n
4. if A[ i ] < A[ smallest]
5. smallest ← i
6. Exchange A[ j ] ↔ A[ smallest]
```

Example 4.9

Sort the given elements (29, 64, 73, 34 and 20) by using selection sort.

We have

29, 64, 73, 34, 20

After the first pass, we get

20, 64, 73, 34, 29

After the second pass, we get

20, 29, 73, 34, 64

After the third pass, we get

20, 29, 34, 73, 64

After the fourth pass, we get

20, 29, 34, 64, 73

After the fifth pass, we get

20, 29, 34, 64, 73

The sorted list is 20, 29, 34, 64, 73.

4.5.4.2 Time Complexity

Number of passes = (Number of elements – 1)

Cost per pass = Swap + Comparisons = $1 + O(n)$

Total cost = $(n - 1)[1 + O(n)]$

So, the worst-case runtime complexity is $O(n^2)$.

The average-case and best-case runtime complexity is $O(n^2)$.

4.5.5 Insertion Sort

The insertion sort inserts each element into its proper position. It chooses one element, inserts it to its position and shifts the rest of the list by one location. This process is repeated until no input element remains. Step-by-step procedure is given as follows.

1. Take an element from unsorted list.
2. Compare that element with sorted list from right to left.
3. Shift the list.

Here, it is assumed that first element is already sorted.

4.5.5.1 Pseudocode for Insertion Sort

```

INSERTION_SORT(A)
1. for j = 2 to n
2.   key ← A[j]
3.   // Insert A[j] into the sorted sequence A[1..j-1]
4.   j ← i - 1
5.   while i > 0 and A[i] > key
6.     A[i+1] ← A[i]
7.   i ← i - 1
8.   A[j+1] ← key

```

Example 4.10

Sorted part is bold in text and unsorted part is non-bold. Suppose 30, 10, 75, 33, 65 needs to be sorted using insertion sort.

After the first pass, we get

10, **30**, 75, 33, 65

After the second pass, we get

10, 30, 75, 33, 65

After the third pass, we get

10, 30, 35, 75, 65

After the fourth pass, we get

10, 30, 35, 65, 75

The sorted list is 10, 30, 35, 65, 75.

4.5.5.2 Recurrence Relation of Insertion Sort

The recurrence relation for insertion sort is

$$T(n) = \begin{cases} \Theta(1), & n = 1 \\ T(n-1) + \Theta(n), & n > 1 \end{cases}$$

4.5.5.3 Time Complexity

In sorting, the most expensive part is the comparison of two elements. Zero comparisons are required to insert first element, one for second element, two comparisons for third element, and so on. Atmost $N - 1$ comparisons are required for the last element in the list.

Complexity is: $1 + 2 + 3 + 4 \cdots + (N - 1) = O(n^2)$

The worst-case running time is $O(n^2)$

The best-case running time is $O(n)$

4.5.6 Heap Sort

Heap sort is one of the comparison based sort that uses heap data structure. It is a special case of selection sort. Its typical implementation is not stable, but can be made stable. Heap can be built by the following two types:

1. **Min-Heap:** The parent node will be less than or equal to its children nodes. So, the minimum element will be at root.
2. **Max-Heap:** The parent node will be greater than or equal to its children nodes. The maximum element will be at the root.

4.5.6.1 Heap Sort Algorithm for Sorting in Increasing Order

1. First build a max-heap from the input List.
2. In max heap, the largest element is at the root. So, swap the root with last element. Re-heapify the

tree. Repeat the above steps until the size of the heap is greater than 1.

- **Heapify function:** Heapify is an important subroutine for manipulating max-heaps. Its inputs are an array A and an index i into the array. When Heapify is called, it is assumed that the binary tree rooted at $\text{left}(i)$ and $\text{right}(i)$ is max-heap, but that $A[i]$ may be smaller than its children, thus violating the max-heap property.
- **Build Heap function:** Heapify procedure is used in a bottom-up to convert an array A into a max-heap. The elements in the subarray $A[(n/2) + 1 \dots n]$ are all leaves of the tree, and so each is a one-element heap to begin with. The procedure BuildHeap goes through the remaining nodes of the tree and runs Heapify on each node.

4.5.6.2 Pseudocode for Heapify Function

```

HEAPIFY(A, i)
1. le <- left(i)
2. ri <- right(i)
3. if (le <= heapsize) and (A[le] > A[i])
4. largest <- le
5. else
6. largest <- i
7. if (ri <= heapsize) and (A[ri] > A[largest])
8. largest <- ri
9. if (largest != i)
10. exchange A[i] <-> A[largest]
11. HEAPIFY(A, largest)

```

Example 4.11

Let $\{6, 5, 3, 1, 8, 7, 2, 4\}$ be the list that we want to sort from the smallest to the largest. (**Note:** For ‘building the heap’ step, larger nodes do not stay below smaller node parents. They are swapped with parents, and then

4.5.6.3 Pseudocode for BuildHeap Function

```

BUILD_HEAP(A)
1. heapsize <- length(A)
2. for i <- floor( length/2 ) down to 1
3. Heapify(A, i)

```

4.5.6.4 Pseudocode for Heap Sort

```

HEAP_SORT(A)
1. BuildHeap(A)
2. for i <- length(A) down to 2
3. exchange A[1] <-> A[i]
4. heapsize <- heapsize - 1
5. Heapify(A, 1)

```

4.5.6.5 Recurrence Relation of Heapify Function

The recurrence relation of Heapify() is

$$T(n) \leq \left\{ T\left(\frac{2n}{3}\right) + \Theta(1) \right\}$$

4.5.6.6 Time Complexity

The time complexity of Heapify() is $O(\log n)$. The time complexity of BuildHeap() is $O(n \log n)$, and the overall time complexity of the heap sort is $O(n \log n)$.

4.5.6.7 Applications of Heap Sort

1. It sorts a nearly sorted (or k -sorted) array.
2. It sorts k largest (or smallest) elements in an array.

recursively checked if another swap is needed, to keep larger numbers above smaller numbers on the heap binary tree.)

1. Build the heap:

Heap	Newly Added Element	Swap Elements
Nil	16	
16	20	
20 16		16 20
20 16 11	11	
20 16 11 8	8	
20 16 11 8	18	
20 16 11 8 18		16 18
20 18 11 8 16	14	
20 18 11 8 16 14		11 14
20 18 14 8 16 11		

2. Sort the list:

Heap	Swap Elements	Delete Element	Sorted Array	Details
20 18 14 8 16 11	20, 11			Swap 20 and 11 in order to delete 20 from the heap.
11 18 14 8 16 20		20		Delete 20 from the heap and add to the sorted array.
11, 18, 14, 8, 16, 20	11, 18		20	Swap 11 and 18 as they are not in order in the heap.
18 11 14 8 16 20	11 16		20	Swap 11 and 16 as they are not in order in the heap.
17, 15, 16, 7, 10, 19 18 16 14 8 11 20	18 11		20	Swap 18 and 11 in order to delete 17 from the heap.
11 16 14 8 18 20		18	20 18	Delete 18 from the heap and add to the sorted array.
11 16 14 8 18 20	11 14		20 18	Swap 11 and 14 as they are not in order in the heap.
11 14 16 8 18 20	14 16		20 18	Swap 14 and 16 in order to delete 14 from the heap.
16 14 11 8 18 20			20 18 16	Delete 16 from the heap and add to the sorted array.
8 14 11 16 18 20	8 14		20 18 16	Swap 14 and 8 as they are not in order in the heap.
14 8 11 16 18 20	14 11		20 18 16	Swap 14 and 11 in order to delete 14 from the heap.
10, 7, 15, 16, 17, 19 11 8 14 16 18 20		14	20 18 16 14	Delete 14 from the heap and add to the sorted array.
11 8 14 16 18 20	11 8		20 18 16 14	Swap 11 and 8 in order to delete 10 from the heap.
8, 11, 15, 16, 17, 19		11	20 18 16 14 11	Delete 11 from the heap and add to the sorted array.
7, 10, 15, 16, 17, 19 8 11 14 16 18 20		8	20 18 16 14 11 8	Delete 8 from the heap and add to the sorted array.
			20 18 16 14 11	Completed

4.5.7 Merge Sort

Merge sort is a divide-and-conquer algorithm. It divides the input array in two subarrays, calls itself for the two sub-arrays and then merges the two sorted arrays. The $\text{MERGE}(A, p, q, r)$ function is used for merging the two arrays. The $\text{MERGE}(A, p, q, r)$ is a key process that assumes that $\text{array}[l \dots m]$ and $\text{array}[m + 1 \dots r]$ are sorted and merges the two sorted sub-arrays into one. Merge sort is a stable sort. It is not an in-place sorting technique.

4.5.7.1 Pseudocode for Merge Sort

$\text{MERGE_SORT}(A, p, r)$

1. if $p < r$
2. Then $q = \text{floor}[(p + r)/2]$

3. $\text{MERGE}(A, p, q)$
4. $\text{MERGE}(A, q + 1, r)$
5. $\text{MERGE}(A, p, q, r)$

Pseudocode for MERGE function is given as follows:

$\text{MERGE}(A, p, q, r)$

1. $n1 \leftarrow q - p + 1$
2. $n2 \leftarrow r - q$
3. create arrays $L[1 \dots n1 + 1]$ and $R[1 \dots n2 + 1]$
4. For $i \leftarrow 1$ to $n1$
5. $\text{doL}[i] \leftarrow A[p + i - 1]$
6. For $j \leftarrow 1$ to $n2$
7. $\text{Do } R[j] \leftarrow A[q + j]$

```

8. L[n1 + 1] ← ∞
9. R[n2 + 1] ← ∞
10. i ← 1
11. j ← 1
12. For k ← p to r
13. Do if L[i] ≤ R[j]
14. then A[k] ← L[i]
15. i ← i + 1
16. Else A[k] ← R[j]
17. j ← j + 1

```

4.5.7.2 Recurrence Relation of Merge Sort

The recurrence relation for merge sort is

$$T(n) = \begin{cases} \Theta(1), & n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n), & n > 1 \end{cases}$$

Example 4.12

Sort the given list of elements: (54, 26, 93, 17, 77, 31, 44, 55 and 20).

Step 1: A recursive merge sort is called on the array of elements. Merge sort is a recursive function which divides an array into two parts; and calls itself recursively to divide the array into sub-parts until each part is having one element.

4.5.7.3 Time Complexity

The above recurrence can be solved using either recurrence tree method or master theorem. It falls under case 2 of the master theorem, and the solution of the recurrence is $O(n \log n)$. The time complexity of merge sort in all three cases (worst, average and best) is the same, as merge sort always divides the array in two sub-arrays and takes linear time to merge the two arrays.

4.5.7.4 Applications of Merge Sort

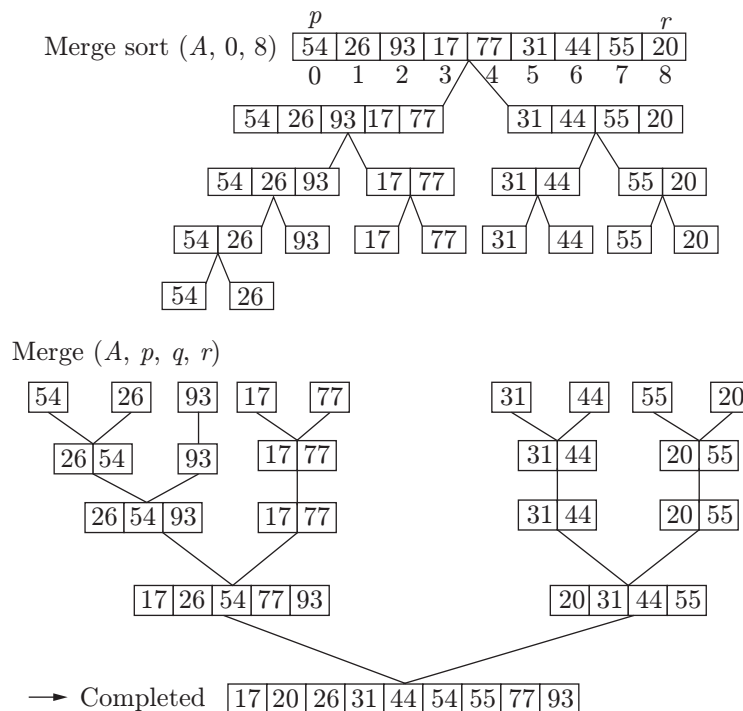
1. Merge sort is useful for sorting linked lists in $O(n \log n)$ time. Other $n \log n$ algorithms such as heap sort and quick sort (average case $n \log n$) cannot be applied to linked lists.
2. It is used in inversion count problem.
3. It is used in external sorting.

Note: Merge sort is not preferable for smaller-size element array.

Step 2: After dividing the array, merge sort calls the MERGE function to put the subarray into a sorted order.

Array A:

54	26	93	17	77	31	44	55	20
----	----	----	----	----	----	----	----	----



4.5.8 Quick Sort

Quicksort is also divide-and-conquer strategy of sorting a list of elements like merge sort. This divides array of elements into sub-array $S[p \dots r]$. The concept is described as follows:

1. **Divide:** Partition $S[p \dots r]$ into two sub-arrays $S[p \dots q-1]$ and $S[q+1 \dots r]$ such that each element of $S[p \dots q-1]$ is less than or equal to $S[q]$, which is, in turn, less than or equal to each element of $S[q+1 \dots r]$. Compute the index q as part of this partitioning procedure.
2. **Conquer:** Sort the two sub-arrays $S[p \dots q-1]$ and $S[q+1 \dots r]$ by recursive calls to quicksort.
3. **Combine:** Since the sub-arrays are sorted in place, no work is needed to combine them; the entire array S is now sorted.

4.5.8.1 Pseudocode for Quick Sort

```

QUICKSORT(S, p, r)
1. If p < r
2. then q ← PARTITION(S, p, r)
3. QUICKSORT(S, p, q-1)
4. QUICKSORT(S, q+1, r)

```

4.5.8.2 Pseudocode for Partition

```

PARTITION(S, p, r)
1. x ← S[r]
2. i ← p-1
3. for j ← p to r-1
4. do if S[j] ≤ x

```

Example 4.13

Sort the given list of elements: (38, 81, 22, 48, 13, 69, 93, 14, 45, 58, 79, 72)

Step 1: Choose the pivot element which is at position $((\text{left} + \text{right})/2)$.

Step 2: Now in partitioning phase: During the partitioning process:

```

5. then i ← i+1
6. swap S[i] ↔ S[j]
7. swap S[i+1] ↔ S[r]
8. return i+1

```

4.5.8.3 Recurrence Relation of Quick Sort

1. **For worst case:** The worst case of quick sort occurs when the pivot we picked turns out to be the least element of the array to be sorted, in every step (i.e. in every recursive call). A similar situation will also occur if the pivot happens to be the largest element of the array to be sorted. Then recurrence relation of quick sort is

$$T(n) = T(1) + T(n-1) + \Theta(n)$$

2. **For best case:** The best case of quicksort occurs when the pivot we picked happens to divide the array into two almost equal parts, in every step. Thus, we have $k = n/2$ and $n - k = n/2$ for the original array of size n . Then the recurrence relation of quick sort is given as

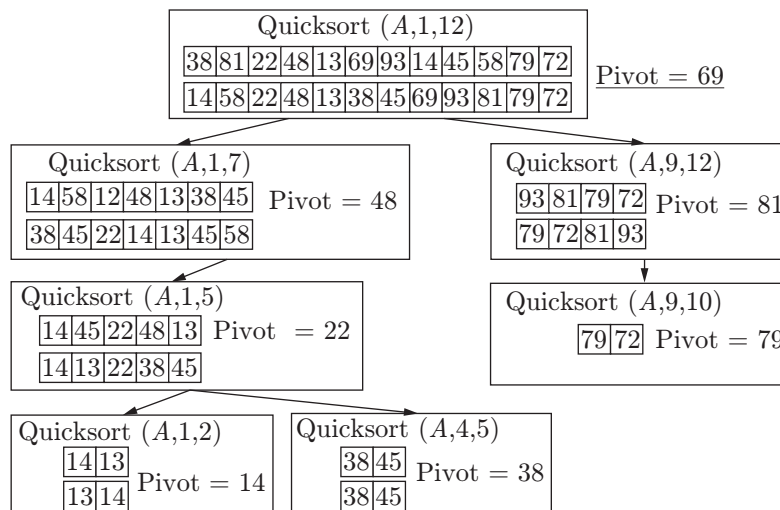
$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

4.5.8.4 Time Complexity

The best-case and average-case running time of quick sort = $O(n \log n)$.

The worst-case running time of quick sort = $O(n^2)$.

Note: The worst case of quick sort occurs when the given elements are already sorted or almost sorted.



4.5.9 Randomized Quick Sort

In the randomized version of quick sort, we impose a distribution on the input. This does not improve the worst-case running time, but it improves the probability of getting the average-case running time.

In this version, we choose a random key for the pivot. Assume that the procedure $\text{Random}(a, b)$ returns a random integer in the range $[a, b]$; there are $b - a + 1$ integers in the range, and the procedure is equally likely to return one of them. The new partition procedure simply implements the swap before actually partitioning.

4.5.9.1 Pseudocode for Randomized Quick Sort

$\text{RANDOMIZED_PARTITION}(A, p, r)$

```
1.  $i \leftarrow \text{RANDOM}(p, r)$ 
2. Exchange  $A[p] \leftarrow A[i]$ 
3. return  $\text{PARTITION}(A, p, r)$ 
```

The pseudocode for randomized quick sort has the same structure as quick sort, except that it calls the randomized version of the partition procedure.

$\text{RANDOMIZED_QUICKSORT}(A, p, r)$

```
1. If  $p < r$  then
2.  $Q \leftarrow \text{RANDOMIZED\_PARTITION}(A, p, r)$ 
3.  $\text{RANDOMIZED\_QUICKSORT}(A, p, q)$ 
4.  $\text{RANDOMIZED\_QUICKSORT}(A, q+1, r)$ 
```

4.5.9.2 Time Complexity

The best- and average-case running time of randomized quick sort = $O(n \log n)$.

The worst-case running time of randomized quick sort = $O(n^2)$.

Note: The worst case of randomized quick sort occurs when all the given elements are equal.

4.5.10 Counting Sort

Counting sort is a sorting technique based on keys within a specific range. It works by counting the number of objects having distinct key values (kind of hashing), and then doing some arithmetic to calculate the position of each object in the output sequence.

4.5.10.1 Pseudocode for Counting Sort

$\text{COUNTING_SORT}(A[], B[], k)$

```
1. for  $i = 1$  to  $k$  do
2.  $C[i] = 0$ 
3. for  $j = 1$  to  $\text{length}(A)$  do
4.  $C[A[j]] = C[A[j]] + 1$ 
```

```
5. for  $2 = 1$  to  $k$  do
6.  $C[i] = C[i] + C[i-1]$ 
7. for  $j = 1$  to  $\text{length}(A)$  do
    $B[C[A[j]]] = A[j]$ 
    $C[A[j]] = C[A[j]] - 1$ 
```

Although this may look complicated, it is actually a very simple and clever algorithm.

1. An array $A[]$ stores the initial data to be sorted.
2. An array $C[]$ is used to count the occurrences of the data values.
3. An array $B[]$ is used to store the final, sorted, list.
4. The first for loop initializes $C[]$ to zero.
5. The second for loop increments the values in $C[]$, according to their frequencies in the data.
6. The third for loop adds all the previous values, making $C[]$ contain a cumulative total.
7. The fourth for loop writes out the sorted data into the array $B[]$.

4.5.10.2 Time Complexity

$O(n + k)$, where n is the number of elements in the input array and k is the range of input.

Note:

1. Counting sort is efficient if the range of input data is not significantly greater than the number of objects to be sorted.
2. It is not a comparison-based sorting. Its running time complexity is $O(n)$ with space proportional to the range of data.
3. This can be used as subroutine to some other algorithm. It is often used as a subroutine to another sorting algorithm, such as radix sort.
4. It uses a partial hashing to count the occurrence of the data object in $O(1)$.
5. It can be extended to work for negative inputs also.

Example 4.14

Consider the data in the range 0 to 9.

Input data: 1, 4, 1, 2, 7, 5, 2

The steps to sort elements through counting sort are as follows:

1. Take a count array to store the count of each unique object:
Index: 0 1 2 3 4 5 6 7 8 9
Count: 0 2 2 0 1 1 0 1 0 0
2. Modify the count array such that each element at each index stores the sum of the previous counts:
Index: 0 1 2 3 4 5 6 7 8 9
Count: 0 2 4 4 5 6 6 7 7 7

The modified count array indicates the position of each object in the output sequence.

Table 4.1 | Sorting techniques

Sorting Algorithm	Average Time	Best Time	Worst Time	Space	Stability	In-Place Sorting
Bubble sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Constant	Stable	Yes
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Constant	Stable	Yes
Insertion sort	$O(n^2)$	$O(n)$	$O(n^2)$	Constant	Stable	Yes
Heap sort	$O(n \times \log(n))$	$O(n \times \log(n))$	$O(n \times \log(n))$	Constant	Instable	Yes
Merge sort	$O(n \times \log(n))$	$O(n \times \log(n))$	$O(n \times \log(n))$	Depends	Stable	No
Quick sort	$O(n \times \log(n))$	$O(n \times \log(n))$	$O(n^2)$	Constant	Stable	Yes
Count sort	$O(n)$	$O(n)$	$O(n)$	Constant	Stable	No

3. Output each object from the input sequence followed by decreasing its count by 1.

- Process the input data: 1, 4, 1, 2, 7, 5, 2. Position of 1 is 2.
- Put data 1 at index 2 in the output. Decrease count by 1 to place the next data 1 at an index 1 smaller than this index.
- Put data 2 at index 4 in the output. Decrease count by 1 to place the next data 2 at an index 3 smaller than this index.
- Now, move to the next element 3 which has count 4, and index 4 is already filled by 2 which means element 3 does not exist in the given input list.
- Follow this process for all the elements.

- It does not have any parallel edge.

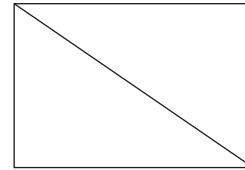


Figure 4.5 | Simple graph.

2. **Multigraph:** A graph with a self-loop and parallel edges is called a multigraph (Fig. 4.6).

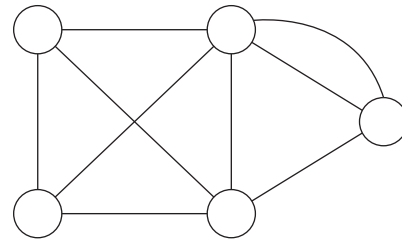


Figure 4.6 | Multigraph.

4.5.11 Comparison of Sorting Techniques

The comparison of the various sorting techniques is given in Table 4.1.

4.6 GRAPH

A graph $G = (V, E)$ is a set of vertices and edges where each edge consist of pair of vertices.

- Finite set of vertices are known as nodes of graph.
- (u, v) is a pair of vertices called edge.
- For directed graph (u, v) is an ordered pair that represents an edge from node u to node v .

4.6.1 Types of Graph

There are two types of graphs:

- Simple graph:** A graph is called a simple graph (Fig. 4.5) if
 - It contains no self-loop.

4.6.2 Types of Simple Graph

4.6.2.1 Null Graph

If in the given graph there is no edge, then it is called a null graph. In other words, a null graph does not have any edge between any two vertices (Fig. 4.7).

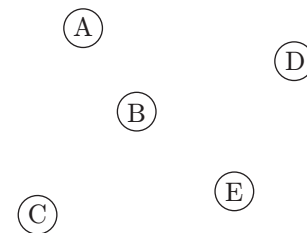


Figure 4.7 | Null graph.

4.6.2.2 Regular Graph

A graph is called a regular graph if all of its vertices have the same degree (Fig. 4.8).

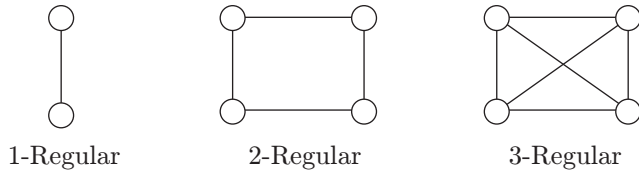


Figure 4.8 | Regular graph.

If a graph has n number of vertices and it is k -regular, where k is a positive constant, then the number of edges $= (n \times k)/2$.

4.6.2.3 Complete Graph

In the given graph, if every vertex is adjacent to all other remaining vertices, then that graph is called a complete graph (Fig. 4.9). If a graph is a complete graph and is having n vertices, then it is called a K_n graph.

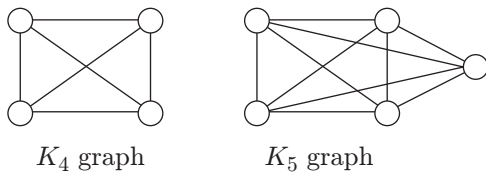


Figure 4.9 | Complete graphs.

A complete graph having n vertices has $[n \times (n - 1)]/2$ edges.

Note: The number of simple graphs possible with n vertices $= 2^{[n(n-1)]/2}$.

4.6.3 Graph Representation

Graph data structure is useful in many real-time applications like they are used to represent complex networks. The network represented by graph can be of city defining paths, of telephone or circuit network.

The following two are the most commonly used representations of a graph:

1. Adjacency matrix
2. Adjacency list

Incidence matrix and incidence list are two other representations for graph. Any representation can be chosen depending on the situation.

4.6.3.1 Adjacency Matrix

An adjacency matrix is a two-dimensional array of size $V \times V$, where V represents the number of vertices in a

graph. If $\text{adj}[i][j] = 1$, this indicates there exists an edge between vertices i and j ; otherwise, the value will be 0. The adjacency matrix for undirected graph is symmetric. Weights can also be represented by this matrix. The value $\text{adj}[i][j] = w$ means weight of edge from i to j is w .

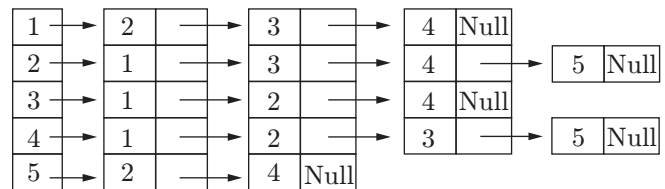
Vertex	1	2	3	4	5
1	0	1	1	1	0
2	1	0	1	1	1
3	1	1	0	1	0
4	1	1	1	0	1
5	0	1	0	1	0

This type of representation is very easy for implementation purpose. $O(1)$ time is required to remove an edge. Testing that an edge exists between two nodes or not can be computed in $O(1)$ time complexity.

Space requirement is more for both dense and sparse matrices is $O(V^2)$. Addition of a vertex takes $O(V^2)$ time.

4.6.3.2 Adjacency List

To implement adjacency list, an array of linked list is used. Array size is kept equal to the total number of vertices. For each vertex i , linked list of its adjacent nodes is stored in $\text{array}[i]$. Weights of edges can also be stored in linked list nodes. The representation of adjacency list is as follows:



The space required for adjacency list is: $O(V+E)$, which is less than that of adjacency matrix. In worst-case, space consumed is $O(V^2)$. Addition of vertex is easy, but finding an edge between vertices, say u and v , is not efficient. It requires $O(V)$ complexity.

4.7 GREEDY APPROACH

A greedy algorithm is an algorithm for solving problems in an optimised way. In this approach, the algorithms execute repeatedly to maximize the outcome, considering the local conditions for some global problem. For some problems, it guarantees the optimal solution while for some it does not. The main strategy is to implement the problem so that it requires minimal resources.

The important terms are given as follows:

1. **Solution space:** The set of all possible solutions for given n inputs is called solution space.
2. **Feasible solution:** The feasible solution is one of the solutions from the solution space which satisfies the given condition.
3. **Optimal solution:** The optimal solution is one of the feasible solutions which optimizes our goal.

4.7.1 Applications of Greedy Approach

The following are the applications of the greedy approach:

1. Knapsack problem (fractional)
2. Job sequence with deadline
3. Huffman coding
4. Minimum cost spanning tree:
 - a. Kruskal's algorithm
 - b. Prim's algorithm
5. Singlesource shortest path:
 - a. Dijkstra's algorithm
 - b. Bellman-Ford algorithm

4.7.2 Fractional Knapsack

In this problem, items are given along with its weight and profit. The target is to maximize the profit considering the weight constraint. Unlike 0-1 knapsack problem, fraction of item can be considered. Fractional knapsack problem can be solved by Greedy approach.

Greedy Algorithm: The solution can be obtained by making different choices. At each stage, the best possible choice is made. In fractional knapsack, first of all profit value/weight ratios are calculated and then sorted in descending order. Item with highest value/weight ratio is chosen and added in the collection. The maximum weight is checked after adding each item. If the entire item cannot be included, then fraction of it is added to the collection.

Example 4.15

There are n items in a store. For $i = 1, 2, \dots, n$, item i has weight $w_i > 0$ and worth $v_i > 0$. A thief can carry a maximum weight of W pounds in a knapsack. In this version of a problem, the items can be broken into smaller pieces, so the thief may decide to carry only a fraction x_i of object i , where $0 \leq x_i \leq 1$. Item i contributes $x_i w_i$ to the total weight in the knapsack, and $x_i v_i$ to the value of the load.

In symbol, the fraction knapsack problem can be stated as follows.

Maximize $nS_i = 1x_i v_i$ subject to constraint $nS_i = 1x_i w_i \leq W$. It is clear that an optimal solution must fill the

knapsack exactly; otherwise, we could add a fraction of one of the remaining objects and increase the value of the load. Thus, in an optimal solution, $nS_i = 1x_i w_i = W$.

4.7.2.1 Pseudocode for Fractional Knapsack

Greedy-fractional-knapsack (w, v, W)

```

1. for i = 1 to n
2. Do  $x[i] = 0$ 
3. weight = 0
4. while weight < W
5. Do i = best remaining item
6. if ( weight +  $w[i] \leq W$ )
7. Then  $x[i] = 1$ 
8. weight = weight +  $w[i]$ 
9. else
10.  $x[i] = (W - \text{weight}) / w[i]$ 
11. weight = W
12. Return x

```

4.7.2.2 Time Complexity

If the ratio of v_i/w_i is already sorted in decreasing order, then the time taken by the while loop will be $O(n)$. So, the total time required will be $O(n \log n)$. If heap data structure is used, which keeps highest value/weight ratio at root. The construction of heap will take $O(n)$ time and while loop will take $O(\log n)$ time (heap property is restored for each root removal). This does not improve the worst case, but it is faster if small number of items are to be filled in knapsack.

Example 4.16

Consider the following details:

1. Knapsack size = 20
2. Number of objects = 3

Object	Obj1	Obj2	Obj3
Profit	25	24	15
Weight	18	15	10

Solution:

Step 1: Calculate ratio (profit/weight):

Object	Obj1	Obj2	Obj3
Profit	25	24	15
Weight	18	15	10
Ratio	1.38	1.6	1.5

Step 2: Put according to decreasing ratio (profit/weight):

Object	Obj2	Obj3	Obj1
Profit	24	15	25
Weight	15	10	18
Ratio	1.6	1.5	1.38

Step 3: Put items into the knapsack till the knapsack is full:

- (a) Put object 2 into the knapsack; it will add (value = 24 and weight = 15) to the knapsack. Knapsack has 5 units capacity remaining.
- (b) Now object 3 cannot be put completely into the knapsack, so put a fraction of it. Add (value = 7.5 and weight = 5) to the knapsack. Knapsack is completely full, which means no other object can be put into it.

Step 4: Total knapsack value = 31.5 (which is optimum). ●

4.7.3 Huffman Coding

Huffman coding is a common technique of encoding, including lossless data compression. The algorithm's output can be viewed as a variable-length code table for encoding a source symbol (such as a character in a file). Huffman coding is based on the frequency of occurrence of a data item (pixel in images). The main aim of Huffman coding is to encode the data that occurs frequently using less number of bits. Code books store codes that are constructed for images. Both the code book and encoded data are transmitted to decode the information.

Huffman codes can be constructed using following two ideas:

1. For an optimum code, symbols that have high probability should be assigned shorter code words.
2. To make optimum prefix code, two symbols that occur least frequently should be assigned same length.

4.7.3.1 Principle of Huffman Codes

1. Starting with the two least probable symbols, γ and δ , of an alphabet A , if the code word for γ is $[m]0$, the code word for δ would be $[m]1$, where $[m]$ is a string of 1s and 0s.
2. Now, the two symbols can be combined into a group, which represents a new symbol ψ in the alphabet set.
3. The symbol ψ has the probability $P(\gamma) + P(\delta)$. Recursively, determine the bit pattern $[m]$ using the new alphabet set.

4.7.3.2 Pseudocode for Huffman Coding

```
HUFFMAN(f[1...n])
1. T = empty binary tree
2. Q = priority queue of pairs (i, f[i]),
   i = 1...n, with f as comparison key
3. for each k = 1...n - 1
4. i = extractMin(Q)
5. j = extractMin(Q)
6. f[n + k] = f[i] + f[j]
7. insertNode(T, n + k) with children i, j
8. insertRear(Q, (n + k, f[n + k]))
9. return T
```

4.7.3.3 Time Complexity

From the above pseudocode, we can see that with each iteration, the problem size is reduced by 1. Hence, there are exactly n iterations. The i th iteration consists of locating the two minimum values in a list of length $n - i + 1$. This is a linear operation, and so Huffman's algorithm clearly has a time complexity of $O(n^2)$.

However, it would be faster to sort the weights initially, and then maintain two lists. The first list consists of weights that have not been combined yet, and the second list consists of trees that have been formed by combining weights. This initial ordering is obtained at the cost of $O(n \log n)$. Obtaining the minimum two trees at each step then consists of two comparisons (comparing the heads of the two lists, and then comparing the larger item with the item after the smaller). The ordering of the second list can be maintained cheaply by using a binary search to insert new elements. Since at step i there are $i - 1$ elements in the second list, $O(\log i)$ comparisons are needed for insertion. Over the entire duration of the algorithm, the cost of keeping this list sorted is $O(n \log n)$. Therefore, the overall time complexity of Huffman's algorithm is $O(n \log n)$.

Example 4.17

In the following example five different symbols are given along with their frequency.

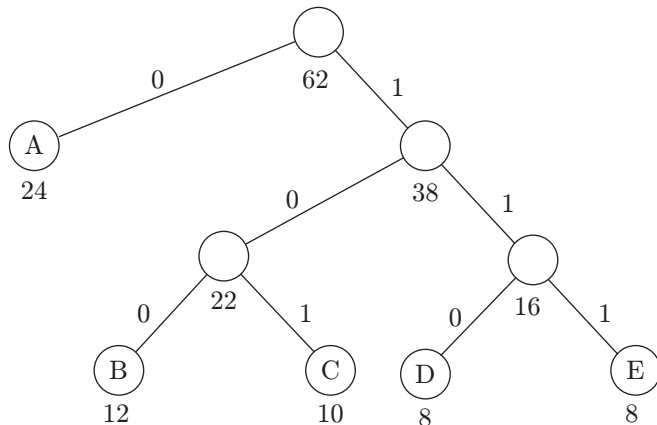
Symbol	Frequency
A	24
B	12
C	10
D	8
E	8

There are a total of 186 bits (with 3 bits per code word).

Here, the least frequency symbols are E and D. First of all, we will connect them and frequency of node will become 16. Then B and C are combined to make the

frequency 22. These will be two new nodes. In the next step, these two nodes are brought together and act as subordinate to the root node. A also acts as subordinate to root node.

4.7.3.4 Code Tree According to Huffman



Symbol	Frequency	Code	Code Length	Total Length
A	24	0	1	24
B	12	100	3	36
C	10	101	3	30
D	8	110	3	24
E	8	111	3	24

The total length is 138 bits.

4.7.4 Minimum Spanning Tree

Spanning tree is a sub-graph that connects all the vertices together. A single graph can have multiple spanning trees. Minimum spanning tree for a weighted undirected graph is the spanning tree with minimum total weights. The weight of spanning tree is addition of weights of all the edges in the spanning tree. An MST with N vertices, has $N - 1$ edges for a given graph.

4.7.4.1 Prim's Algorithm

Prim's algorithm is a method to find minimum spanning tree. In this algorithm minimum spanning tree formed should always be connected. Prim's algorithm is an example of Greedy approach.

Properties of Prim's Algorithm

Prim's algorithm has the following properties:

1. Prime's algorithm always results into a single tree.
2. Tree grows until it covers all the vertices of the graph.

3. At each step, an edge is added to the graph that has minimum weight and is the neighbour to the previous vertices.

Steps of Prim's Algorithm

1. Find the minimum weight edge from graph and mark it.
2. Consider the neighboring edges of the selected edges and mark the minimum weight edge among those edges.
3. Continue this until we cover $n-1$ edges.
4. The resultant tree is minimum spanning tree and it should always be connected.

Input: Undirected connected weighted graph $G = (V, E)$ in adjacency list representation, source vertex s in V and w is an array representing weights of edges.

Output: $p[1 \dots |V|]$, representing the set of edges composing an MST of G .

Note: The field $\pi(v)$ names the parent of v in the tree.

$\text{Key}(v)$ is the minimum weight of any edge connecting v to a vertex in tree. $\text{Key}(u) \leftarrow \infty$ means there is no such edge.

```

PRIM (G, w, s)
1. for each u in V(G)
2. do key(u) ← ∞
3. π(v) ← NIL
4. key[s] ← 0
5. Q ← V [G]
6. while Q != empty
7. do u ← Extract-Min(Q)
8. for each v in Adjacent[u]
9. do if v ∈ Q and w(u, v) < key[u]
10. then π[v] ← u
11. key[v] ← w(u, v)

```

Time Complexity

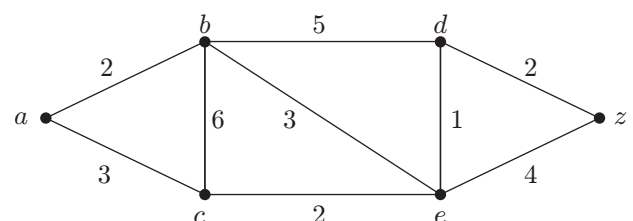
Prims algorithm use min heap data structure and its time complexity is $O(V \log V + E \log V)$.

Maximum edges in tree, $E = V - 1$

Therefore, $O(V \log V + (V - 1) \log V) = O(2V \log V - \log V)$

Thus, complexity = $O(V \log V)$

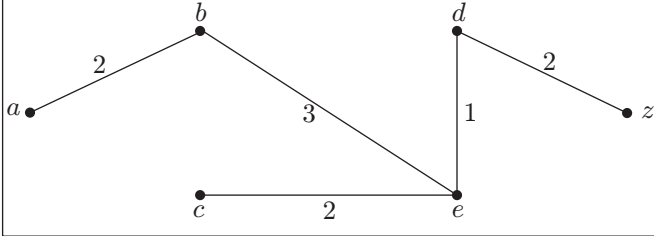
Problem 4.5: Use Prim's algorithm to find an MST in the following weighted graph. Use alphabetical order to break ties.



Solution: Prim's algorithm will proceed as follows.

1. First we add edge $\{d, e\}$ of weight 1.
2. Next, we add edge $\{c, e\}$ of weight 2.
3. Next, we add edge $\{d, z\}$ of weight 2.
4. Next, we add edge $\{b, e\}$ of weight 3.
5. And finally, we add edge $\{a, b\}$ of weight 2.

This produces a MST of weight 10.



4.7.4.2 Kruskal's Algorithm

One another algorithm to find the minimum spanning tree is Kruskal's. This method considers all minimum edges of graph one by one in increasing order and gives a resultant minimum spanning tree. In Kruskal's algorithm, it is not necessary for a tree to be connected.

Implementation steps:

1. Start from the source node.
2. Scan the weights and include the minimum weights to the tree in increasing order.
3. Stop if $N - 1$ edges are included.

Uses a disjoint-set data structure to determine whether an edge connects vertices in different components.

Data Structure

Before formalizing the above idea, let us quickly understand the disjoint-set data structure:

1. **Make-SET(v):** It creates a new set whose only member is pointed to by v . Note that for this operation, v must already be in a set.
2. **FIND-SET(v):** It returns a pointer to the set containing v .
3. **UNION(u, v):** It unites the dynamic sets that contain u and v into a new set that is a union of these two sets.

Algorithm

Starting with an empty set A , select the shortest edge that has not been chosen or rejected at every step, regardless of the position of this edge in the graph. The pseudocode for Kruskal algorithm is given:

KRUSKAL(V, E, w)

1. $A \leftarrow \{\emptyset\}$ $\triangleright$ Set A will ultimately contain the edges of the MST
2. **For** each vertex v in $V[G]$

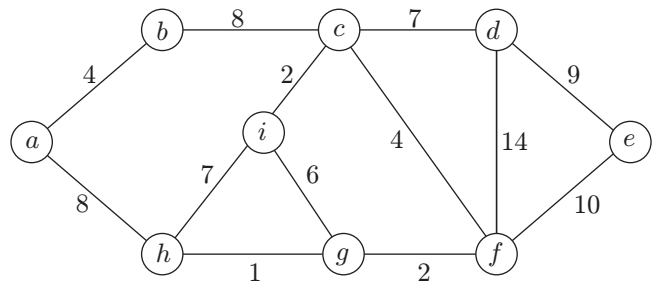
3. **Do** MAKE-SET(v)
4. Sort E into non-decreasing order by weight w
5. **For** each edge $(u, v) \in E$, taken in non-decreasing order by weight w
6. **do if** FIND-SET(u) $\neq$ FIND-SET(v)
7. **Then** $A \leftarrow A \cup \{(u, v)\}$
8. UNION(u, v)
9. **Return** A

Time Complexity

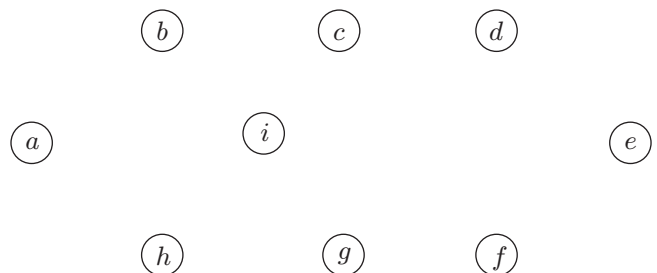
The edge weight can be compared in constant time. Initialization of priority queue takes $O(E \log E)$ time by repeated insertion. At each iteration of while loop, minimum edge can be removed in $O(\log E)$ time, which is $O(\log V)$, since the graph is simple. The total running time is $O((V + E) \log V)$, which is $O(E \log V)$ since the graph is simple and connected.

Example 4.18

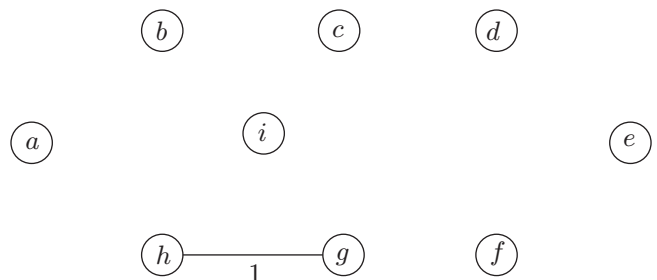
Let us use the Kruskal's algorithm to solve the following example.



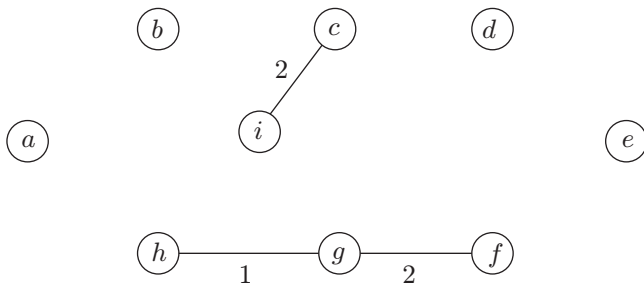
Step 1: Take only the vertices of graph.



Step 2: Choose the minimum weight edge from the original graph and draw that edge in disconnected graph containing n connected components as shown below:



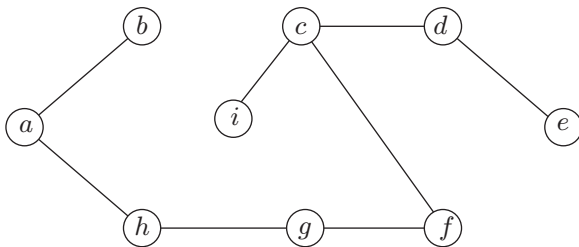
Step 3: Now select the next minimum weight edge and draw the corresponding edge in disconnected graph.



Here, there are two edges containing weight 2, add all possible edges of same weight until they form loop in the tree (since tree doesnot contain any loop but a graph may contain). That is why edge(g, f) and edge(i, c) both are added.

Step 4: Continue this process and check for the next smallest edge and add it into tree. Do this till we get $(n - 1)$ edges in the tree.

The final minimum spanning tree would be:



4.7.5 Single-Source Shortest Path

Given a weighted graph G , find the shortest path from a given vertex to every other vertex in G . Note that we can solve this problem quite easily with BFS traversal in the special case when all weights are equal. The Greedy approach to this problem is repeatedly selecting the best choice from those available at that time.

4.7.5.1 Dijkstra's Algorithm

Dijkstra's algorithm is a single source shortest path algorithm. It calculates the shortest distance starting from the start node till another node in the graph. While calculating the shortest distance, it excludes the longer distance paths. Dijkstra's algorithm does not operate on negative weights. For calculating shortest distance with negative edges, Bellman-Ford algorithm is used. Steps to perform Dijkstra's algorithm are as follows:

1. Assign all the nodes with distance infinity and 0 to initial node. Distance of start node s permanent and of all the other nodes is temporary. Set start node as active node.

2. Calculate the temporary distance of neighbours of active nodes. Distance is calculated by adding up the weights associated with those edges.
3. Update the distance, and set the current node an antecessor if such a calculated distance of a node is smaller as the current one. This step is also called update and is Dijkstra's central idea.
4. Set the node as active node which has minimal temporary distance. Mark the distance as permanent.
5. Repeat steps 2 to 4 until there are no nodes left with a permanent distance, whose neighbours still have temporary distances.

Dijkstra's Pseudocode

```

DIJKSTRA (G, S):
1. for each vertex v in Graph:
2.   distance[v] := infinity
3.   previous[v] := undefined
4.   distance[S] := 0
5. Q := the set of all nodes in Graph
6. While Q is not empty:
7.   u := node in Q with smallest distance[ ]
8.   remove u from Q
9.   for each neighbour v of u:
10.    alt := distance[u] + distance_between(u, v)
11.    If alt < distance[v]
12.    distance[v] := alt
13.    previous[v] := u
14. Return previous[ ]

```

Time Complexity

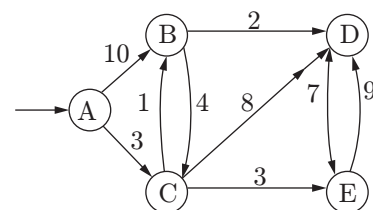
Like Prim's algorithm, Dijkstra's algorithm runs in $O(E \log V)$ time.

1. Dijkstra's algorithm with list: $O(V^2)$
2. Dijkstra's algorithm with modified binary heap: $O((E + V) \log V)$
3. Dijkstra's algorithm with Fibonacci heap: $O(E + V \log V)$

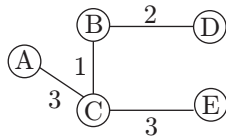
Example

The following are the steps:

1. Start from the source node; find the distance of all nodes reachable from the source.
2. Then pick the node which has the minimum distance, from the pool of nodes.
3. Calculate again the distance from the selected node, and follow the above two steps till the entire nodes are selected.



Source	(A)	B	C	D	E	
	∞	∞	∞	∞	∞	→ Cost
	Nil	Nil	Nil	Nil	Nil	→ Parent
	O	10	(3)	∞	∞	
→ A	Nil	A	A	Nil	Nil	
	O	(4)		11	6	
→ AC	Nil	C		C	C	
	O			(6)	6	
→ ACB	Nil			B	C	
	O				(6)	
→ ACBD	Nil				C	



4.7.5.2 Bellman–Ford Algorithm

The Bellman–Ford algorithm is used to compute single source shortest path to all other vertices of weighted directed graph. This algorithm also considers negative weights of graph edges. This algorithm is slower but is capable of even finding a negative weight cycle in graph.

Bellman-Ford Pseudocode

```

1. d[s] ← 0
2. for each v ∈ V - {s}
3. do d[v] ← ∞
4. for i ← 1 to |V| - 1 do
5. for each edge (u, v) ∈ E do
6. if d[v] > d[u] + w(u, v) then
7. d[v] ← d[u] + w(u, v)
8. π[v] ← u
9. for each edge (u, v) ∈ E
10. do if d[v] > d[u] + w(u, v)
11. then report that a negative-weight
    cycle exists
  
```

Time Complexity

The time complexity of Bellman-Ford algorithm is $O(VE)$, where $E = V^2$, so the worst-case running time of Bellman–Ford algorithm is $O(V^3)$.

4.8 GRAPH TRAVERSAL

Graph traversal means visiting all the nodes in a graph in some order. In graph traversal, some nodes may be visited more than once. This is because it is not necessary

to know that the node has been explored before or not. The re-traversal becomes more for dense graph, which increase computation time. Tree traversal is a special case of graph traversal. The following are two graph traversal methods:

1. Breadth-first traversal (BFT)
2. Depth-first traversal (DFT)

4.8.1 Breadth-First Traversal

The breath-first traversal starts from the root node and then traverses all its neighbours. Then for each neighbour, its corresponding unvisited neighbour is processed. The pseudocode for BFT is given as follows:

```

BST (V)
1. visited(V) = 1
2. add(V, Q)
3. while(Q is not empty)
4. u = delete(Q)
5. for all x adjacent to u
6. if( x is not visited)
7. visited(x) = 1
8. add(Q, x)
  
```

4.8.1.1 Time Complexity

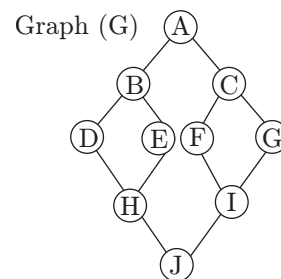
The time complexity of BFT is $O(V + E)$, where V is the number of vertices in the graph and E is the number of edges in the graph.

4.8.1.2 Application of BFT

The following are the applications of BFT. It can be used to find out

1. whether the given graph is connected.
2. the number of connected components in a graph.
3. whether the given graph contains a cycle.
4. the shortest path if all the edges have the same weight.
5. whether the given graph is a bipartite graph.

Problem 4.6: Consider the graph $G = (V, E)$ and find the BFT starting node A.



Solution: A is the starting node and Q is a queue data structure to store the values.

BFT (A) :

1. Visited (A) = 1
Add (Q, A)

A				
----------	--	--	--	--

2. $u = \text{delete } (Q) \rightarrow A$
For all adjacent nodes to A which are not already added in the queue, Add (Q, X), where $X = (B, C)$ is the adjacent node to A.

B	C			
----------	----------	--	--	--

3. $u = \text{delete } (Q) \rightarrow B$
For all adjacent nodes to B which are not already added in the queue, Add (Q, X), where $X = (D, E)$ is the adjacent node to B.

C	D	E		
----------	----------	----------	--	--

4. $u = \text{delete } (Q) \rightarrow C$
For all adjacent nodes to C which are not already added in the queue, Add (Q, X), where $X = (F, G)$ is the adjacent node to C.

D	E	F	G	
----------	----------	----------	----------	--

5. $u = \text{delete } (Q) \rightarrow D$
For all adjacent nodes to D which are not already added in the queue, Add (Q, X), where $X = (H)$ is the adjacent node to D.

E	F	G	H	
----------	----------	----------	----------	--

6. $u = \text{delete } (Q) \rightarrow E$
For all adjacent nodes to E which are not already added in the queue, Add (Q, X), where $X = (\Phi)$ is the adjacent node to E.

F	G	H		
----------	----------	----------	--	--

7. $u = \text{delete } (Q) \rightarrow F$
For all adjacent nodes to F which are not already added in the queue, Add (Q, X), where $X = (I)$ is the adjacent node to F.

G	H	I		
----------	----------	----------	--	--

8. $u = \text{delete } (Q) \rightarrow G$
For all adjacent nodes to G which are not already added in the queue, Add (Q, X), where $X = (I)$ is the adjacent node to G.

H	I			
----------	----------	--	--	--

9. $u = \text{delete } (Q) \rightarrow H$
For all adjacent nodes to H which are not already added in the queue, Add (Q, X), where $X = (J)$ is the adjacent node to H.

I	J			
----------	----------	--	--	--

10. $u = \text{delete } (Q) \rightarrow I$

For all adjacent nodes to I which are not already added in the queue, Add (Q, X), where X is the adjacent node to I.

J				
----------	--	--	--	--

11. $u = \text{delete } (Q) \rightarrow J$

For all adjacent nodes to J which are not already added in the queue, Add (Q, X), where X is the adjacent node to J.

--	--	--	--	--

12. The queue is empty, which means graph is traversed completely.

4.8.2 Depth-First Traversal

As already discussed, there are many ways of tree traversal. Consider the tree given in Fig. 4.10:

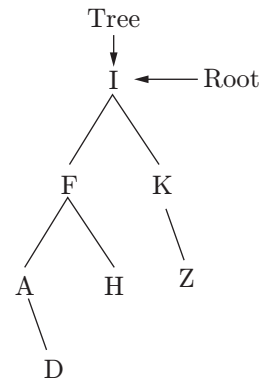


Figure 4.10 | DFT of a tree.

Preorder traversal for given tree: I F A D H K Z

Preorder traversal also results in depth first traversal. This is because the traversal goes deeper before traversing its siblings. In this tree, the descendants of F, that is, A, H, D are traversed first and after that sibling of F, that is, K are traversed.

4.8.2.1 Pseudocode for DST

DFT (V)

```

1. visited (V) =1
2. for all x adjacent to V
3. if (x is not visited)
4. visited (x) =1
5. DFT (x)
  
```

The above is the recursive code for DFT.

4.8.2.2 Time Complexity

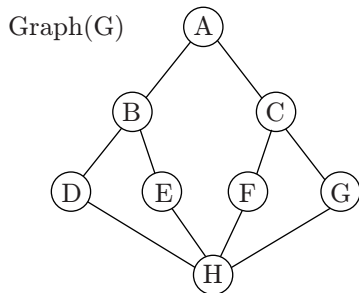
The time complexity of DFT is $O(V + E)$, where V is the number of vertices in the graph and E is the number of edges in the graph.

4.8.2.3 Application of DFT

The following are the applications of BFT. It can be used to find out

1. whether the given graph is connected.
2. the number of connected components in a graph.
3. whether the given graph contains a cycle.
4. whether the given directed graph is strongly connected.

Problem 4.7: Consider the graph $G = (V, E)$ and find the DFT starting node A.



Solution: A is the starting node and S is a stack data structure to store the values. Stack is internally used by the recursive call.

DFT (A)

1. Visited (A) = 1

PUSH (S , A)

A				
---	--	--	--	--

2. POP (S) $\rightarrow$ A

For all adjacent nodes to A which are not already added in the stack, PUSH (S , X). Every neighbour element will be stored in the stack from right to left. Here, $X = (B, C)$ is the adjacent node to A.

B	C			
---	---	--	--	--

3. POP (S) $\rightarrow$ B

For all adjacent nodes to B which are not already added in the stack, PUSH (S , X). Every neighbour element will be stored in the stack from right to left. Here, $X = (D, E)$ is the adjacent node to B.

D	E	C		
---	---	---	--	--

4. POP (S) $\rightarrow$ D

For all adjacent nodes to D which are not already added in the stack, PUSH (S , X). Every neighbour element will be stored in the stack from right to left. Here, $X = (H)$ is the adjacent node to D.

H	E	C		
---	---	---	--	--

5. POP (S) $\rightarrow$ H

For all adjacent nodes to H which are not already added in the stack, PUSH (S , X). Every neighbour element will be stored in the stack from right to left. Here, $X = (F, G)$ is the adjacent node to H.

F	G	E	C	
---	---	---	---	--

6. POP (S) $\rightarrow$ F

For all adjacent nodes to F which are not already added in the stack, PUSH (S , X). Every neighbour element will be stored in the stack from right to left. Here, $X = (\Phi)$ is the adjacent node to F.

G	E	C		
---	---	---	--	--

7. POP (S) $\rightarrow$ G

For all adjacent nodes to G which are not already added in the stack, PUSH (S , X). Every neighbour element will be stored in the stack from right to left. Here, $X = (\Phi)$ is the adjacent node to G.

E	C			
---	---	--	--	--

8. POP (S) $\rightarrow$ E

For all adjacent nodes to E which are not already added in the stack, PUSH (S , X). Every neighbour element will be stored in the stack from right to left. Here, $X = (\Phi)$ is the adjacent node to E.

C				
---	--	--	--	--

9. POP (S) $\rightarrow$ C

For all adjacent nodes to C which are not already added in the stack, PUSH (S , X). Every neighbour element will be stored in the stack from right to left. Here, $X = (\Phi)$ is the adjacent node to C.

--	--	--	--	--

10. The stack is empty, which means the graph is traversed completely.

4.9 DYNAMIC PROGRAMMING

Dynamic programming is a method for solving complex problems by breaking them down into simpler sub-problems. It is applicable to problems exhibiting the properties of overlapping sub-problems and optimal substructure.

Dynamic programming algorithms are used for optimization (e.g. finding the shortest path between two points, or the fastest way to multiply many matrices). A dynamic programming algorithm will examine all possible ways to solve the problem and will pick the best solution.

4.9.1 Applications of Dynamic Programming

The following are the applications of dynamic programming:

1. Fibonacci series
2. 0/1 knapsack
3. Largest common subsequence
4. All-pair shortest path

4.9.2 Fibonacci Series

Fibonacci series is the sequence produced by adding the previous two numbers in the sequence. First two numbers in sequences are 0 1 or 1 1. The recurrence relation to find the next term in the sequence is given below:

$$F_n = F_{n-1} + F_{n-2}$$

with initial values $F_1 = 0$ and $F_2 = 1$ or $F_1 = 1$ and $F_2 = 1$.

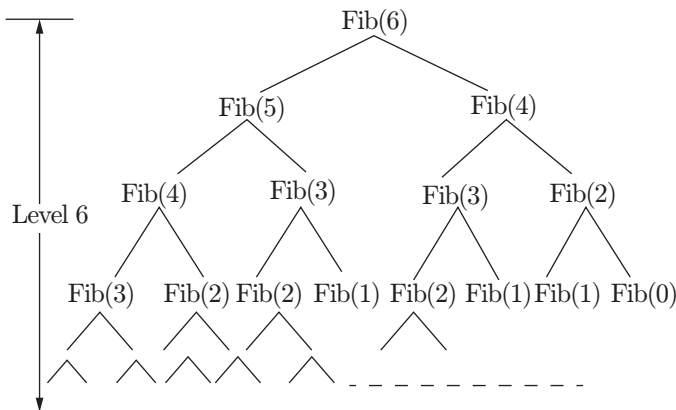
Example: Fibonacci series 0, 1, 1, 2, 3, 5, 8, 13, 21,

4.9.2.1 Recurrence Relation of Fibonacci

$$\text{fib}(n) = \begin{cases} 0, & \text{if } n < 0 \\ 1, & \text{if } n = 1 \\ \text{fib}(n-1) + \text{fib}(n-2), & \text{if } n > 1 \end{cases}$$

Example 4.19

The solution of Fibonacci through recursion tree is shown as follows:



Total number of nodes = $(2^n - 1)$.

So, the total number of function calls = 2^n , and one function call takes $O(1)$ time.

The total running time = $O(2^n)$.

Note: Most of the recursive programs contain very less number of distinct calls and more number of repeated sub-problems or function calls. These problems are known as overlapping sub-problems.

4.9.2.2 Dynamic Solution of Fibonacci

In dynamic programming technique, we solve every problem exactly once and store the results in memory. That result is used later to get an optimum output (Fig. 4.11).

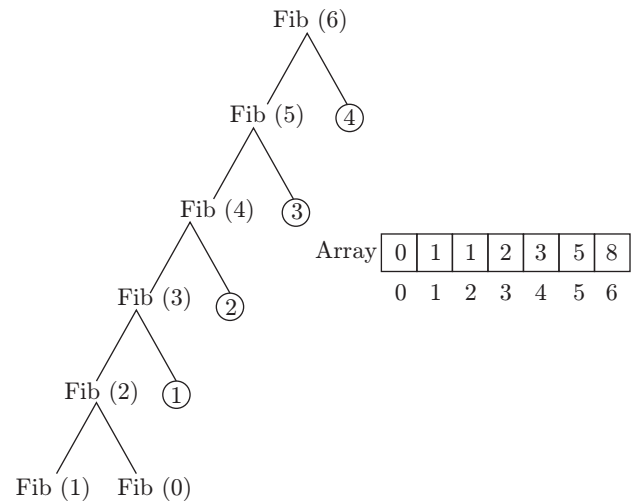


Figure 4.11 | Dynamic programming approach for Fibonacci.

Pseudocode for Dynamic Solution of Fibonacci Series

1. Algorithm Fast-Fibonacci(n)
2. Let fib[0] and fib[1] be 1
3. for each i from 2 to n, do
4. Let fib[i] = fib[i - 2] + fib[i - 1]
5. end of loop
6. return fib[n]

Time Complexity

1. Fibonacci series without dynamic programming [space = $O(n)$ and time = $O(2^n)$]
2. Fibonacci series with dynamic programming [space = $O(n)$ and time = $O(n)$]

4.9.3 0/1 Knapsack

One of the problems in combinatorial optimization is knapsack problem. In this problem, the target is to maximize the profit by choosing items. Each item is associated with a weight and profit. The collection of items is formed such that the chosen total weight should be less than or equal to the given limit. The profit can be as large as possible.

4.9.3.1 Recurrence Relation 0/1 Knapsack

The recurrence relation of 0/1 Knapsack problem is

$$KS(n, m) = \begin{cases} 0, & \text{if } m = 0 \text{ or } n = 0 \\ KS(n-1, m) & \text{if } m < w[n] \\ \text{Max}[\{KS(n-1, m-w[n] + p[n])\} & \text{otherwise} \\ \{KS(n-1, m) + 0\}], \end{cases}$$

4.9.3.2 Time Complexity

The time complexity of the knapsack problem is as follows:

1. Without dynamic programming = $O(2^n)$
2. With dynamic programming = $O(m^n)$

It is one of the NPC problems.

4.9.4 Longest Common Subsequence

A subsequence of a given sequence is just the given subsequence in which zero or more symbols are left out.

Let sequence $(S) = \{A, B, B, A, B, B\}$, subsequence $(S_1) = \{A, A\}$, subsequence $(S_2) = \{A, B, A, B\}$ and subsequence $(S_3) = \{B, A, B, A\}$.

4.9.4.1 Common Subsequence

In the given two sequences X and Y , we say that a sequence Z is a common subsequence of X and Y , if Z is a subsequence of both X and Y .

Example 4.20

$X = \{A, B, B, A, B, B\}$

$Y = \{B, A, A, B, A, A\}$

$Z = \{B, B, A\}$ is a common subsequence. But $Z = \{A, B, A, B\}$ is not a common subsequence.

4.9.4.2 Recurrence Relation of LCS

The recurrence relation of longest common subsequence is

$$LCS(i, j) = \begin{cases} 0, & \text{if } i = 0 \text{ or } j = 0 \\ 1 + LCS(i-1, j-1), & \text{if } X[i] = Y[j] \\ \text{Max}[\{LCS(i-1, j)\} \{LCS(i, j-1)\}], & \text{if } X[i] \neq Y[j] \end{cases}$$

4.9.4.3 Time Complexity

Complexity	LCS with Recursive Solution	LCS with Dynamic Programing
Time	If $(m = n)$ $LCS(m, n) = O(2^n)$ Else $LCS(m, n) = O(2^{m+n})$	$T(n) = O(m^n)$
Space	$O(m + n)$	$O(mn)$

4.10 ALL-PAIR SHORTEST PATH

Given a directed graph $G = (V, E)$, where each edge (v, w) has a non-negative cost $C[v, w]$, for all pairs of vertices (v, w) , find the cost of the lowest-cost path from v to w . It is a generalization of the single-sourceshortest-path problem.

4.10.1 Floyd's Algorithm

Let $A^k(i, j)$ be the minimum cost required to go from vertex i to vertex j for which all intermediate vertices are in the set $(0, 1, 2, 3, \dots, k)$ (Fig. 4.12).

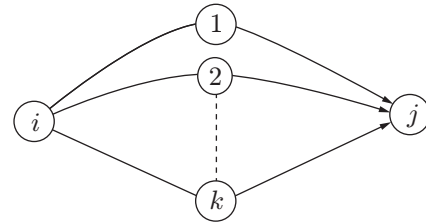


Figure 4.12 | Different paths from i to j .

4.10.1.1 Recurrence Relation of Floyd's Algorithm

The recurrence relation of Floyd's algorithm is

$$A^k(i, j) = \text{Min}\{A^{k-1}(i, j), A^{k-1}(i, k) + A^{k-1}(k, j)\}$$

4.10.1.2 Time Complexity

The running time complexity: $T(n) = O(V^3)$.

4.11 CONCEPTS OF COMPLEXITY CLASSES

4.11.1 P-Complexity Class

P in computational complexity theory is known as polynomial time or deterministic time. This is the most basic complexity class. This consists of all the problems that

are solvable in polynomial time. These problems can be solved in $O(n^k)$ time, where k is some constant. In class P , problems are efficiently solvable or traceable.

A language L is in class P iff there exists some Turing machine M such that:

1. Turing machine runs in polynomial time for all the inputs.
2. For all y in L , M must return 1, otherwise 0.

P includes many natural problems like linear programming, finding maximum matching and calculating the greatest common divisor. Prime number is also a part of class P now.

4.11.2 NP Complexity Class

NP class consists of problems that are verifiable in polynomial time. This class contains those languages that can be defined by second order logic.

Note: The most important open question in complexity theory is the $P = NP$ problem (Fig. 4.13).

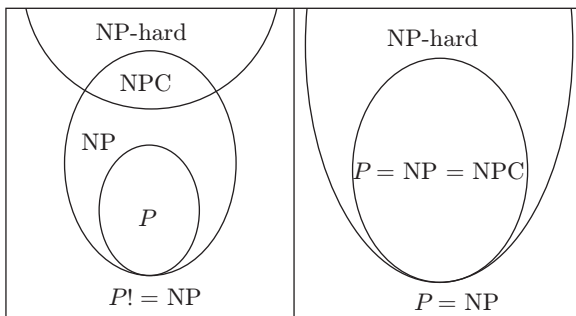


Figure 4.13 | Relation between P , NP , NP -complete and NP -hard problems.

4.11.3 NP-Complete

These are the hardest problems in NP . In the computational complexity theory, the complexity class NP -complete (abbreviated NP -C or NPC) is a class of decision problems. A decision problem L is NP -complete if it is in the set of NP problems and also in the set of NP -hard problems.

NP -complete is a subset of NP , the set of all decision problems whose solutions can be verified in polynomial time; NP may be equivalently defined as the set of decision problems that can be solved in polynomial time on a non-deterministic TM. A problem p in NP is also NP -complete if every other problem in NP can be transformed into p in polynomial time.

The following list contains some well-known problems that are NP -complete when expressed as decision problems:

1. Boolean satisfiability problem (SAT)
2. Knapsack problem
3. Hamiltonian path problem
4. Travelling salesman problem

5. Subgraph isomorphism problem
6. Subset sum problem
7. Clique problem
8. Vertex cover problem
9. Independent set problem
10. Dominating set problem
11. Graph colouring problem

4.11.4 NP-Hard

NP -hard is a class that includes problems which are “at least as hard as hardest problem in NP ”. A problem is said to be NP hard if

1. It is in NP
2. It is NP -complete

This means a problem H is NP -hard if any problem L has polynomial time reduction from $L \rightarrow H$. The problem can be a decision problem, optimization problem or a search problem.

Examples of NP -hard problem is the traveling salesman problem. It is an optimization problem of finding the least-cost cyclic route through all nodes of a weighted graph. There are decision problems that are NP -hard but not NP -complete, for example, the halting problem. This is the problem which asks, ‘given a program and its input, will it run forever?’ That is a yes/no question, so this is a decision problem.

Note:

1. Let ‘ A ’ be an NP -complete problem and ‘ B ’ be an unknown problem; if ‘ A ’ is a polynomial that reduces to ‘ B ’ ($A \leq_p B$), then ‘ B ’ is NP -hard.
2. Let ‘ A ’ be an NP -hard problem and ‘ B ’ be an unknown problem; if ‘ A ’ is a polynomial that reduces to ‘ B ’ ($A \leq_p B$), then ‘ B ’ is NP -hard.
3. Let ‘ A ’ be a P -class problem and ‘ B ’ be an unknown problem; if ($B \leq_p A$), then ‘ B ’ is a P -class problem.
4. Let ‘ A ’ be a P -class problem and ‘ B ’ be an unknown problem; if ($B \leq_p A$), then ‘ B ’ is an NP -class problem.

Problem 4.8: Both P and NP are closed under the operation of:

- | | |
|----------------------|-------------------|
| (A) Union | (B) Intersection |
| (C) Kleene’s closure | (D) None of these |

Solution: (D) Both P and NP are closed under concatenation and Kleene’s closure. The Kleene closure of a language L , denoted by L^* is defined as

$$\{w_1 w_2 \dots w_k \mid k \in N, w_i \in L \forall i = 1, 2, \dots, k\}.$$

P is closed under Kleene’s closure as for all $L \in P$, $L^* \in P$.

NP is closed under Kleene’s closure as for all $L \in NP$, $L^* \in NP$.

IMPORTANT FORMULAS

1. Theta notation (Θ): $0 < c_1(g(n)) \leq f(n) \leq c_2(g(n)) \quad \forall n \geq n_0$
2. Big-O notation (O): $0 < f(n) \leq c(g(n)) \quad \forall n \geq n_0$.
3. Omega notation (Ω): $0 < c(g(n)) \leq f(n) \quad \forall n \geq n_0$.
4. Small-o notation (o): $0 < f(n) < c(g(n)) \quad \forall n \geq n_0$.
5. Small omega notation (ω): $0 < c(g(n)) < f(n) \quad \forall n \geq n_0$.
6. Master theorem: $T(n) = aT(n/b) + f(n)$.

Case 1: $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.

Case 2: $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$.

Case 3: $f(n) = \Omega(n^{\log_b a + \epsilon})$.

$af(n/b) \leq cf(n)$ for some constant $c < 1$ and large n , then $T(n) = \Theta(f(n))$.
7. Linear search: $O(n)$.
8. Binary search: $\Theta(\log n)$.
9. Bubble sort: $O(n^2)$.
10. Selection sort: $O(n^2)$.
11. Insertion sort: $O(n^2)$.
12. Heap sort: $O(n \log n)$.
13. Merge sort: $O(n \log n)$.
14. Quick sort: $O(n^2)$ (worst case).
15. Quick sort: $O(n \log n)$ (average case).
16. Counting sort: $O(n + k)$.
17. If a graph has n number of vertices and it is k -regular, where k is a positive constant, then the number of edges $= (n \times k)/2$.
18. A complete graph having n vertices has $[n(n-1)]/2$ edges.
19. Number of simple graphs possible with n vertices $= 2^{[n(n-1)]/2}$.
20. Prim's algorithm: $O(E \log V)$.
21. Kruskal's algorithm: $O((V + E) \log V)$.
22. Dijkstra's algorithm: $O(E \log V)$.
23. Bellman-Ford algorithm: $O(V^3)$.
24. BFT and DFT: $O(V + E)$.

SOLVED EXAMPLES

1. Consider $T(n) = 6T(n/2) + n^3$; then $T(n)$ is equal to

- (a) $(n/2)$ (b) $O(n^3 \log n)$
 (c) $(n^2 \log n)$ (d) $O(n^3)$

Solution:

$T(n) = 6T(n/2) + n^3$; By using Master Theorem.

$$a = 6, b = 2, f(n) = n^3$$

$$g(n) = (n^{\log_b a}) = n^{\log_2 6} = \Theta(n^{2.58})$$

$f(n) = \Omega(n^{\log_2 6 + \epsilon})$, $f(n)$ is larger than $g(n)$, but is not a larger polynomial.

By case 3, $T(n) = \Theta(f(n)) = n^3$

Ans. (d)

2. Solve the following recurrence relation: $T(n) = 4T(n/2) + n$.

- (a) $O(n^2 \log n)$ (b) $O(n^2)$
 (c) $O(n/2)$ (d) $O(\log n)$

Solution:

$T(n) = 4T(n/2) + n$; By using Master Theorem.

$$a = 4, b = 2, f(n) = n$$

$$g(n) = (n^{\log_b a}) = n^{\log_2 4} = \Theta(n^2)$$

$f(n) = O(n^{\log_2 4 - \epsilon})$, $f(n)$ is smaller than $g(n)$, but is not a smaller polynomial.

By case 1, $T(n) = \Theta(n^2)$

Ans. (b)

3. Consider the following statements:

1. An algorithm is the number of steps to be performed to solve a problem.
2. An algorithm is the number of steps and their implementation in any language to a given problem to solve a problem.
3. To solve a given problem there may be more than one algorithm.

Which of the following is true?

- (a) 1 is correct. (b) 2 is correct.
(c) 1 and 3 are correct. (d) 2 and 3 are correct.

Solution: Both statements 1 and 3 are correct.

Ans. (c)

4. Compilers require efficient searching strategies for which they rely on

- (a) Hash tables (b) String matching
(c) Binary search tables (d) Binary search trees

Solution: Hash tables have less complexity, so they are used by the compiler for efficient searching.

Ans. (a)

5. Consider a binary search tree having n elements. The time required to search a given element will be:

- (a) $O(n)$ (b) $O(\log n)$
(c) $O(n^2)$ (d) $O(n \log n)$

Solution: The time required for search operation in a binary search depends on the height of the tree. In a complete binary tree with n nodes, search operation takes $\Theta(\log_2 n)$ in worst case scenario. But if all the nodes of tree are in linear chain, then the same operations will take $O(n)$ in worst-case scenario. So, option (a) is correct.

Ans. (a)

6. The average time required to perform a successful sequential search for an element in an array $A(1:n)$ is given by

- (a) $\log n$ (b) n^2
(c) $(n+1)/2$ (d) $(n \times n - 1)/2$

Solution: The average time required for sequential search or linear search in this case is $(n+1)/2$.

Ans. (c)

7. Which sort will operate in quadratic time relative to the number of elements in the array (on the average)?

- (a) Heap sort (b) Bubble sort
(c) Radix sort (d) Merge sort

Solution: The average time for bubble sort is $O(n^2)$, so it will operate in quadratic time.

Ans. (b)

8. Binary expression tree is traversed in _____ traversal.

- (a) Postorder (b) Preorder
(c) Both A and B (d) Reverse polish order

Solution: Binary tree is traversed in both preorder as well as postorder traversal.

Ans. (c)

9. The time complexity of searching an element in a linked list of length n will be

- (a) $O(n)$ (b) $O(n^2 - 1)$
(c) $O(\log n)$ (d) $O(\log n^2)$

Solution: The worst-case time complexity for searching an element in a linked list is $O(n)$, so option (a) is correct. If in the question it is not mentioned about (best or worst or average) time complexity then we always calculate as worst time complexity.

Ans. (a)

10. A hashing algorithm uses linear probing; the average search time will be less if the load factor

- (a) is less than one (b) equals one
(c) is greater than one (d) equals two

Solution: As load time is the ratio of number of records present to the total number of records, so is load time is less than 1, the probability of collisions decreases and hence, search time reduces.

Ans. (a)

11. If a node has K children in B tree, then the node contains exactly _____ keys.

- (a) K^2 (b) $K-1$ (c) $K+1$ (d) $\sqrt{K}$

Solution: The number of keys = $K-1$.

Ans. (b)

12. Big-O estimate for the factorial function, that is, $n!$, is given by

- (a) $O(n!)$ (b) $O(n^n)$
(c) $O(n \times n!)$ (d) $O(\sqrt{n})$

Solution: Factorial function $f(n) = n! = n(n-1) \times (n-2) \dots 1$

$n! = n(n-1)(n-2) \dots 1 \leq n \times n \dots \times n = (n^n)$

So, Big O Estimation of $f(n!) = O(n^n)$

Ans. (b)

13. Let A and B be two $n \times n$ matrices. The efficient algorithm to multiply the two matrices has the time complexity $O(n^x)$, where x is _____.

Solution: Strassen's surprising algorithm can multiply two $n \times n$ matrices in $\Theta(n^{\log_2 7})$ time, which is equal to $O(n^{2.81})$ time.

Ans. (2.81)

14. The searching technique in which less number of comparisons are done is

- (a) Binary searching
(b) Linear search searching
(c) Hashing
(d) Woline's search

Solution:

Search Complexity	Average Case	Worst Case
Binary	$O(\log n)$	$O(n)$
Hash table	$O(1)$	$O(n)$
Linear search	$O(n)$	$O(n)$

Thus, hash table takes less search time.

Ans. (c)

15. Which of the following statements is false about Prim's algorithm?

- (a) The time complexity is $O(E \log V)$ using a binary heap.
- (b) It may use binomial max-heap to represent the priority queue.
- (c) The time complexity is $O(E + V \log V)$ using a Fibonacci heap.
- (d) Initially the roots key and nodes are set to zero.

Solution:

- (a) True, The time complexity is $O(E \log V)$ using a binary heap.
- (c) True, Prim's algorithm's time complexity is $O(E + V \log V)$ using a Fibonacci heap.
- (d) True, Initial distance to the roots key and nodes is kept Zero.

Only option (b) is false.

Ans. (b)

16. Express the following recurrence relation in asymptotic notations:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

- (a) $\Theta(n \log n)$
- (b) $\Theta(n \log^2 n)$
- (c) $\Theta(n^2)$
- (d) $O(n^3)$

Solution: By using Master Theorem,

$$T(n) = 2T(n/2) + n$$

$$a = 2, b = 2, f(n) = n$$

$$g(n) = (n^{\log_b a}) = n^{\log_2 2} = \Theta(n)$$

$$\text{Order of } g(n) = \text{order of } f(n), \text{ so by case 2, } T(n) = \Theta(f(n) \cdot \log n) = \Theta(n \log n)$$

Ans. (a)

17. What is the height of n -node, k -ary heap?

- (a) $\Theta(\log_k n)$
- (b) $\Theta(nk)$
- (c) $\Theta(k \log_2 n)$
- (d) $\Theta(n/k)$

Solution: If h is the height of k -ary heap ($h \geq 2$) H , then the number of nodes ' n ' in H such that

$$\frac{k(h) - 1}{k - 1} < n \leq \frac{k(h + 1) - 1}{k - 1}$$

So $h = \Theta(\log_k n)$.

Ans. (a)

18. Solve the following given recurrence relation:
 $T(n) = 4T(n/2) + n^2$.

- (a) $\Theta(n^2)$
- (b) $\Theta(n^2 \log_2 n)$
- (c) $\Theta(n \log_2 n)$
- (d) None of the above

Solution: Using the master theorem, we have $a = 4$, $b = 2$ and $f(n) = n^2$. So

$$g(n) = n^{\log_b a} = n^{\log_2 4} = n^2$$

Here, order of $f(n)$ = order of $g(n)$, so we get

$$T(n) = O(n^{\log_b a} \log_2 n) = O(n^2 \log_2 n)$$

Ans. (b)

19. Solve the following given recurrence relation:
 $T(n) = 16T(n/4) + n^3$.

- (a) $\Theta(n \log_2 n)$
- (b) $\Theta(n^2 \log_2 n)$
- (c) $\Theta(n^3)$
- (d) None of the above

Solution: From the master theorem, we have $a = 16$, $b = 4$ and $f(n) = n^3$. So

$$g(n) = n^{\log_b a} = n^{\log_4 16} = n^2$$

Order of $f(n) >$ order of $g(n)$, so we get

$$T(n) = O(f(n))$$

Ans. (c)

20. Consider the following two functions:

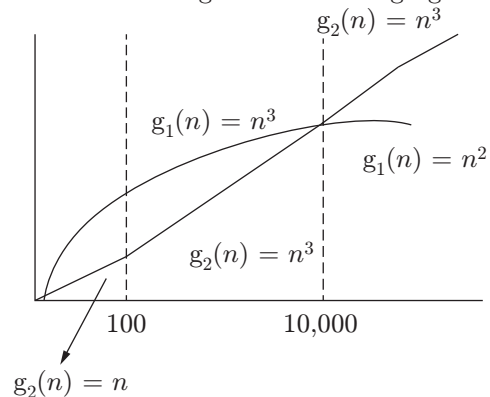
$$g_1(n) = \begin{cases} n^3, & \text{for } 0 \leq n \leq 10,000 \\ n^3, & \text{for } 0 \leq n \leq 10,000 \end{cases}$$

$$g_2(n) = \begin{cases} n, & \text{for } 0 \leq n \leq 10,000 \\ n^3, & \text{for } n > 10,000 \end{cases}$$

Which of the following is true?

- (a) $g_1(n)$ is $O(g_2(n))$.
- (b) $g_1(n)$ is $O(n^3)$.
- (c) $g_2(n)$ is $O(g_1(n))$.
- (d) $g_2(n)$ is $O(n)$.

Solution: Looking at the following figure:



Therefore, we get

$$n^2 \leq n^3 \quad \text{for } N \geq 10,000$$

$$g_1(n) = O(g_2(n))$$

Ans. (a)

GATE PREVIOUS YEARS' QUESTIONS

1. Consider the following three claims:

- (I) $(n + k)^m = \Theta(n^m)$, where k and m are constants
 (II) $2^{n+1} = O(2^n)$
 (III) $2^{2n+1} = O(2^n)$

Which of these claims is correct?

- (a) I and II (b) I and III
 (c) II and III (d) I, II and III

(GATE 2003: 1 Mark)

Solution: Option I: Consider $k = 1$; the expansion of $(n + 1)^m = f(n)$.

Option II: $2^{n+1} = 2 \cdot 2^n = O(2^n)$.

Option III: $2^{2n+1} = 2 \cdot 2^{2n} = O(2^{2n})$.

Options I and II are correct.

Ans. (a)

2. Let G be an arbitrary graph with n nodes and k components. If a vertex is removed from G , the number of components in the resultant graph must necessarily lie between

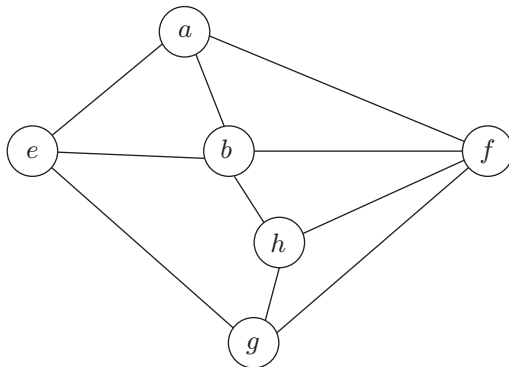
- (a) k and n (b) $k - 1$ and $k + 1$
 (c) $k - 1$ and $n - 1$ (d) $k + 1$ and $n - k$

Solution: Minimum components comes out to be when the independent vertex, which itself is a component, is removed. In that case, the number of components left is $k - 1$.

The maximum range comes when the removed vertex isolates all the vertices. In that case, the number of components comes out to be $n - 1$.

Ans. (c)

3. Consider the following graph:



Among the following sequences:

- (I) a b e g h f (II) a b f e h g
 (III) a b f h g e (IV) a f g h b e

Which are depth-first traversals of the above graph?

- (a) I, II and IV only (b) I and IV only
 (c) II, III and IV only (d) I, III and IV only

(GATE 2003: 1 Mark)

Solution: Option II is incorrect. There is no edge between f and e .

Ans. (d)

4. The usual $\Theta(n^2)$ implementation of insertion sort to sort an array uses linear search to identify the position where an element is to be inserted into the already sorted part of the array. If, instead, we use binary search to identify the position, the worst-case running time will

- (a) remain $\Theta(n^2)$ (b) become $\Theta(n(\log n)^2)$
 (c) become $\Theta(n \log n)$ (d) become $\Theta(n)$

(GATE 2003: 1 Mark)

Solution: Worst-case complexity will remain $O(n^2)$. This is because binary search is used to locate the position of element will take $\log_2(n)$ time, but inserting an element at its desired position will take $O(n)$ time in worst case.

Ans. (a)

5. In a heap with n elements with the smallest element at the root, the 7th smallest element can be found in time

- (a) $\Theta(n \log n)$ (b) $\Theta(n)$
 (c) $\Theta(\log n)$ (d) $\Theta(1)$

(GATE 2003: 1 Mark)

Solution: Minimum element as root means it is min heap. Searching 7th minimum element takes $O(\log n)$ time.

Ans. (c)

6. It has not been proved yet that $P = NP$.

Consider the language L defined as follows:

$$L = \begin{cases} (0+1)^* & \text{if } P = NP \\ \emptyset & \text{otherwise} \end{cases}$$

Which of the following statements is true?

- (a) L is recursive.
 (b) L is recursively enumerable but not recursive.
 (c) L is not recursively enumerable.
 (d) Whether L is recursive or not will be known after we find out if $P = NP$.

(GATE 2003: 1 Mark)

Solution: If a language L belongs to class P and class NP then it is called recursive.

Ans. (a)

7. A graph $G = (V, E)$ satisfies $|E| \leq 3|V| - 6$. The min-degree of G is defined as

$$\min_{v \in V} \{\text{degree}(v)\}$$

Therefore, min-degree of G cannot be

- (a) 3 (b) 4 (c) 5 (d) 6

(GATE 2003: 2 Marks)

Solution: Substituting value of $E = V(V-1)/2$, we get $V = 3$ or 4 . Thus, the minimum degree of each vertex could be 1 or 2. Hence, 3 cannot be the min degree.

Ans. (a)

8. Consider two languages L_1 and L_2 , each on the alphabet Σ . Let $f: \Sigma \rightarrow \Sigma$ be a polynomial time computable bijection such that $(\forall x)[x \in L_1 \text{ iff } f(x) \in L_2]$. Further, let f^{-1} also be polynomial time computable.

Which of the following CANNOT be true?

- (A) $L_1 \in P$ and L_2 is finite
(B) $L_1 \in NP$ and $L_2 \in P$
(C) L_1 is undecidable and L_2 is decidable
(D) L_1 is recursively enumerable and L_2 is recursive

(GATE 2003: 2 Marks)

Solution: L_1 is undecidable and L_2 is decidable are contradictory, as $L_1 \leq_p L_2$. Thus, if L_2 is decidable $\Rightarrow L_1$ is decidable.

Ans. (c)

Common Data Questions 8 and 9: In a permutation $a_1 \dots a_n$ of n distinct integers, an inversion is a pair (a_i, a_j) such that $i < j$ and $a_i > a_j$.

9. If all permutations are equally likely, what is the expected number of inversions in a randomly chosen permutation of $1 \dots n$?

- (a) $\frac{n(n-1)}{2}$ (b) $\frac{n(n-1)}{4}$
(c) $\frac{n(n+1)}{4}$ (d) $2n[\log_2 n]$

(GATE 2003: 2 Marks)

Solution: Inversion means $i < j$ and $a_i > a_j$ in pair (a_i, a_j) .

$$\text{Average number of inversions} = \frac{1}{2}({}^nC_2) = \frac{n(n-1)}{2}$$

Ans. (b)

10. What would be the worst-case time complexity of the insertion sort algorithm, if the inputs are restricted to permutations of $1 \dots n$ with at most n inversions?

- (a) $\Theta(n^2)$ (b) $\Theta(n \log n)$
(c) $\Theta(n^{1.5})$ (d) $\Theta(n)$

(GATE 2003: 2 Marks)

Solution: With at most n inversions, insertion sort takes $\log n$ time. So, if there are n elements, complexity will be $O(n \log n)$.

Ans. (a)

11. The cube root of a natural number n is defined as the largest natural number m such that $m^3 \leq n$. The complexity of computing the cube root of n (n is represented in binary notation) is

- (a) $O(n)$ but not $O(n^{0.5})$
(b) $O(n^{0.5})$ but not $O((\log n)^k)$ for any constant $k > 0$
(c) $O((\log n)^k)$ for some constant $k > 0$ but not $O((\log \log n)^m)$ for any constant $m > 0$
(d) $O((\log \log n)^k)$ for some constant $k > 0.5$ but not $O((\log \log n)^{0.5})$

(GATE 2003: 2 Marks)

Solution: The complexity of computing the cube root of n is $O(K^3 n^2 \log(B))$. Using binary search it takes $O((\log n)^k)$ for some constant $k > 0$ but not $O((\log n)^m)$ for any constant $m > 0$.

Ans. (c)

12. Let $G = (V, E)$ be an undirected graph with a sub-graph $G_1 = (V_1, E_1)$. Weights are assigned to edges of G as follows:

$$w(e) = \begin{cases} 0 & \text{if } e \in E_1 \\ 1 & \text{otherwise} \end{cases}$$

A single-source shortest path algorithm is executed on the weighted graph (V, E, w) with an arbitrary vertex v_1 of V_1 as the source. Which of the following can always be inferred from the path costs computed?

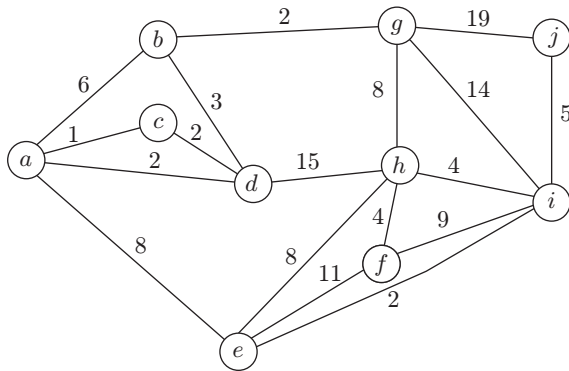
- (a) Number of edges in the shortest paths from v_1 to all vertices of G .
(b) G_1 is connected.
(c) V_1 forms a clique in G .
(d) G_1 is a tree.

(GATE 2003: 2 Marks)

Solution: Let us take four vertices v_1, v_2, v_3 and v_4 . The vertex v_1 is in G_1 such that v_1 is adjacent to v_3 and v_2 is adjacent to v_4 . Now after apply shortest path algorithm we get $v_1 v_2 = 1$, $v_1 v_4 = 1$ and $v_1 v_3 = 0$. From these results, we can say that v_1 is not a clique in G_1 .

Ans. (b)

13. What is the weight of a minimum spanning tree of the following graph?



- (a) 29 (b) 31 (c) 38 (d) 41
(GATE 2003: 2 Marks)

Solution: A minimum spanning tree will contain $n - 1$ edges. Here, $10 - 1 = 9$ edges will be included. Total weight = $1 + 2 + 2 + 2 + 3 + 4 + 4 + 5 + 8 = 31$.

Ans. (b)

14. The following are the starting and ending times of activities A, B, C, D, E, F, G and H, respectively, in chronological order:

$\langle a_s, b_s, a_s, a_e, d_s, a_e, e_s, f_s, b_e, d_e, g_s, e_e, f_e, h_s, g_e, h_e \rangle$

Here, x_s denotes the starting time and x_e denotes the ending time of activity X . We need to schedule the activities in a set of rooms available to us. An activity can be scheduled in a room only if the room is reserved for the activity for its entire duration. What is the minimum number of rooms required?

- (a) 3 (b) 4 (c) 5 (d) 6
(GATE 2003: 2 Marks)

Solution: We have $\langle a_s, b_s, c_s, a_e, d_s, c_e, e_s, f_s, b_e, d_e, g_s, e_e, f_e, h_s, g_e, h_e \rangle$. For this, the minimum number of rooms required is 4.

a_s	b_s	c_s	
d_s	b_s	c_s	
d_s	b_s	e_s	f_s
		e_s	f_s
g_s		e_s	f_s
g_s			
g_s	h_s		

Ans. (b)

15. Let $G = (V, E)$ be a directed graph with n vertices. A path from v_i to v_j in G is a sequence of vertices $(v_i, v_{i+1}, \dots, v_j)$ such that $(v_k, v_{k+1}) \in E$ for all k in i through $j - 1$. A simple path is a path in which no vertex appears more than once.

Let A be an $n \times n$ array initialized as follows:

$$A[j, k] = \begin{cases} 1 & \text{if } (j, k) \in E \\ 0 & \text{otherwise} \end{cases}$$

Consider the following algorithm:

```
for i = 1 to n
  for j = 1 to n
    for k = 1 to n
      A[j, k] = max (A[j, k], (A[j, i] + A[i, k]));
```

Which of the following statements is necessarily true for all j and k after terminal of the above algorithm?

- (a) $A[j, k] \leq n$.
(b) If $A[j, j] \geq n - 1$, then G has a Hamiltonian cycle.
(c) If there exists a path from j to k , $A[j, k]$ contains the longest path length from j to k .
(d) If there exists a path from j to k , every simple path from j to k contain most $A[j, k]$ edges.

(GATE 2003: 2 Marks)

Solution: If there exists a path from j to k , every simple path from j to k , it must contain $A[j, k]$ edges.

Ans. (d)

16. Consider the label sequences obtained by the following pairs of traversals on a labelled binary tree:

- (i) Preorder and postorder
(ii) Inorder and postorder
(iii) Preorder and inorder
(iv) Level order and postorder

Which of these pairs identifies a tree uniquely?

- (a) (i) only (b) (ii) and (iii)
(c) (iii) only (d) (iv) only

(GATE 2004: 2 Marks)

Solution: Only if we have inorder traversal with any one of postorder or preorder traversal of tree, can we uniquely identify a tree.

Ans. (b)

17. Two matrices M_1 and M_2 are to be stored in arrays A and B , respectively. Each array can be stored in either row-major or column-major order in contiguous memory locations. The time complexity of an algorithm to compute $M_1 M_2$ will be

- (a) best if A is in row-major and B is in column-major order.
(b) best if both are in row-major order.
(c) best if both are in column-major order.
(d) independent of the storage scheme.

(GATE 2004: 2 Marks)

Solution: Complexity of matrices multiplication is irrespective of storage scheme of the arrays.

Ans. (d)

18. Suppose each set is represented as a linked list with elements in an arbitrary order. Which of the operations among union, intersection, membership and cardinality will be the slowest?

(a) Union only
(b) Intersection and membership
(c) Membership and cardinality
(d) Union and intersection

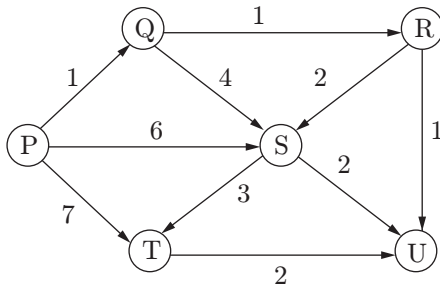
(GATE 2004: 2 Marks)

Solution: To calculate cardinality just counts the number of nodes in a list and for membership, just traverse the list and look for a match. So, cardinality and membership is not the slowest.

But For getting intersection and union of L_1 and L_2 , we have to perform search in both list L_1 and L_2 . So, they are the slowest one in the given list.

Ans. (d)

19. Suppose we run Dijkstra's single-source shortest-path algorithm on the following edge-weighted directed graph with vertex P as the source:



In what order do the nodes get included into the set of vertices for which the shortest path distances are finalized?

(a) P, Q, R, S, T, U (b) P, Q, R, U, S, T
(c) P, Q, R, U, T, S (d) P, Q, T, R, U, S

(GATE 2004: 2 Marks)

Solution: Nodes with minimum weight are included first. The sequence will be P, Q, R, U, S, T.

Ans. (b)

20. Let $\hat{A}[1, \dots, n]$ be an array storing a bit (1 or 0) at each location, and $f(m)$ is a function whose time complexity is $\Theta(m)$.

Consider the following program fragment written in a C-like language:

```
counter = 0;
for (i = 1 ; i <= n; i++)
{if (A[i] == 1) counter++;
else {f(counter); counter = 0;}}
}
```

The complexity of this program fragment is

(a) $\Omega(n^2)$ (b) $\Omega(n \log n)$ and $O(n^2)$
(c) $\Theta(n)$ (d) $o(n)$

(GATE 2004: 2 Marks)

Solution: The maximum time taken by a for loop is n . So, the complexity is $\Theta(n)$.

Ans. (c)

21. The time complexity of the following C function is (assume $n > 0$):

```
int recursive (int n) {
    if (n == 1)
        return (1);
    else
        return (recursive (n - 1) + recursive (n - 1));
}
```

(a) $O(n)$ (b) $O(n \log n)$
(c) $O(n^2)$ (d) $O(2^n)$

(GATE 2004: 2 Marks)

Solution: We have

$$T(n) = 2T(n-1), \quad n > 1$$

$$T(2) = 2T(1) = 2^1$$

$$T(3) = 2T(2) = 2^2$$

...

$$T(n) = 2^{n-1} = O(2^n)$$

Ans. (d)

22. The recurrence equation

$$T(1) = 1$$

$$T(n) = 2T(n-1) + n, \quad n \geq 2$$

evaluates to:

(a) $2^{n+1} - n - 2$ (b) $2^n - n$
(c) $2^{n+1} - 2n - 2$ (d) $2^n + n$

(GATE 2004: 2 Marks)

Solution: We have

$$T(n) = 2T(n-1) + n, \quad n \geq 2$$

$$T(2) = 2T(1) + 2 = 4$$

$$T(3) = 2T(2) + 3 = 11$$

$$T(4) = 2T(3) + 4 = 26$$

...

$$T(n) = 2^{n+1} - n - 2$$

Ans. (a)

23. Let $G_1 = (V, E_1)$ and $G_2 = (V, E_2)$ be connected graphs on the same vertex set V with more than two vertices. If $G_1 \cap G_2 = (V, E_1 \cap E_2)$ is not a connected graph, then the graph $G_1 \cup G_2 = (V, E_1 \cup E_2)$

(a) cannot have a cut vertex.
(b) must have a cycle.
(c) must have a cut-edge (bridge).
(d) has chromatic number strictly greater than those of G_1 and G_2 .

(GATE 2004: 2 Marks)

Solution: Option (b) is the most appropriate option.

Ans. (b)

24. A program takes as input a balanced binary search tree with n leaf nodes and computes the value of a function $g(x)$ for each node x . If the cost of computing $g(x)$ is $\min\{\text{no. of leaf-nodes in left-subtree of } x, \text{ number of leaf-nodes in right-subtree of } x\}$, then the worst-case time complexity of the program is

- (a) $O(n)$ (b) $O(n \log n)$
(c) $O(n^2)$ (d) $O(n^2 \log n)$

(GATE 2004: 2 Marks)

Solution: The complexity of $g(x)$ will be $O(n)$, using balanced binary tree.

Ans. (a)

25. The time complexity of computing the transitive closure of a binary relation on a set of n elements is known to be:

- (a) $O(n)$ (b) $O(n \log n)$
(c) $O(n^{3/2})$ (d) $O(n^3)$

(GATE 2005: 1 Mark)

Solution: The transitive closure of a binary relation can be computed using Warshall's algorithm. The complexity of finding it is $O(n^3)$.

Ans. (d)

26. Suppose $T(n) = 2T(n/2) + n$, $T(0) = T(1) = 1$.

Which one of the following is false?

- (a) $T(n) = O(n^2)$ (b) $T(n) = \Theta(n \log n)$
(c) $T(n) = \Omega(n^2)$ (d) $T(n) = O(n \log n)$

(GATE 2005: 2 Marks)

Solution: This can be solved using the master theorem.

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Here we have $a = 2$, $b = 2$ and $f(n) = n$. So, it is the second case of master theorem. Therefore,

$$T(n) = \Theta(n^{\log_b a} \log n)$$

$$T(n) = \Theta(n \log n)$$

Options (a), (b) and (d) are true because options (a) and (d) are indicating the upper bound which is greater than $\Theta(n \log n)$. But, option (c) is false which indicates that the lower bound is n^2 .

Ans. (c)

27. An undirected graph G has n nodes. Its adjacency matrix is given by an $n \times n$ square matrix whose (i) diagonal elements are 0s and (ii) non-diagonal elements are 1s. Which one of the following is true?

- (a) Graph G has no minimum spanning tree (MST).
(b) Graph G has a unique MST of cost $n - 1$.

- (c) Graph G has multiple distinct MSTs, each of cost $n - 1$.
(d) Graph G has multiple spanning trees of different costs.

(GATE 2005: 2 Marks)

Solution: Since diagonal elements are 0, there is no self-loop. And every node is connected to other $n - 1$ nodes.

Ans. (c)

28. Let $G(V, E)$ be an undirected graph with positive edge weights. Dijkstra's single source shortest path algorithm can be implemented using the binary heap data structure with time complexity:

- (a) $O(|V|^2)$ (b) $O(|E| + |V| \log |V|)$
(c) $O(|V| \log |V|)$ (d) $O((|E| + |V|) \log |V|)$

(GATE 2005: 2 Marks)

Solution: Complexity using Dijkstra's algorithm for single source shortest path algorithm is

$$O(E + V \log V)$$

Ans. (b)

29. Consider the following two problems on undirected graphs:

α : Given $G(V, E)$, does G have an independent set of size $|V| - 4$?

β : Given $G(V, E)$, does G have an independent set of size 5?

Which one of the following is TRUE?

- (a) α is in P and β is NP-complete
(b) α is NP-complete and β is in P
(c) α Both and β are NP-complete
(d) α Both and β are in P

(GATE 2005: 2 Marks)

Solution: Independent set decision problems of graph are NP-complete problems.

Ans. (c)

Linked Answer Questions 30 and 31: Consider the following C-function:

```
double foo (int n) {
    int i;
    double sum;
    if (n==0) return 1.0;
    else {
        sum = 0.0;
        for (i =0; i<n; i++)
            sum +=foo(i);
        return sum;
    } }
```

(GATE 2005: 2 Marks)

30. The space complexity of the above function is:

- (a) $O(1)$ (b) $O(n)$
(c) $O(n!)$ (d) $O(n^n)$

Solution: The space complexity is the number of elements to be stored. This will depend on the height of tree produced. Maximum storage required is $O(n)$.

Ans. (b)

31. Suppose we modify the above function `foo()` and store the values of `foo(i)`, $0 \leq i < n$, as and when they are computed. With this modification, the time complexity for function `foo()` is significantly reduced. The space complexity of the modified function would be:

- (a) $O(1)$ (b) $O(n)$
(c) $O(n^2)$ (d) $O(n!)$

Solution: Complexity will remain the same. With modification also, the required space will be same.

Ans. (b)

Linked Answer Questions 32 and 33: We are given 9 tasks T1, T2, ..., T9. The execution of each task requires one unit of time. We can execute one task at a time. Each task T_i has a profit P_i and a deadline d_i . Profit P_i is earned if the task is completed before the end of the $(d_i)^{\text{th}}$ unit of time.

Task	T1	T2	T3	T4	T5	T6	T7	T8	T9
Profit	15	20	30	18	18	10	23	16	25
Deadline	7	2	5	3	4	5	2	7	3

32. Are all tasks completed in the schedule that gives maximum profit?

- (a) All tasks are completed
(b) T1 and T6 are left out
(c) T1 and T8 are left out
(d) T4 and T6 are left out

(GATE 2005: 2 Marks)

Solution:

T2	T7	T9	T5	T3	T8	T1
----	----	----	----	----	----	----

T4 and T6 are left out.

Ans. (d)

33. What is the maximum profit earned?

- (a) 147 (b) 165 (c) 167 (d) 175

(GATE 2005: 2 Marks)

Solution: Maximum profit = $20 + 23 + 25 + 18 + 30 + 15 + 16 = 147$

Ans. (a)

34. Let S be an NP-complete problem and Q and R be two other problems not known to be in NP. Q is polynomial time reducible to S , and S is polynomial time reducible to R . Which one of the following statements is true?

- (a) R is NP-complete. (b) R is NP-hard.
(c) Q is NP-complete. (d) Q is NP-hard.

(GATE 2006: 1 mark)

Solution: R is NP-hard because S is polynomial time reducible to R .

Ans. (b)

35. A set X can be represented by an array $x[n]$ as follows:

$$x[i] = \begin{cases} 1 & \text{if } i \in X \\ 0 & \text{otherwise} \end{cases}$$

Consider the following algorithm in which x , y and z are Boolean arrays of size n :

```
algorithm zzz(x[], y[], z [])
{
  int i;
  for (i=0; i<n; ++i)
    z[i] = (x[i] ^ ~y[i]) V (~x[i] ^ y[i])
}
```

The set Z computed by the algorithm is:

- (a) $(X \text{ intersection } Y)$
(b) $(X \text{ union } Y)$
(c) $(X - Y) \text{ intersection } (Y - X)$
(d) $(X - Y) \text{ union } (Y - X)$

(GATE 2006: 2 Marks)

Solution:

$x[i] \wedge \sim y[i]$ = only 1's in x where corresponding entry in y is 0 $\rightarrow X - Y$

$\sim x[i] \wedge y[i]$ = only 1's in y where corresponding entry in x is 0 $\rightarrow Y - X$

The operator ' V ' results in a union of the above two sets.

Ans. (d)

36. Let SHAM3 be the problem of finding a Hamiltonian cycle in a graph $G = (V, E)$ with V divisible by 3 and DHAM3 be the problem of determining if a Hamiltonian cycle exists in such graphs. Which one of the following is true?

A	B	AoB
True	True	True
True	False	True
False	True	False
False	False	True

- (a) Both DHAM3 and SHAM3 are NP-hard
 (b) SHAM3 is NP-hard, but DHAM3 is not
 (c) DHAM3 is NP-hard, but SHAM3 is not
 (d) Neither DHAM3 nor SHAM3 is NP-hard

(GATE 2006: 2 Marks)

Solution: Both are NP-hard problems, and finding the Hamiltonian cycle is also an NP-hard problem.

Ans. (a)

37. Consider the following recurrence:

$$T(n) = 2T(\lceil \sqrt{n} \rceil) + 1, \quad T(1) = 1$$

Which one of the following is true?

- (a) $T(n) = \Theta(\log \log n)$ (b) $T(n) = \Theta(\log n)$
 (c) $T(n) = \Theta(\sqrt{\log n})$ (d) $T(n) = \Theta(n)$

(GATE 2006: 2 Marks)

Solution: Let $n = 2^m$. Then, we have

$$T(2^m) = T(2^{m/2}) + 1$$

Let $T(2^m) = S(m)$. Then, we have

$$S(m) = 2S\left(\frac{m}{2}\right) + 1$$

The above expression is a binary tree traversal recursion whose time complexity is $\theta(m)$. You can also prove this using the master theorem:

$$S(m) = \theta(m) = \theta(\log n)$$

since $n = 2^m$.

Now, let us go back to the original recursive function $T(n)$:

$$T(n) = T(2^m) = S(m) = \Theta(\log n)$$

Ans. (b)

38. Consider the polynomial $p(x) = a_0 + a_1x + a_2x^2 + a_3x^3$, where $a_i \neq 0$, for all i . The minimum number of multiplications needed to evaluate p on an input x is

- (a) 3 (b) 4 (c) 6 (d) 9

(GATE 2006: 2 Marks)

Solution: We have

$$p(x) = a_0 + a_1x + a_2x^2 + a_3x^2$$

$x^2 = 1$ multiplication

$$a_1x + a_2x^2 + a_3x^3 = 3 \text{ multiplications}$$

Total four multiplications are required.

Ans. (b)

39. To implement Dijkstra's shortest path algorithm on unweighted graphs so that it runs in linear time, the data structure to be used is:

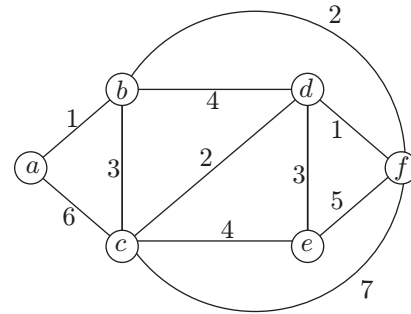
- (a) Queue (b) Stack (c) Heap (d) B-tree

(GATE 2006: 2 Marks)

Solution: If we use Queue data structure to implement Dijkstra's shortest path algorithm on unweighted graphs Then shortest path will be calculated in linear time $O(|V| + |E|)$. Basically we do BFS traversal of graph to get the shortest path.

Ans. (a)

40. Consider the following graph:



Which one of the following cannot be the sequence of edges added, in that order, to a minimum spanning tree using Kruskal's algorithm?

- (a) (a-b), (d-f), (b-f), (d-c), (d-e)
 (b) (a-b), (d-f), (d-c), (b-f), (d-e)
 (c) (d-f), (a-b), (d-c), (b-f), (d-e)
 (d) (d-f), (a-b), (b-f), (d-e), (d-c)

(GATE 2006: 2 Marks)

Solution: Edges are added according to increasing number of weights. In option (d), d-e is added before d-c.

Ans. (d)

41. The median of n elements can be found in $O(n)$ time. Which one of the following is correct about the complexity of quick sort, in which median is selected as pivot?

- (a) $O(n)$ (b) $O(n \log n)$
 (c) $O(n^2)$ (d) $O(n^3)$

(GATE 2006: 2 Marks)

Solution: Randomized quick sort:

$$T(n) = 2T(n/2) + 1 = O(n \log n).$$

Ans. (b)

42. Consider a weighted complete graph G on the vertex set $\{v_1, v_2, \dots, v_n\}$ such that the weight of the edge (v_i, v_j) is $2|i - j|$. The weight of a minimum spanning tree of G is

- (a) $n - 1$ (b) $2n - 2$ (c) nC_2 (d) 2

(GATE 2006: 2 Marks)

Solution: Weight of the minimum spanning tree is

$$2|2 - 1| + 2|3 - 2| + 2|4 - 3| + 2|5 - 4| + \dots + 2|n - (n - 1)| = 2n - 2$$

Ans. (b)

43. Let w be the minimum weight among all edge weights in an undirected connected graph. Let e be a specific edge of weight w . Which of the following is *false*?

- (a) There is a minimum spanning tree containing e .
- (b) If e is not in a minimum spanning tree T , then in the cycle formed by adding e to T , all edges have the same weight.
- (c) Every minimum spanning tree has an edge of weight w .
- (d) e is present in every minimum spanning tree.

(GATE 2007: 2 Marks)

Solution: Options (a), (b) and (c) are correct. But option (d) is incorrect as there may be many edges of weight w in the graph and edge e may not be in some of the minimum spanning trees.

Ans. (d)

44. Consider the following C code segment:

```
int IsPrime(n)
{
    int i, n;
    for(i=2; i<=sqrt(n); i++)
        if(n%i == 0)
            {printf('Not Prime\n'); return 0;}
    return 1;
}
```

Let $T(n)$ denote the number of times the `for` loop is executed by the program on input n . Which of the following is *true*?

- (a) $T(n) = O(\sqrt{n})$ and $T(n) = \Omega(\sqrt{n})$
- (b) $T(n) = O(\sqrt{n})$ and $T(n) = \Omega(1)$
- (c) $T(n) = O(n)$ and $T(n) = \Omega(\sqrt{n})$
- (d) None of the above

(GATE 2007: 2 Marks)

Solution: The `for` loop runs maximum for $\sqrt{n}$ and minimum for 1. Therefore,

$$T(n) = O(\sqrt{n}) \text{ and } T(n) = \Omega(1)$$

Ans. (b)

45. In an unweighted, undirected connected graph, the shortest path from a node S to every other node is computed most efficiently in terms of time complexity by

- (a) Dijkstra's algorithm starting from S
- (b) Warshall's algorithm
- (c) Performing a DFS starting from S
- (d) Performing a BFS starting from S

(GATE 2007: 2 Marks)

Solution: Time complexity of BFS is $O(|E| + |V|)$, whereas time complexity of Dijkstra's is $O(|V|^2 + E)$ and Warshall's algorithm is $O(|V|^3)$ and DFS cannot be used for finding shortest path.

Ans. (d)

46. In the following C function, let $n \geq m$:

```
int gcd(n, m)
{
    if (n%m == 0) return m;
    n = n%m;
    return gcd(m, n);
}
```

How many recursive calls are made by this function?

- (a) $\Theta(\log_2 n)$
- (b) $\Omega(n)$
- (c) $\Theta(\log_2 \log_2 n)$
- (d) $\Theta(\sqrt{n})$

(GATE 2007: 2 Marks)

Solution:

$$\text{Here, } T_n = 1 + \frac{1}{n}(T_0 T_1 + \dots + T_{n-1}) = \Theta(\log_2 n)$$

Ans. (a)

47. What is the time complexity of the following recursive function?

```
int DoSomething (int n)
{
    if (n <= 2)
        return 1;
    else
        return (DoSomething (floor(sqrt(n))) + n);
}
```

- (a) $\Theta(n)$
- (b) $\Theta(n \log_2 n)$
- (c) $\Theta(\log n)$
- (d) $\Theta(\log_2 \log_2 n)$

(GATE 2007: 2 Marks)

Solution: Recursive relation for the `DoSomething()` is

$$(T(n) = T(\sqrt{n}) + C_1) \text{ if } n > 2$$

$$\text{Let } n = 2^m, \text{ so } T(n) = T(2^m)$$

$$\text{Let } T(2^m) = S(m)$$

From the above two expressions, we have

$$T(n) = S(m)$$

$$S(m) = S(m/2) + C_1$$

$$S(m) = O(\log_2 m) \text{ (by Master Theorem)}$$

$$S(m) = O(\log_2 \log_2 n) \text{ (because } n = 2^m)$$

From the above, we have $S(m) = T(n) = O(\log_2 \log_2 n)$

Ans. (d)

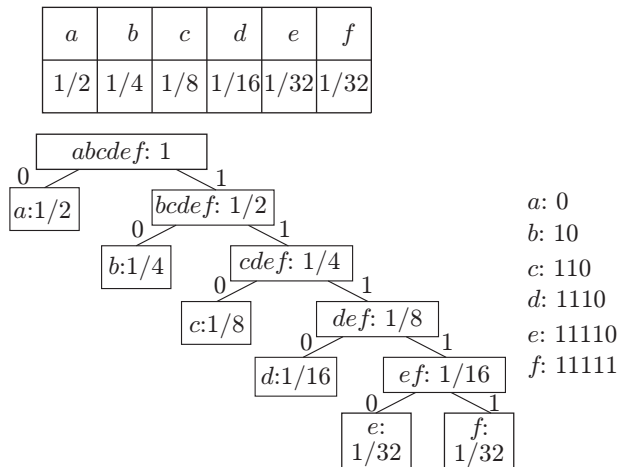
Linked Answer Questions 48 and 49: Suppose the letters a, b, c, d, e, f have probabilities $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{32}, \frac{1}{32}$, respectively.

48. Which of the following is the Huffman code for the letter a, b, c, d, e, f?

- (a) 0, 10, 110, 1110, 11110, 11111
- (b) 11, 10, 011, 010, 001, 000
- (c) 11, 10, 01, 001, 0001, 0000
- (d) 110, 100, 010, 000, 001, 111

(GATE 2007: 2 Marks)

Solution:



Ans. (a)

49. What is the average length of the correct answer to the above question?

(a) 3 (b) 2.1875 (c) 2.25 (d) 1.9375

(GATE 2007: 2 Marks)

Solution: The average length =

$$1 \times \frac{1}{2} + 2 \times \frac{1}{4} + 3 \times \frac{1}{8} + 4 \times \frac{1}{16} + 5 \times \frac{1}{32} + 5 \times \frac{1}{32} = 1.9375$$

Ans. (d)

50. The most efficient algorithm for finding the number of connected components in an undirected graph on n vertices and m edges has time complexity

(a) $\Theta(n)$ (b) $\Theta(m)$
(c) $\Theta(m + n)$ (d) $\Theta(mn)$

(GATE 2008: 1 Mark)

Solution: DFS can be used to find out the number of connected components in an undirected graph. The time complexity of DFS is $O(m + n)$.

Ans. (c)

51. The minimum number of comparisons required to determine if an integer appears more than $n/2$ times in a sorted array of n integers is

(a) $\Theta(n)$ (b) $\Theta(\log n)$
(c) $\Theta(\log^* n)$ (d) $\Theta(1)$

(GATE 2008: 1 Mark)

Solution: The minimum number of comparison would be 1 since if an element exists more than $n/2$ times in a sorted list than it has to be in middle. The middle index of an array can be reached in a constant time.

Ans. (a)

52. G is a graph on n vertices and $2n - 2$ edges. The edges of G can be partitioned into two edge-disjoint spanning trees. Which of the following is NOT true for G ?

(a) For every subset of k vertices, the induced sub-graph has at most $2k - 2$ edges.
(b) The minimum cut in G has at least two edges.
(c) There are two edge-disjoint paths between every pair of vertices.
(d) There are two vertex-disjoint paths between every pair of vertices.

(GATE 2008: 2 Marks)

Solution: Option (a) is true since the induced sub-graph on k vertices can be constructed as the union of the induced sub-graphs on k vertices of two edge disjoint spanning tree and each of the induced spanning trees has at most $k - 1$ edges.

Option (b) is true otherwise G cannot be partitioned into two edge disjoint spanning trees.

Option (c) is true otherwise G has minimum cut of one edge.

Option (d) is false.

Ans. (d)

53. Consider the Quicksort algorithm. Suppose there is a procedure for finding a pivot element which splits the list into two sub-lists each of which contains at least one-fifth of the elements. Let $T(n)$ be the number of comparisons required to sort n elements. Then

(a) $T(n) \leq 2T\left(\frac{n}{5}\right) + n$
(b) $T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{4n}{5}\right) + n$
(c) $T(n) \leq 2T\left(\frac{4n}{5}\right) + n$
(d) $T(n) \leq 2T\left(\frac{n}{2}\right) + n$

(GATE 2008: 2 Marks)

Solution: The recurrence relation is $T(n) = T(a) + T(b) + n, a \geq n/5, b \geq n/5$ and $a + b = n$

Thus, $a \leq 4n/5$ and $b \leq 4n/5$, so $T(n) \leq 2T\left(\frac{4n}{5}\right) + n$

Ans. (c)

54. The subset-sum problem is defined as follows: Given a set S of n positive integers and a positive integer W , determine whether there is a subset of S whose elements sum to W . An algorithm Q solves this problem in $O(nW)$ time. Which of the following statements is false?

- (A) Q solves the subset-sum problem in polynomial time when the input is encoded in unary
 (B) Q solves the subset-sum problem in polynomial time when the input is encoded in binary
 (C) The subset sum problem belongs to the class NP
 (D) The subset sum problem is NP-hard

(GATE 2008: 2 Marks)

Solution: Since we know that subset sum is a NP-hard problem, thus problem may take linear time when the input encoded is unary.

Ans. (b)

Linked Answer Questions 55 and 56: The subsetsum problem is defined as follows. Given a set of n positive integers, $S = \{a_1, a_2, a_3, \dots, a_n\}$ and positive integer W , is there a subset of S whose elements sum to W ? A dynamic program for solving this problem uses a two-dimensional Boolean array X , with n rows and $W + 1$ columns. $X[i, j]$, $1 \leq i \leq n$, $0 \leq j \leq W$, is *true* if and only if there is a subset of $\{a_1, a_2, \dots, a_i\}$ whose elements sum to j .

55. Which of the following is valid for $2 \leq i \leq n$ and $ai \leq j \leq W$?

- (a) $X[i, j] = X[i - 1, j] \vee X[i, j - a_i]$
 (b) $X[i, j] = X[i - 1, j] \vee X[i - 1, j - a_i]$
 (c) $X[i, j] = X[i - 1, j] \vee X[i, j - a_i]$
 (d) $X[i, j] = X[i - 1, j] \vee X[i - 1, j - a_i]$

(GATE 2008: 2 Marks)

Solution: $X[i, j]$ ($2 \leq i \leq n$ and $a_i \leq j \leq W$) is true if any of the following is true:

- (1) Sum of weights excluding a_i is equal to j , that is, if $X[i - 1, j]$ is true.
 (2) Sum of weights including a_i is equal to j , that is, if $X[i - 1, j - a_i]$ is true so that we get $(j - a_i) + a_i$ as j .

Ans. (b)

56. Which entry of the array X , if *true*, implies that there is a subset whose elements sum to W ?

- (a) $X[1, W]$ (b) $X[n, 0]$
 (c) $X[n, W]$ (d) $X[n - 1, n]$

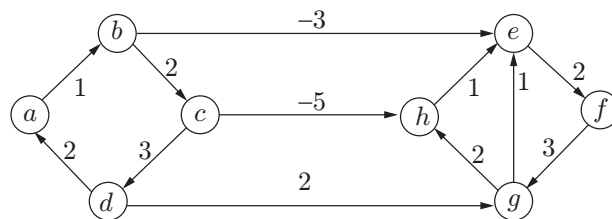
(GATE 2008: 2 Marks)

Solution:

If we get the entry $X[n, W]$ as true, then there is a subset of $\{a_1, a_2, \dots, a_n\}$ that has sum as W .

Ans. (c)

57. Dijkstra's single-source shortest path algorithm when run from vertex a in the following graph computes the correct shortest path distance to



- (a) Only vertex a
 (b) Only vertices a, e, f, g and h
 (c) Only vertices a, b, c and d
 (d) All of the vertices

(GATE 2008: 2 Marks)

Solution: Following the Dijkstra's algorithm's single-source shortest paths, we can reach all the vertices here.

Taking 'a' as the source: minimum distance is computed by checking all the adjacent nodes, as follows:

$$D(a) = 0$$

$$D(b) = \text{cost}(a \rightarrow b) = 1$$

Here, the minimum distance from source is computed, but since there is only one adjacent node, there is no need to check any other node for minimum.

Now, here b node has two adjacent nodes e and c . But since $\text{cost}(b \rightarrow e)$ has a negative weight, we ignore that path as Dijkstra does not entertain negative weights. Thus, we will proceed with node c :

$$D(c) = D(b) + \text{cost}(b \rightarrow c) = 2 + 1 = 3$$

Also c node could be reached only through b , so that is the only possible minimum weight of c .

As similar to node b , node c is also left with one child d due to negative edge removal $c \rightarrow h$:

$$D(d) = D(c) + \text{cost}(c \rightarrow d) = 3 + 3 = 6$$

Similarly, proceeding further the same way, we can reach all nodes and compute the shortest path.

Ans. (d)

58. Consider the following functions:

$$f(n) = 2^n \quad g(n) = n! \quad h(n) = n^{\log n}$$

Which of the following statements about the asymptotic behaviour of $f(n)$, $g(n)$ and $h(n)$ is true?

- (a) $f(n) = O(g(n))$; $g(n) = O(h(n))$
 (b) $f(n) = \Omega(g(n))$; $g(n) = O(h(n))$
 (c) $g(n) = O(f(n))$; $h(n) = O(f(n))$
 (d) $h(n) = O(f(n))$; $g(n) = \Omega(f(n))$

(GATE 2008: 2 Marks)

Solution: The functions can be arranged in an increasing order as $h(n) < f(n) < g(n)$. Hence, option (d) is the correct one.

Ans. (d)

59. Let X be a problem that belongs to the class NP. Then which one of the following is *true*?

- (a) There is no polynomial time algorithm for X .
- (b) If X can be solved deterministically in polynomial time, then $P = NP$.
- (c) If X is NP-hard, then it is NP-complete.
- (d) X may be undecidable.

(GATE 2009: 1 mark)

Solution: Option (a) is incorrect because NP includes both P and NPC.

Option (b) is also incorrect because X may belong to P but it does not mean that $P = NP$.

Option (c) is correct because NP-complete set is intersection of NP and NP-hard sets.

Option (d) is incorrect because all NP problems are decidable in finite set of operations.

Ans. (c)

60. What is the minimum number of swaps required to sort n elements using selection sort, in the worst case?

- (a) $\Theta(n)$
- (b) $\Theta(n \log n)$
- (c) $\Theta(n^2)$
- (d) $\Theta(n^2 \log n)$

(GATE 2009: 1 mark)

Solution: Selection sort select the minimum element and the swap with first position. In the worst case, all n elements will be swapped.

Ans. (a)

61. The keys 12, 18, 13, 2, 3, 23, 5 and 15 are inserted into an initially empty hash table of length 10 using open addressing with hash function $h(k) = k$ and linear probing. What is the resultant hash table?

(A)	(B)	(C)	(D)
0	0	0	0
1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4
5	5	5	5
6	6	6	6
7	7	7	7
8	8	8	8
9	9	9	9

(GATE 2009: 2 Marks)

Solution:

$$12 \bmod 10 \rightarrow 2$$

$$18 \bmod 10 \rightarrow 8$$

$$13 \bmod 10 \rightarrow 3$$

$$2 \bmod 10 \rightarrow 2 \text{ [2, 3 are occupied]} \text{ so will go to 4}$$

$$3 \bmod 10 \rightarrow 3 \text{ [3, 4 are occupied]} \text{ so will go to 5}$$

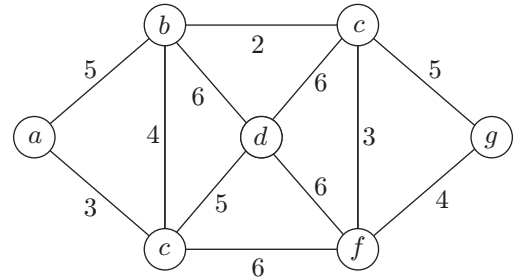
$$23 \bmod 10 \rightarrow 3 \text{ [3, 4, 5 are occupied]} \text{ so will go to 6}$$

$$5 \bmod 10 \rightarrow 5 \text{ [5, 6 occupied]} \text{ so will go to 7}$$

$$15 \bmod 10 \rightarrow 5 \text{ [5, 6, 7, 8 are occupied]} \text{ will go to 9}$$

Ans. (c)

62. Consider the following graph:



(GATE 2009: 2 Marks)

Which one of the following is NOT the sequence of edges added to the minimum spanning tree using Kruskal's algorithm?

- (A) (b, e) (e, f) (a, c) (b, c) (f, g) (c, d)
- (C) (b, e) (a, c) (e, f) (b, c) (f, g) (c, d)
- (B) (b, e) (e, f) (a, c) (f, g) (b, c) (c, d)
- (D) (b, e) (e, f) (b, c) (a, c) (f, g) (c, d)

(GATE 2009: 2 Marks)

Solution: In Kruskal's algorithm edges are chosen in increasing order of weights. So, (b, e) = 2, (e, f) = 3, (b, c) = 4, (a, c) = 3, (f, g) = 4 and (c, d) = 5.

Ans. (d)

63. The running time of an algorithm is represented by the following recurrence relation:

$$\text{if } n \leq 3 \text{ then } T(n) = n \\ \text{else } T(n) = T(n/3) + cn$$

Which one of the following represents the time complexity of the algorithm?

- (a) $\Theta(n)$
- (b) $\Theta(n \log n)$
- (c) $\Theta(n^2)$
- (d) $\Theta(n^2 \log n)$

(GATE 2009: 2 Marks)

Solution: As per the master theorem, case 3, we have $a = 1$, $b = 3$ and $f(n) = cn$. Thus

$$\log_b a = \log_3 1 = 0$$

$$n^0 < f(n)$$

So, the answer is $\Theta(n)$.

Ans. (a)

64. In quick sort, for sorting n elements, the $(n/4)$ th smallest element is selected as pivot using an $O(n)$ time algorithm. What is the worst-case time complexity of the quick sort?

- (a) $\Theta(n)$
- (b) $\Theta(n \log n)$
- (c) $\Theta(n^2)$
- (d) $\Theta(n^2 \log n)$

(GATE 2009: 2 Marks)

Solution: The recursive expression is

$$T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + cn$$

$$T(n) = \Theta(n \log n)$$

This expression is of randomized quick sort.

Ans. (b)

Common Data Questions 65 and 66: A subsequence of a given sequence is just the given sequence with some elements (possibly none or all) left out. We are given two sequences $X[m]$ and $Y[n]$ of lengths m and n , respectively, with indexes of X and Y starting from 0.

- 65.** We wish to find the length of the longest common subsequence (LCS) of $X[m]$ and $Y[n]$ as $l(m, n)$, where an incomplete recursive definition for the function $l(i, j)$ to compute the length of the LCS of $X[m]$ and $Y[n]$ is given as follows:

1 $(i, j) \geq 0$, if either $i=0$ or $j=0$
 = expr1, if $i, j > 0$ and $X[i-1] = Y[j-1]$
 = expr2, if $i, j > 0$ and $X[i-1] \neq Y[j-1]$

Which one of the following options is correct?

- (a) $\text{expr1} \equiv (i-1, j) + 1$
 (b) $\text{expr1} \equiv l(i, j-1)$
 (c) $\text{expr2} \equiv \max(l(i-1, j), l(i, j-1))$
 (d) $\text{expr2} \equiv \max(l(i-1, j-1), l(i, j))$

(GATE 2009: 2 Marks)

Solution: LCS problem is solved using dynamic programming in which every row is dependent upon previous row and column. So, if two characters at i^{th} row and j^{th} column do not match then the length is calculated as:

$$\text{expr2} = \max(l(i-1, j), l(i, j-1))$$

The values of $l(i, j)$ could be obtained by dynamic programming based on the correct recursive definition of $l(i, j)$ of the form given above, using an array $L[M, N]$, where $M = m + 1$ and $N = n + 1$, such that $L[i, j] = l(i, j)$.

Ans. (c)

- 66.** Which one of the following statements would be *true* regarding the dynamic programming solution for the recursive definition of $l(i, j)$?

- (a) All elements L should be initialized to 0 for the values of $l(i, j)$ to be properly computed.
 (b) The values of $l(i, j)$ may be computed in a row-major order or column-major order of $L(M, N)$.
 (c) The values of $l(i, j)$ cannot be computed in either row-major order or column-major order of $L(M, N)$.
 (d) $L[p, q]$ needs to be computed before $L[r, s]$ if either $p < r$ or $q < s$.

(GATE 2009: 2 Marks)

Solution: The value can be computed in either row-major or column-major order.

Ans. (b)

- 67.** Two alternative packages A and B are available for processing a database having 10^k records. Package A requires $0.0001 n^2$ time units and package B requires $10n \log_{10} n$ time units to process n records. What is the smallest value of k for which the package B will be preferred over A?

- (a) 12 (b) 10
 (c) 6 (d) 5

(GATE 2010: 1 mark)

Solution:

$$0.0001 n^2 < 10 n \log n \Rightarrow$$

$$10^{-5} n < \log n \Rightarrow 10^{-5} \times 10^k < k \text{ (as } \log_{10} n = k)$$

Hence, $k - 5 > 0$ or $k > 5$. Thus, for $k = 6$, B will be preferred.

Ans. (c)

- 68.** The weight of a sequence $a_0, a_1, \dots, a_{n-1}$ of real numbers is defined as

$$a_0 + a_1/2 + \dots + a_{n-1}/2^{n-1}$$

A subsequence of a sequence is obtained by deleting some elements from the sequence, keeping the order of the remaining elements the same. Let X denote the maximum possible weight of a subsequence of $a_0, a_1, \dots, a_{n-1}$ and Y the maximum possible weight of a subsequence of $a_1, a_2, \dots, a_{n-1}$. Then X is equal to

- (a) $\max(Y, a_0 + Y)$ (b) $\max(Y, a_0 + Y)$
 (c) $\max(Y, a_0 + 2Y)$ (d) $a_0 + Y/2$

Solution: This involves dynamic programming in algorithms. Given that

X = Maximum weight from the sequence $(a_0, a_1, a_2,$

$$\dots, a_{n-1}) = a_0 + \frac{a_1}{2} + \frac{a_2}{4} + \dots + \frac{a_{n-1}}{2^{n-1}}$$

Y = Maximum weight from the sequence $(a_1, a_2, \dots$

$$a_{n-1}) = a_1 + \frac{a_2}{2} + \frac{a_3}{4} + \dots + \frac{a_{n-1}}{2^{n-2}}$$

Here, we have to find X in terms of Y .

$$\text{So, } X = a_1 + \frac{a_2}{2} + \frac{a_3}{4} + \dots + \frac{a_{n-1}}{2^{n-2}} = Y$$

$$\text{if } a_0 + \frac{a_1}{2} + \frac{a_2}{4} + \dots + \frac{a_{n-1}}{2^{n-1}}$$

$$< a_1 + \frac{a_2}{2} + \frac{a_3}{4} + \dots + \frac{a_{n-1}}{2^{n-2}}$$

OR

$$X = a_0 + \frac{a_1}{2} + \frac{a_2}{4} + \cdots + \frac{a_{n-1}}{2^{n-1}} = a_0 + Y/2 \text{ if}$$

$$a_0 + \frac{a_1}{2} + \frac{a_2}{4} + \cdots + \frac{a_{n-1}}{2^{n-1}} > a_1 + \frac{a_2}{2} + \frac{a_3}{4} + \cdots + \frac{a_{n-1}}{2^{n-2}}$$

Hence, we sum up as: $X = \max(Y, a_0 + Y/2)$

Ans. (b)

Common Data Questions 69 and 70: Consider a complete undirected graph with vertex set $\{0, 1, 2, 3, 4\}$. Entry W_{ij} in the following matrix W is the weight of the edge $\{i, j\}$:

$$W = \begin{pmatrix} 0 & 1 & 8 & 1 & 4 \\ 1 & 0 & 12 & 4 & 9 \\ 8 & 12 & 0 & 7 & 3 \\ 1 & 4 & 7 & 0 & 2 \\ 4 & 9 & 3 & 2 & 0 \end{pmatrix}$$

69. What is the minimum possible weight of a spanning tree T in this graph such that vertex 0 is a leaf node in the tree T ?

- (a) 7 (b) 8
(c) 9 (d) 1

(GATE 2010: 2 Marks)

Solution: To get the minimum spanning tree with vertex 0 as leaf, first remove 0th row and 0th column and then get the minimum spanning tree (MST) of the remaining graph. Once we have MST of the remaining graph, connect the MST to vertex 0 with the edge with minimum weight (we have two options as there are two 1's in the 0th row).

Ans. (d)

70. What is the minimum possible weight of a path P from vertex 1 to vertex 2 in this graph such that P contains at most three edges?

- (a) 7 (b) 8
(c) 9 (d) 10

(GATE 2010: 2 Marks)

Solution: Path: $1 \rightarrow 0 \rightarrow 4 \rightarrow 2$ Weight: $1 + 4 + 3 = 8$

Ans. (b)

Linked Answer Questions 71 and 72: A hash table of length 10 uses open addressing with hash

function $h(k) = k \bmod 10$, and linear probing. After inserting 6 values into an empty hash table, the v is as shown below:

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	

71. Which one of the following choices gives a possible order in which the key values could have been inserted in the table?

- (a) 46, 42, 34, 52, 23, 33
(b) 34, 42, 23, 52, 33, 46
(c) 46, 34, 42, 23, 52, 33
(d) 42, 46, 33, 23, 34, 52

(GATE 2010: 2 Marks)

Solution: Check the options according to linear probing.

The correct sequence is 46, 34, 42, 23, 52, 33.

Ans. (c)

72. How many different insertion sequences of the key values using the same hash function and linear probing will result in the hash table shown above?

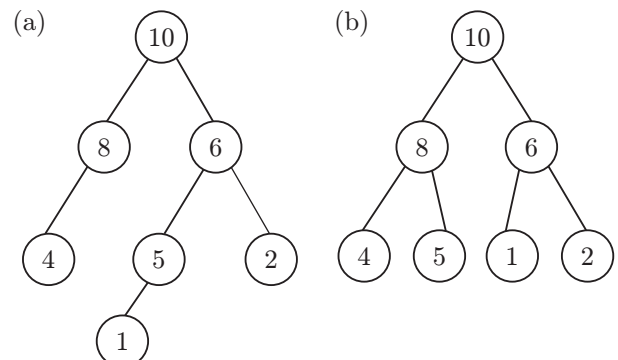
- (a) 10 (b) 20
(c) 30 (d) 40

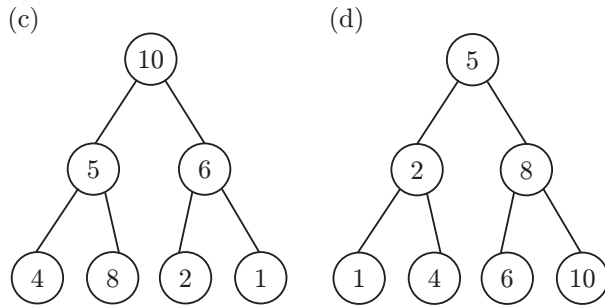
(GATE 2010: 2 Marks)

Solution: We have $n(n-1) = 6 \times 5 = 30$

Ans. (c)

73. A max-heap is a heap where the value of each parent is greater than or equal to the value of its children. Which of the following is a max-heap?





(GATE 2011: 1 mark)

Solution: Heap is also a complete binary tree.

Ans. (b)

74. An algorithm to find the length of the longest monotonically increasing sequence of numbers in an array $A[0:n-1]$ is given below.

Let L_i denote the length of the longest monotonically increasing sequence starting at index i in the array.

Initialize $L_{n-1} = 1$.

For all i such that $0 \leq i \leq n-2$

$$L_i = \begin{cases} 1 + L_{i+1} & \text{if } A[i] < A[i+1] \\ 1 & \text{otherwise} \end{cases}$$

Finally, the length of the longest monotonically increasing sequence is $\text{Max}(L_0, L_1, \dots, L_{n-1})$.

Which of the following statements is *true*?

- (a) The algorithm uses dynamic programming paradigm.
- (b) The algorithm has a linear complexity and uses branch and bound paradigm.
- (c) The algorithm has a non-linear polynomial complexity and uses branch and bound paradigm.
- (d) The algorithm uses divide and conquer paradigm.

(GATE 2011: 1 Mark)

Solution: In the given algorithm, length of the longest monotonically increasing sequence is calculated as L_i in terms of L_{i+1} . So, it uses dynamic programming technique.

Ans. (a)

75. We are given a set of n distinct elements and an unlabelled binary tree with n nodes. In how many ways can we populate the tree with the given set so that it becomes a binary search tree?

- (a) 0 (b) 1 (c) $n!$ (d) $\frac{1}{n+1} 2^n C_n$

(GATE 2011: 2 Marks)

Solution: We can populate only in one way because minimum value will be in the leftmost node, and the maximum value will be in the rightmost node. Recursively, we can define for other nodes.

Ans. (b)

76. Which of the given options provides the increasing order of asymptotic complexity of functions f_1 , f_2 , f_3 and f_4 ?

$$f_1(n) = 2^n; f_2(n) = n^{3/2}; f_3(n) = n \log_2 n;$$

$$f_4(n) = n^{\log_2 n}$$

(a) f_3, f_2, f_4, f_1

(b) f_3, f_2, f_1, f_4

(c) f_2, f_3, f_1, f_4

(d) f_2, f_3, f_4, f_1

(GATE 2011: 2 Marks)

Solution: Take n as a large value and evaluate functions. Let $n = 32$, then

$$f_1 = 4294967296$$

$$f_2 = 181.019$$

$$f_3 = 160$$

$$f_4 = 33554432$$

$$f_3 < f_2 < f_4 < f_1$$

Ans. (a)

77. Consider a relational table r with sufficient number of records, having attributes $A_1, A_2, \dots, A_n$ and let $1 \leq p \leq n$. Two queries $Q1$ and $Q2$ are given as follows:

$$Q1: \Pi_{A_1, \dots, A_p} (\sigma_{A_p = c}(r)), \text{ where } c \text{ is a constant}$$

$$Q2: \Pi_{A_1, \dots, A_p} (\sigma_{c_1 \leq A_p \leq c_2}(r)), \text{ where } c_1 \text{ and } c_2 \text{ are constants}$$

The database can be configured to do ordered indexing on A_p or hashing on A_p . Which of the following statements is *true*?

- (a) Ordered indexing will always outperform hashing for both queries.
- (b) Hashing will always outperform ordered indexing for both queries.
- (c) Hashing will outperform ordered indexing on $Q1$, but not on $Q2$.
- (d) Hashing will outperform ordered indexing on $Q2$, but not on $Q1$.

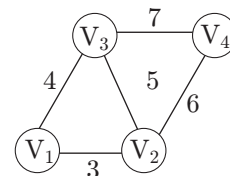
(GATE 2011: 2 Marks)

Solution:

Hashing will outperform ordered indexing on $Q1$, but not $Q2$.

Ans. (c)

Linked Answer Questions 78 and 79: An undirected graph $G(V, E)$ contains n ($n > 2$) nodes named $v_1, v_2, \dots, v_n$. Two nodes v_i and v_j are connected if and only if $0 < i - j \leq 2$. Each edge (v_i, v_j) is assigned a weight $i + j$. A sample graph with $n = 4$ is shown as follows:



78. What will be the cost of the minimum spanning tree (MST) of such a graph with n nodes?

- (a) $(1/12)(11n^2 - 5n)$ (b) $n^2 - n + 1$
 (c) $6n - 11$ (d) $2n + 1$

(GATE 2011: 2 Marks)

Solution:

Let us have 2 nodes in the tree, then minimum spanning tree is

$$v_1 - v_2$$

Total weight = 3

Minimum spanning tree for 3 nodes would be

$$\begin{array}{c} v_1 - v_2 \\ | \\ v_3 \end{array}$$

Total weight = $3 + 4 = 7$

Minimum spanning tree for 4 nodes would be

$$\begin{array}{c} v_1 - v_2 - v_4 \\ | \\ v_3 \end{array}$$

Total weight = $3 + 4 + 6 = 13$

From the above examples, we observe that when we add i^{th} node, the weight of spanning tree increases by $2i - 2$. Let $T(n)$ be the weight of minimum spanning tree.

$$T(n) = \begin{cases} T(n-1) + (2n-2), & n > 2 \\ 3, & n = 1 \\ 0, & n = 2 \end{cases}$$

The recurrence can be written as sum of series $(2n-2) + (2n-4) + (2n-6) + (2n-8) + \dots + 3$, and solution of this recurrence is $n^2 - n + 1$.

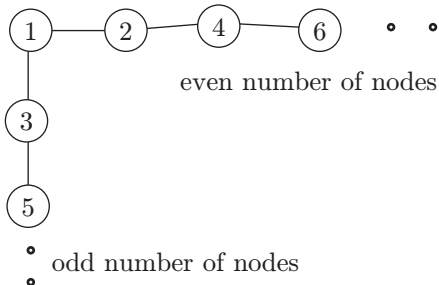
Ans. (b)

79. The length of the path from v_5 to v_6 in the MST of previous question with $n = 10$ is

- (a) 11 (b) 25 (c) 31 (d) 41

(GATE 2011: 2 Marks)

Solution: Basic structure of all MSTs with more than 5 nodes (for the above given graph specification) would be following:



MST with 6 nodes have weight = $3 + 4 + 6 + 8 + 10 = 31$

Ans. (c)

80. Assuming $P \neq \text{NP}$, which of the following is true?

- (a) NP-complete = NP
 (b) $\text{NP-complete} \cap P = \Phi$
 (c) NP-hard = NP
 (d) $P = \text{NP-complete}$

(GATE 2012: 1 mark)

Solution:

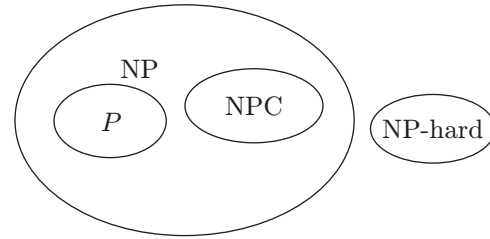


Diagram clearly shows that $\text{NP-complete} \cap P = \emptyset$.

Ans. (b)

81. The worst-case running time to search for an element in a balanced binary search tree with $n2^n$ elements is

- (a) $O(n \log n)$ (b) $O(n2^n)$
 (c) $O(n)$ (d) $O(\log n)$

(GATE 2012: 1 mark)

Solution: The worst-case complexity of a balanced binary search tree is $\Theta(\log n)$, where n is the number of elements; here, n is $n2^n$, so

$$\log(n2^n) = \log n + n \log 2 = \log n + n$$

So, the complexity is $\Theta(n)$.

Ans. (c)

82. The recurrence relation capturing the optimal execution time of the Tower of Hanoi problem with n discs is

- (a) $T(n) = 2T(n-2) + 2$
 (b) $T(n) = 2T(n-1) + n$
 (c) $T(n) = 2T(n/2) + 1$
 (d) $T(n) = 2T(n-1) + 1$

(GATE 2012: 1 mark)

Solution: Recurrence relation for Tower of Hanoi is

$$T(n) = 2T(n-1) + 1$$

Ans. (d)

83. Let G be a simple undirected planar graph on 10 vertices with 15 edges. If G is a connected graph, then the number of bounded faces in any embedding of G on the plane is equal to

(a) 3 (b) 4
(c) 5 (d) 6

(GATE 2012: 1 mark)

Solution: We have

Given that vertices (V) = 10 and edges (E) = 15
Number of bounded and unbounded faces (F) = ?
We know that $V - E + F = 2$

$$10 - 15 + F = 2 \Rightarrow F = 7$$

So, the bounded faces = $F - 1 = 7 - 1 = 6$

Ans. (d)

84. Let $w(n)$ and $A(n)$ denote, respectively, the worst-case and average-case running time of an algorithm executed on an input of size n . Which of the following is *always true*?

(a) $A(n) = \Omega(W(n))$ (b) $A(n) = \Theta(W(n))$
(c) $A(n) = O(W(n))$ (d) $A(n) = o(W(n))$

(GATE 2012: 1 mark)

Solution:

$$A(n) = O(W(n))$$

Ans. (c)

85. Let G be a complete undirected graph on 6 vertices. If vertices of G are labelled, then the number of distinct cycles of length 4 in G is equal to

(a) 15 (b) 30
(c) 90 (d) 360

(GATE 2012: 2 Marks)

Solution: The number of distinct cycles of length 4 is

$${}^6C_4 \times (4-1)! = {}^6C_4 \times 3! = 90$$

Ans. (c)

86. A list of n strings, each of length n , is sorted into lexicographic order using the mergesort algorithm. The worst-case running time of this computation is

(a) $O(n \log n)$ (b) $O(n^2 \log n)$
(c) $O(n^2 + \log n)$ (d) $O(n^2)$

(GATE 2012: 2 Marks)

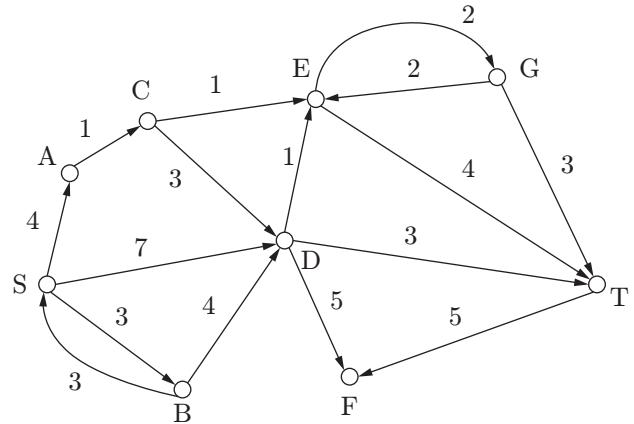
Solution: The worst-case runtime: $n \log n$.

For n strings: $n(n \log n)$, which gives $n^2 \log n$.

Ans. (b)

87. Consider the directed graph shown in the figure below. There are multiple shortest paths between vertices

S and T. Which one will be reported by Dijkstra's shortest path algorithm? Assume that, in any iteration, the shortest path to a vertex v is updated only when a strictly shorter path to v is discovered.



(a) SDT (b) SBDT
(c) SACDT (d) SACET

(GATE 2012: 2 Marks)

Solution: Apply Dijkstra's shortest path algorithm.

S	A	B	C	D	E	G	T
0	-	-	-	-	-	-	-
0	4	3	-	7	-	-	-
0	4	3	5(A)	7	-	-	-
0	4	3	5(A)	7	6(C)	-	-
0	4	3	5(A)	7	6(C)	8(E)	10(E)

So, the minimum path will cover $S \rightarrow A \rightarrow C \rightarrow E \rightarrow T$, with total weight 10.

Ans. (d)

88. Which one of the following is the tightest upper bound that represents the number of swaps required to sort n numbers using selection sort?

(a) $O(\log n)$ (b) $O(n)$
(c) $O(n \log n)$ (d) $O(n^2)$

(GATE 2013: 1 mark)

Solution: The maximum number of swaps required for n elements is n .

Ans. (b)

89. Which one of the following is the tightest upper bound that represents the time complexity of inserting an object into a binary search tree of n nodes?

(a) $O(1)$ (b) $O(\log n)$
(c) $O(n)$ (d) $O(n \log n)$

(GATE 2013: 1 mark)

Solution: In case of skewed binary tree, the tightest upper bound of the insert node is $O(n)$.

Ans. (c)

90. What is the time complexity of Bellman-Ford single-source shortest path algorithm on a complete graph of n vertices?

(A) $\Theta(n^2)$ (B) $\Theta(n^2 \log n)$
(C) $\Theta(n^3)$ (D) $\Theta(n^3 \log n)$

(a)A (b)B (c)C (d)D

(GATE 2013: 1 mark)

Solution: The complexity for Bellman-Ford = $O(E \times V)$. Thus, we have

$$|E| = \frac{n(n-1)}{2} = n^2$$

$$|V| = n$$

$$|E| \times |V| = n^3$$

Ans. (c)

91. Which of the following statements are **TRUE**?

- (1) The problem of determining whether there exists a cycle in an undirected graph is in P.
- (2) The problem of determining whether there exists a cycle in an undirected graph is in NP.
- (3) If a problem A is NP-Complete, there exists a non-deterministic polynomial time algorithm to solve A.

(a) 1, 2 and 3 (b) 1 and 2 only
(c) 2 and 3 only (d) 1 and 3 only

(GATE 2013: 1 mark)

Solution:

- (1) True. Complexity for detecting cycle using DFS is $O(E+V) = O(V^2)$, which is solvable in polynomial time.
- (2) True. Every problem in P is also NP.
- (3) True. NP-Complete problems are solvable in non-deterministic time.

Ans. (a)

92. The line graph $L(G)$ of a simple graph G is defined as follows.

There is exactly one vertex $v(e)$ in $L(G)$ for each edge e in G .

For any two edges e and e' in G , $L(G)$ has an edge between $v(e)$ and $v(e')$, if and only if e and e' are incident with the same vertex in G .

Which of the following statements is/are *true*?

- (P) The line graph of a cycle is a cycle.
- (Q) The line graph of a clique is a clique.
- (R) The line graph of a planar graph is planar.
- (S) The line graph of a tree is a tree.

(a) P only (b) P and R only
(c) R only (d) P, Q and S only

(GATE 2013: 2 Marks)

Solution:

It can be observed by taking different examples for graph with cycle, clique and planar graph and with tree. Then we find that only option (a) is correct.

Ans. (a)

93. The number of elements that can be sorted in $\Theta(\log n)$ time using heap sort is

(A) $\Theta(1)$ (B) $\Theta(\sqrt{\log n})$

(C) $\Theta\left(\frac{\log n}{\log \log n}\right)$ (D) $\Theta(\log n)$

(a) A (b) B (c) C (d) D

(GATE 2013: 2 Marks)

Solution:

Heap sort sorts k elements in time $\Theta(k \log k)$

$$K = \Theta\left(\frac{\log n}{\log \log n}\right)$$

$$\Theta(k \log k) = \Theta\left(\frac{\log n}{\log \log n} \log \frac{\log n}{\log \log n}\right)$$

$$\Rightarrow \Theta\left(\frac{\log n}{\log \log n} (\log \log n - \log \log \log n)\right)$$

$$\Rightarrow \Theta\left(\frac{\log n}{\log \log n} (\log \log n)\right) \Rightarrow \Theta(\log n)$$

Ans. (d)

94. Consider the following function:

```
int unknown(int n) {
    int i, j, k = 0;
    for (i = n/2; i <= n; i++)
        for (j = 2; j <= n; j = j * 2)
            k = k + n/2;
    return k;
}
```

(a) $\Theta(n^2)$ (b) $\Theta(n^{2 \log n})$
(c) $\Theta(n^3)$ (d) $\Theta(n^{3 \log n})$

(GATE 2013: 2 Marks)

Solution:

```
for(i=n/2; i<=n; i++) ---- > n/2, n/2+1...
n => n/2 times
{
    for(j=2; j<=n; j=j*2) -----> 2, 4, 8...
    n => log n times
    {
        k = k + n/2;
    }
    return k;
}
```

Ans. (b)

95. Consider the following operation along with enqueue and dequeue operations on queues, where k is a global parameter:

```
MultiDequeue(Q) {
    m = k
    while (Q is not empty and m > 0) {
        Dequeue(Q)
        m = m - 1
    }
```

What is the worst-case time complexity of a sequence of n MultiDequeue() operations on an initially empty queue?

- (a) $\Theta(n)$ (b) $\Theta(n + k)$
(c) $\Theta(nk)$ (d) $\Theta(n^2)$

(GATE 2013: 2 Marks)

Solution: Any combination of n enqueue and dequeue operations will take $\Theta(n)$.

Ans. (a)

PRACTICE EXERCISES

Set 1

- What will be the height of a d -ary heap having n elements?

(a) $O(\log n \times d)$ (b) $O(d \log n)$
(c) $O(\log n / \log d)$ (d) $O(\log d / \log n)$
- Quick sort gives $O(n \log n)$ worst-case performance if the pivot is selected as:

(a) First element of the array
(b) Median of first, last and middle elements
(c) Arithmetic mean of the first and last elements
(d) None of the above
- A complete full binary tree with 10 leaves

(a) cannot have more than 4 nodes
(b) has exactly 19 nodes
(c) has exactly 17 nodes
(d) cannot have more than 19 nodes
- What is the minimum number of edges to be removed from a complete bipartite graph of 6 nodes $K(6)$, so that graph becomes planar?

(a) 1 (b) 3 (c) 4 (d) 6
- A digraph is represented using adjacency matrix, which is a

(a) Square matrix (b) Symmetric matrix
(c) Asymmetric matrix (d) Unit matrix
- Merge sort uses

(a) divide and conquer approach
(b) heuristic approach
(c) greedy approach
(d) back tracking approach
- If B is a circuit matrix of a graph having K components, e edges and n vertices, the rank of B is:

(a) $e + n + k$ (b) $e - n + k$
(c) $e - n - k$ (d) $e + n - k$
- The number of nodes in a complete binary tree of level 5 is _____.

(a) It does not contain subgraphs homeomorphic to K_5 and $K_{3,3}$.
(b) It does not contain subgraphs isomorphic to K_5 and $K_{3,3}$.
(c) It does contain subgraph isomorphic to K_5 and $K_{3,3}$.
(d) It does contain subgraph homeomorphic to K_5 and $K_{3,3}$.
- To find all pairs of shortest distance in a graph, we can use

(a) dynamic programming approach
(b) bidirectional search approach
(c) merging approach
(d) recursion approach
- Sorting is useful for

(a) report generation
(b) memory storage
(c) compiler
(d) making searching easier and efficient
- A sorting technique that guarantees that records with the same primary key occur in the same order in the sorted list as in the original unsorted list is said to be

(a) internal (b) external
(c) stable (d) inconsistent
- Stack cannot be used to

(a) evaluate an arithmetic expression in postfix form
(b) call different subroutines of a program
(c) allocate resources (like CPU) by the operating system
(d) implement recursion

14. A sorting method that uses a lesser number of swappings is
 (a) quick sort (b) bubble sort
 (c) selection sort (d) radix sort
15. Merge sort uses recursion for
 (a) postorder traversal of a tree
 (b) preorder traversal of a tree
 (c) inorder traversal of a tree
 (d) both b and c
16. A binary search tree
 (a) gives the descending order for pre order traversal
 (b) gives the ascending order for in order traversal
 (c) gives the ascending order for post order traversal
 (d) gives the ascending order for pre order traversal
17. Walk of a graph can be defined as a sequence of nodes and edges in which
 (a) edges are not repeated
 (b) edges may be repeated
 (c) all nodes may be covered
 (d) nodes may be repeated
18. A sorting technique that uses binary tree in which the larger number acts as a root of a subtree to all the numbers is called
 (a) heap sort (b) quick sort
 (c) merge sort (d) bucket sort
19. The reverse polish notation equivalent to the infix expression $((A + B) \times C + D)/(E + F + G)$ is
 (a) $AB + C \times D + EF + G + /$
 (b) $AB + CD \times + EF G +$
 (c) $AB + C \times D + EF + G + /$
 (d) $AB + CD \times + E + FG + /$
20. Big-O estimate for the logarithm of the factorial function, that is, $\log n!$, is given by
 (a) $O(n \log n^2)$ (b) $O(n \log n)$
 (c) $O(\log n!)$ (d) $O(\log \sqrt{n})$
21. If h is a hash function used to hash n keys into a table of size m , where $n < m$, then the expected number of collisions involving a particular key K is
 (a) Less than 1
 (b) Less than $\log n$
 (c) Greater than $\log n$
 (d) Greater than $\log n + 1$
22. In merge sort, what process is performed before merging?
 (a) The array elements form a heap.
 (b) Elements in each half of the array are sorted among themselves.
 (c) Elements in the first half of the array are made equal to elements in second half of the array.
 (d) The arrays are sorted and form a heap.
23. An algorithm is made up of two modules M1 and M2. If time complexities of modules M1 and M2 are $h(n)$ and $g(n)$, respectively, the time complexity of the algorithm is
 (a) $\text{Mean}(h(n), g(n))$ (b) $\max(h(n), g(n))$
 (c) $h(n) + g(n)$ (d) $\min(h(n), g(n))$
24. A vertex cover of an undirected graph $G(V, E)$ is a subset $V_1 \subseteq V$ vertices such that
 (a) Each pair of vertices in V_1 is connected by an edge
 (b) If $(u, v) \in E$, then $u \in V_1$ and $v \in V_1$
 (c) If $(u, v) \in E$, then $u \in V_1$ or $v \in V_1$
 (d) All pairs of vertices in V_1 are not connected by an edge
25. Which one of the following is not an informed search technique?
 (a) Hill climbing search (b) AO* search
 (c) A* search (d) Breadth-first search
26. The map colouring problem can be solved using which of the following techniques?
 (a) Depth-first search
 (b) Constraint satisfaction
 (c) AO* search
 (d) Bidirectional search
27. The value of postfix expression $8\ 3\ 4\ +\ -3\ 8\ 2\ /\ +\ \times\ 2\ 3\ +$ is _____.
28. Which of the following connected simple graphs has exactly one spanning tree?
 (a) Graph with four nodes and five edges
 (b) Graph with five nodes and five edges
 (c) Graph with five nodes and four edges
 (d) None of the above
29. Assume that there are n keys and each key is in the range $[0, m - 1]$. The runtime of bucket sort is
 (a) $O(m - 1)$ (b) $O(m \log n)$
 (c) $O(n \log m)$ (d) $O(n + m)$
30. Consider $f(x) = (x + 1)\log(x^2 + 1) + 3x^2$; its time complexity in big-O notation is
 (a) $O(x)$ (b) $O(x^2)$
 (c) $O(x^3)$ (d) $O(\sqrt{n})$
31. How many edges are there in a forest of t trees containing a total of n vertices?
 (a) $n \times t$ (b) $n - t$
 (c) n/t (d) t/n

- 32.** Which of the following sorting algorithms is stable?
- Quick sort and insertion sort
 - Insertion sort and bubble sort
 - Quick sort and heap sort
 - Quick sort and bubble sort
- 33.** The minimum number of edges in a connected cyclic graph with n vertices is
- $\sqrt{n}$
 - $n - 1$
 - n
 - $n + 1$
- 34.** If there exists at least one path between every pair of vertices in a graph, the graph is called
- Complete graph
 - Hamiltonian graph
 - Connected graph
 - Mesh
- 35.** The depth of a complete binary tree with n nodes is
- $\log(n + 1) - 1$
 - $\log(n) - 1$
 - $\log(n - 1)$
 - $\log(n)$
- 36.** The maximum number of nodes contained by a binary tree of level l is
- $2l$
 - 2^l
 - l^2
 - $\log(l - 1)$
- 37.** $f(n) = O(g(n))$ implies
- $f(n) = O(g(n))$ only
 - $f(n) = \Omega(g(n))$ only
 - $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$
 - $f(n) = \Theta(g(n))$ only
- 38.** Assuming $P \neq NP$, which of the following is true?
- $P = NP$
 - $NP\text{-complete} \cap P = \Phi$
 - $NP\text{-hard} = NP$
 - $NP\text{-complete} = P$
- 39.** Which of the following statements is *true*?
- The problem of determining whether there exists a cycle in an undirected graph is in P .
 - The problem of determining whether there exists a cycle in an undirected graph is in NP .
 - If a problem A is NP -complete, there exists a non-deterministic polynomial time algorithm to solve A .
- 1, 2 and 3
 - 1 and 3
 - 2 and 3
 - 1 and 3
- 40.** Let X is a problem that belongs to the class NP . Which of the following is *true*?
- There is no polynomial time algorithm for X .
 - If X can be solved, then $P = NP$.
 - If X is NP -hard, then it is NP -complete.
 - X may be represented by Bachmann-Landau notations.
- 41.** Which of the following is not the in-place sorting?
- Insertion sort
 - Heap sort
 - Merge sort
 - Selection sort
- 42.** Which of the following does not use the concept of dynamic programming?
- Bellman-Ford algorithm
 - Kruskal's algorithm
 - Dijkstra's algorithm
 - Prim's minimum spanning tree
- 43.** We use dynamic programming approach when
- It can be implemented on 0-1 knapsack problem
 - The solution has optimal substructure
 - The given problem can be reduced to the 3-SAT problem
 - The brute-force algorithm is not applicable
- 44.** Which of the following algorithms is *not* a divide and conquer algorithm by nature?
- Dijkstra's algorithm
 - Heap sort
 - Selection sort
 - Quick sort
- 45.** What is recurrence for worst case of quick sort and what is the time complexity in worst case?
- Recurrence is $T(n) = T(n/2) + O(n/2)$ and time complexity is $O(n^2)$.
 - Recurrence is $T(n) = T(n - 1) + O(n)$ and time complexity is $O(n^2)$.
 - Recurrence is $T(n) = 2T(n/2) + O(\log n)$ and time complexity is $O(n \log n)$.
 - Recurrence is $T(n) = T(n/10) + O(n \times 9/10)$ and time complexity is $O(n^3)$.

Set 2

- Map the following statements regarding quick sort with true (T) or false (F), respectively.
 S1: Quick sort is an in-place sorting algorithm.
 S2: Quick sort is stable.
 S3: Quick sort performs best when numbers are sorted or almost sorted.
 S4: The worst running time of quick sort using randomized partition is $O(n \log n)$.
 S5: It is an example of greedy technique.
 - TFFFF
 - TTFTT
 - TFFTF
 - FTFTF
- What will be the output after the second pass of bubble sort to sort the following number sequence in an ascending order?
 3 5 7 6 4 2
 - 3 5 6 4 2 7
 - 3 5 4 6 2 7
 - 3 5 4 2 6 7
 - 2 3 4 6 5 7

3. The number of spanning trees for a complete graph with seven vertices is

(a) 2^5 (b) 7^5
(c) 3^5 (d) $2^{2 \times 5}$

4. If BFS is run on a complete bipartite graph $K_{q,r}$, $q \geq 1$ and $r \geq 1$, starting with a vertex 'X', then maximum possible height of BFS tree, rooted at 'X', is:

(a) 1 (b) 2
(c) $q + r - 1$ (d) $\text{Max}(q, r)$

Common data questions 5 and 6: Consider the following algorithm

```
Add(k) { // Assume n is a positive integer values
  If (k <= 1)
    Return k;
  Total = 0;
  For i = 1 to k-1
    Total = Total + Add(i);
  Return Total; }
```

5. If $T(k)$ denotes the time complexity of the algorithm, then $T(k)$ is

(a) $\Theta(n^2)$ (b) $\Theta(n!)$
(c) $\Theta(2^n)$ (d) $\Theta(n^3)$

6. If $S(k)$ denotes the space complexity of the algorithm, then $S(k)$ (in terms of $T(k)$) is

(a) $\Theta(\log \log(T(n)))$
(b) $\Theta(\log(T(n)))$
(c) $\Theta(\log(\frac{T(n)}{\log(n)}))$
(d) $\Theta(\sqrt{T(n)})$

7. If A is the best algorithm to check whether a given simple undirected graph $G(V, E)$ is a tree or not, then which of the following best describes the time complexity $T(V, E)$ of A ? (Assume that G is represented as adjacency list.)

(a) $T(V, E) = O(|V|)$
(b) $T(V, E) = O(|V| + |E|)$
(c) $T(V, E) = \Theta(|V|^2)$
(d) $T(V, E) = \Theta \log(|V| + |E|)$

8. Express the following recurrence relation in asymptotic notation: $T(n) = 4T(n/4) + \sqrt{n}$.

(a) $\Theta(\sqrt{n})$ (b) $\Theta(n)$
(c) $\Theta(n^2)$ (d) $\Theta(n^{3/2})$

9. The depth of the binary tree with n nodes is

(a) $\log(n+1) - 1$ (b) $\log n$
(c) $\log(n-1) + 1$ (d) $\log(n) + \log(n-1)$

10. Solve the following given recurrence relation: $T(n) = 2T(n/2) + n^2$.

(a) $\Theta(n^2)$ (b) $\Theta(n^2 \log n)$
(c) $\Theta(n \log_2 n)$ (d) None of the above

11. Given

(i) $\frac{1}{2}n^2 = \omega(n)$
(ii) $\frac{1}{2}n^2 = \omega(n^2)$

Which of the following is true?

(a) I is correct.
(b) II is correct.
(c) Both I and II are correct.
(d) None of them is correct.

12. $f(n) = O(g(n))$ if and only if

(a) $g(n) = O(f(n))$ (b) $g(n) = \omega(f(n))$
(c) $g(n) = \Omega(f(n))$ (d) None

13. What is the generating function $G(z)$ for the sequence of Fibonacci number?

(a) $\frac{z}{z^2 - z - 1}$ (b) $\frac{z}{1 - z - z^2}$
(c) $\frac{z}{1 + z - z^2}$ (d) $\frac{z}{1 + z + z^2}$

14. $\sum_{i=0}^n O(n)$, where $O(n)$ stands for order n is

(a) $O(n)$ (b) $O(n^2)$ (c) $O(n^3)$ (d) $O(3n^2)$

15. In the worst case, the number of comparisons needed to search singly linked list of length n for a given element is

(a) $\log_2 n$ (b) $n/2$ (c) $\log_2 n - 1$ (d) n

16. Consider the following algorithm for searching for a given number x in an unsorted array $A[1..n]$ having n distinct values:

```
1. Choose an  $i$  uniformly at random from  $1..n$ ;
2. If  $A[i] = x$  then Stop else Goto 1;
```

Assuming that x is present on A , what is the expected number of comparisons made by the algorithm before it terminates?

(a) n (b) $n-1$ (c) $2n$ (d) $n/2$

17. The running time of the following algorithm:

```
Procedure A(n)
```

```
If  $n \leq 2$  return (1) else return (A( $\lfloor \sqrt{n} \rfloor$ ))
```

is best described by

(a) $O(n)$ (b) $O(\log n)$
(c) $O(\log \log n)$ (d) $O(1)$

18. Let P be the quick sort program to sort numbers in ascending order. Let t_1 and t_2 be the time taken by the program for the input $[1\ 2\ 3\ 4]$ and $[5\ 4\ 3\ 2\ 1]$, respectively. Which of the following holds?
- (a) $t_1 = t_2$ (b) $t_1 > t_2$
(c) $t_1 < t_2$ (d) $t_1 = t_2 + 5 \log 5$
19. Which of the following algorithm(s) can be used to sort n integers in the range $[1 \dots n^3]$ in $O(n)$ time?
- (a) Heap sort (b) Quick sort
(c) Merge sort (d) Radix sort
20. The recurrence relation that arises in relation with the complexity of binary search is
- (a) $T(n) = T(n/2) + k$, where k is a constant
(b) $T(n) = 2T(n/2) + k$, where k is a constant
(c) $T(n) = T(n/2) + \log n$
(d) $T(n) = T(n/2) + n$
21. Which one of the following statements is false?
- (a) Optimal binary search tree construction can be performed efficiently using dynamic programming.
(b) Breadth-first search cannot be used to find connected component of a graph.
(c) Given the prefix and postfix walks over a binary tree, the binary tree cannot be uniquely constructed.
(d) Depth-first search can be used to find connected components of a graph.
22. The recurrence relation $T(1) = 2$ and $T(n) = 3T(n/4) + n$ has the solution $T(n)$ equal to
- (a) $O(n)$ (b) $O(\log n)$
(c) $O(n^{3/4})$ (d) None of the above
23. Quick sort is run on two inputs shown below to sort in ascending order
- (i) $1, 2, 3, \dots, n$
(ii) $n, n-1, n-2, \dots, 2, 1$
- Let C_1 and C_2 be the number of comparisons made for the inputs (i) and (ii), respectively. Then
- (a) $C_1 < C_2$
(b) $C_1 > C_2$
(c) $C_1 = C_2$
(d) We cannot compare for arbitrary n
24. Let $T(n)$ be the function defined by $T(1) = 1$, $T(n) = 2T(\lfloor n/2 \rfloor) + \sqrt{n}$ for $n \geq 2$. Which of the following statements is true?
- (a) $T(n) = O(\sqrt{n})$ (c) $T(n) = O(\log n)$
(b) $T(n) = O(n)$ (d) None of the above
25. A sorting technique is called stable if
- (a) it takes $O(n \log n)$ time
(b) it maintains the relative order of occurrence of non-distinct elements
(c) it uses divide and conquer paradigm
(d) it takes $O(n)$ space
26. If one uses straight two-way merge sort algorithm to sort the following elements in ascending order: 20, 47, 25, 8, 9, 4, 40, 30, 12, 17, then the order of these elements after second pass of the algorithm is
- (a) 8, 9, 15, 20, 47, 4, 12, 7, 30, 30
(b) 8, 15, 20, 47, 4, 9, 30, 40, 12, 17
(c) 15, 20, 47, 4, 8, 9, 12, 30, 40, 17
(d) 4 8, 9, 15, 20, 47, 12, 17, 30, 40
27. Randomized quick sort is an extension of quick sort where the pivot is chosen randomly. What is the worst-case complexity of sorting n numbers using randomized quick sort?
- (a) $O(n)$ (b) $(n \log n)$
(c) $O(n^2)$ (d) $O(n!)$
28. Kruskal's algorithm for finding a minimum spanning tree of a weighted graph G with n vertices and m edges has the time complexity of
- (a) $O(n^2)$ (b) $O(mn)$
(c) $O(m^2)$ (d) $O(m \log n)$
29. A binary search tree is generated by inserting in order the following integers: 50, 15, 62, 5, 20, 58, 91, 3, 8, 37, 60, 24. The number of nodes in the left subtree and right subtree of the root, respectively, is
- (a) (4, 7) (b) (7, 4)
(c) (8, 3) (d) (3, 8)
30. Which one of the following algorithm design techniques is used in finding all pairs of shortest distances in a graph?
- (a) Dynamic programming (b) Backtracking
(c) Greedy (d) Divide and conquer
31. Give the correct matching for the following pairs:
- | | |
|------------------|-------------------|
| A. $O(\log n)$ | P. Selection |
| B. $O(n)$ | Q. Insertion sort |
| C. $O(n \log n)$ | R. Binary search |
| D. $O(n^2)$ | S. Merge sort |

- (a) A – R; B – P; C – Q; D – S
- (b) A – R; B – P; C – S; D – Q
- (c) A – P; B – R; C – S; D – Q
- (d) A – P; B – S; C – R; D – Q

32. Consider an undirected unweighted graph G . Let a breadth-first traversal of G be done starting from a node r . Let $d(r, u)$ and $d(r, v)$ be the lengths of the shortest paths from r to u and v , respectively, in G . If u is visited before v during the breadth-first traversal, which of the following statements is correct?

- (a) $d(r, u) < d(r, v)$
- (b) $d(r, u) > d(r, v)$
- (c) $d(r, u) \leq d(r, v)$
- (d) None of the above

33. How many undirected graphs (not necessarily connected) can be constructed out of a given set $V = \{V_1, V_2, \dots, V_n\}$ of n vertices?

- (a) $n(n-1)/2$
- (b) 2^n
- (c) $n!$
- (d) $2^{(n(n-1)/2)}$

34. The most appropriate matching for the following pairs is:

X: depth-first search	1: heap
Y: breadth-first search	2: queue
Z: sorting	3: stack

- (a) X– 1; Y– 2; Z– 3
- (b) X– 3; Y– 1; Z– 2
- (c) X– 3; Y– 2; Z– 1
- (d) X– 2; Y– 3; Z– 1

ANSWERS TO PRACTICE EXERCISES

Set 1

- | | | | | | |
|---------|---------|---------|----------|---------|---------|
| 1. (c) | 9. (d) | 17. (b) | 25. (d) | 33. (c) | 41. (c) |
| 2. (d) | 10. (a) | 18. (a) | 26. (b) | 34. (c) | 42. (d) |
| 3. (c) | 11. (d) | 19. (a) | 27. (52) | 35. (a) | 43. (b) |
| 4. (b) | 12. (c) | 20. (b) | 28. (d) | 36. (b) | 44. (b) |
| 5. (c) | 13. (c) | 21. (a) | 29. (d) | 37. (c) | 45. (b) |
| 6. (a) | 14. (c) | 22. (b) | 30. (b) | 28. (b) | |
| 7. (c) | 15. (b) | 23. (b) | 31. (b) | 39. (a) | |
| 8. (63) | 16. (b) | 24. (c) | 32. (c) | 40. (c) | |

Set 2

1. (a) Quick sort is an in-place sorting technique but not stable. It performs worst when numbers are sorted. The worst running time of quick sort using randomized partition is $O(n^2)$. Quick sort is an example of a divide-and-conquer algorithm technique.

2. (c)

Pass 1:	Pass 2:	After second pass:
3 5 7 6 4 2	3 5 6 4 2 7	3 5 4 2 6 7
3 5 7 6 4 2	3 5 6 4 2 7	
3 5 6 7 4 2	3 5 4 6 2 7	
3 5 6 4 2 7	3 5 4 2 6 7	

3. (b) The number of spanning trees for K_n is $n^{n-2} = 7^{7-2} = 7^5$.

4. (b) Let the two sets be Q and R and if $X \in Q$, then the BFS tree contains X in level 0, all the nodes of R in level 1 and remaining nodes of Q other than X in level 2, so maximum height is 2.

5. (c) The recurrence relation $T(n)$ for time complexity of linear(n) is

$$T(k) = c + \sum_{i=1}^{k-1} T(i), \quad \text{if } k > 1 = c$$

= else

$$T(k) = c + \sum_{i=1}^{k-2} T(i) + T(k-1), \quad \text{for } k > 1$$

$$T(k) = T(k-1) + T(k-1) = 2 \times T(k-1)$$

So

$$T(k) = 2^{k-1}c$$

6. (b) Space complexity of $\text{large}(k)$ is proportional to the height of recursion tree which is k . Thus, $S(k) = k = \Theta(\log(T(k)))$, since $T(k) = \Theta(2^k)$.

7. (a) Can be done using DFS in $O(|V|)$ time.

8. (b) By case 1 of the Master Theorem

9. (a) The depth of a binary tree with n nodes is

$$2^{d+1} - 1 = N$$

So, we have

$$2^{d+1} = N + 1$$

Taking log, we get

$$(d + 1)\log 2 = (N + 1)$$

$$d + 1 = \log_2(N + 1)$$

$$d = \log_2(N + 1) - 1$$

10. (a) From the master theorem, here we have $a = 2$, $b = 2$ and $f(n) = n^2$. So

$$g(n) = n^{\log_b a} = n$$

Here, order of $f(n) >$ order of $g(n)$, so we get

$$T(n) = O(f(n)) = O(n^2) \approx \Theta(n^2)$$

11. (a) For ω , there is $f(n) > cg(n)$. So there is no equality. Thus, I is correct and II is false.

12. (c) We have $f(n) = O(g(n))$ which means $f(n) < cg(n)$

It is true only if $g(n) > cf(n)$ which means $g(n) = \Omega(f(n))$

13. (b) The generating function for the Fibonacci number $G(z)$ is

$$G(z) = \frac{z}{1 - z - z^2}$$

14. (b) We have

$$\begin{aligned} \sum_{i=0}^n O(n) &= O(1) + O(2) + O(3) + \cdots + O(n) \\ &= O\left(\frac{n(n+1)}{2}\right) = O(n^2) \end{aligned}$$

15. (d) The worst-case search time in a singly linked list is when the element is in the last position or the element does not exist in the linked list. So it is similar to linear search and requires ' n ' comparisons in the worst case.

16. (a) (Number of comparisons for a case $\times$ probability for the case)

Case 1: If $A[i]$ is found in the first attempt, number of comparisons = 1. Thus, probability of the case = $1/n$.

Case 2: If $A[i]$ is found in the second attempt, number of comparisons = 2. Thus, probability of the case is given as

$$\frac{n-1}{n} \times \frac{1}{n}$$

Case 3: If $A[i]$ is found in the third attempt, number of comparisons = 3. Thus, probability of the case is given as

$$\frac{n-1}{n} \times \frac{n-1}{n} \times \frac{1}{n}$$

There are actually infinite such cases. So, we have the following infinite series:

$$\frac{1}{n} + \frac{n-1}{n} \times \frac{1}{n} \times 2 + f = \frac{n-1}{n} \times \frac{n-1}{n} \times \frac{1}{n} \times 3 + \cdots$$

After solving this series, we will get ' n ' comparisons.

17. (c) Recursive relation for procedure $A(n)$ is

$$T(n) = T(\sqrt{n}) + c1 \quad \text{if } n > 2$$

Use substitution method to solve this.

18. (a) Quick sort takes the worst time if the elements are already in sorted orders (it could be in ascending or descending). As both the inputs are already sorted, it will take the same time in both the cases.

19. (d) Quick sort, merge sort and heap sort running time are $(n \log n)$, whereas radix sort is a non-comparative integer sorting algorithm that sorts data in $O(n)$ time.

20. (a) In binary search, only half of the array where the element exists is searched and the other half is left. So, we get

$$T\left(\frac{n}{2}\right) + K$$

21. (b) Depth-first search and breadth-first search are capable of finding whether a given graph is connected and are also capable of finding connected components of a graph in linear time. So, option (b) is false.

22. (a) Using case 1 of master theorem, we get $n^{\log_4 3} < n$

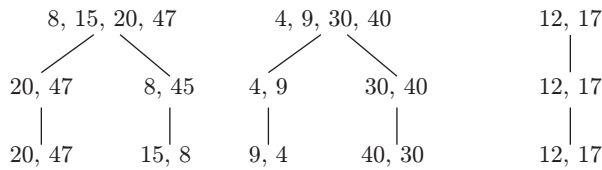
23. (c) Both of the given cases are quick sort worst-case problems, so the number of comparisons will be equal.

24. (b) Use case 1 of master theorem.

25. (b) A sorting algorithm is said to be stable if two objects with equal keys appear in the same order in the sorted output as they appear in the input unsorted array. Some sorting algorithms are stable by nature such as insertion sort, merge sort, bubble sort, etc. And some sorting algorithms are not, such as heap sort, quick sort, etc.

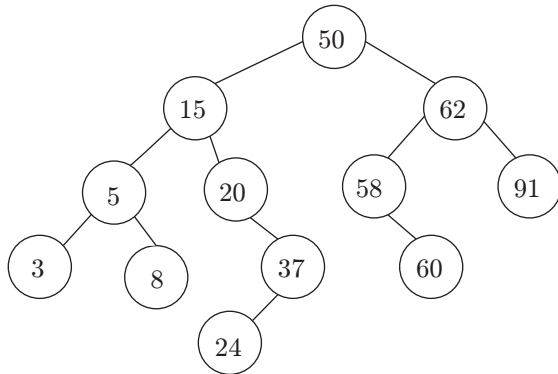
26. (b) Given: 20, 47, 15, 8, 9, 4, 40, 30, 12, 17.

Here, two-way merge sort is used, so a group of two is taken at once. The second pass is shown as follows:



After second pass of the algorithm element, the order is 8, 15, 20, 47, 4, 9, 30, 40, 12, 17.

27. (c) Randomized quick sort worst-case complexity is $O(n^2)$; when all the elements are same, it took worst time.
28. (d)
29. (b) Look at the following figure:



The number of nodes in the left subtree and the right subtree of the root, respectively, is (7, 4). Therefore, option (b) is correct.

30. (a) Bellman-Ford algorithm is used to find all-pair shortest distances in a graph, and it is a dynamic programming technique.
31. (b) Merge sort took $O(n \log n)$ to sort a list of element. Binary search time complexity is $O(\log n)$. Insertion sort worst-case running time complexity is $O(n^2)$.
32. (c) $d(r, u)$ and $d(r, v)$ will be equal when u and v are at same level; otherwise, $d(r, u)$ will be less than $d(r, v)$.
33. (d) In an undirected graph, if it is a complete graph, then it has $n(n-1)/2$ edges. We can choose to have (or not have) any of the $n(n-1)/2$ edges. So, the total number of undirected graphs with n vertices is $2^{(n(n-1)/2)}$.
34. (c) Depth-first search uses stack for storing values; queue is used by breadth-first search. Heap is used for sorting the set of elements.